

DSA Notes

1. Introduction to Algorithm Analysis

Definition

An algorithm is a finite, step-by-step procedure to solve a problem. Algorithm analysis tells us how efficient this procedure is, in terms of:

1. **Time complexity** – how much time (steps) an algorithm takes as input size grows.
2. **Space complexity** – how much memory an algorithm uses as input size grows.

Why analysis is necessary

Two programs may perform the same task but differ drastically in performance:

Example: Sorting 10 million records

- Bubble sort may take hours
- Merge sort takes seconds
- Quick sort may be fastest on average

We analyze algorithms to:

- Predict performance before implementation
- Compare solutions
- Choose optimal algorithm/data structure

Real-world analogy

Consider two delivery services:

Company A: Delivers parcels one by one on a bicycle

Company B: Uses a truck and groups parcels efficiently

Both finish the job, but as number of parcels grows:

- Bicycle becomes extremely slow
- Truck scales better

Algorithms behave similarly. Efficiency matters when input grows.

2. Time Complexity (Big-O Notation)

Big-O expresses the upper bound of running time as input size **n** increases.

Why we ignore constants

Big-O describes growth rate, not exact time.

If an algorithm does:

$$T(n) = 5n + 20$$

$$\text{Big-O} = O(n)$$

Because for large n , constant terms do not matter.

Most common time complexities

Ordered from fastest to slowest:

Complexity Name	Example
$O(1)$	Constant Accessing array index
$O(\log n)$	Logarithmic Binary search
$O(n)$	Linear Loop through array
$O(n \log n)$	Linearithmic Merge sort, heap sort
$O(n^2)$	Quadratic Nested loops (i, j)
$O(n^3)$	Cubic Triple nested loops
$O(2^n)$	Exponential Subset generation
$O(n!)$	Factorial Travelling salesman brute force

ASCII growth visualization

Time

|
| $n!$
| /
| /
| 2^n
| /
| n^3
| n^2
| /

$| n \log n$
 $| /$
 $| n$
 $| \backslash$
 $| \log n$
 $| \backslash$
 $| O(1)$
 +----- Input size n

3. Space Complexity

Space complexity measures the **additional** memory an algorithm uses.

Example:

- Storing an array of size $n \rightarrow O(n)$
- Using constant number of variables $\rightarrow O(1)$
- Using recursion $\rightarrow$ additional stack frames

Space complexity components

1. **Input space**
2. **Auxiliary space**
3. **Output space**

Interviewers focus mainly on **auxiliary space**.

4. Step-by-Step Analysis Examples

Example 1: Single loop

```
for (int i = 0; i < n; i++) {
    System.out.println(i);
}
```

Time: $O(n)$

Space: $O(1)$

Example 2: Nested loops

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        // constant work  
    }  
}
```

Time: $O(n^2)$

Space: $O(1)$

Example 3: Binary search

```
int binarySearch(int[] arr, int target) {  
    int left = 0, right = arr.length - 1;  
    while (left <= right) {  
        int mid = left + (right - left)/2;  
        if (arr[mid] == target) return mid;  
        else if (arr[mid] < target) left = mid + 1;  
        else right = mid - 1;  
    }  
    return -1;  
}
```

Time: $O(\log n)$

Space: $O(1)$

5. Recursion

Definition

A function calling itself with a smaller subproblem.

Real-world analogy

A to-do list:

1. If list is empty $\rightarrow$ task done
2. Otherwise $\rightarrow$ pick one task, solve it, and repeat with remaining tasks

Each recursive call reduces the problem size.

Structure of a recursive function

1. **Base case** – stop condition
2. **Recursive case** – work + smaller call

Example: Factorial

```
int fact(int n) {  
    if (n == 0) return 1; // base case  
    return n * fact(n - 1); // recursive case  
}
```

Execution tree for n = 4

```
fact(4)  
|  
|--> 4 * fact(3)  
    |  
    |--> 3 * fact(2)  
        |  
        |--> 2 * fact(1)  
            |  
            |--> 1 * fact(0)  
                |  
                --> return 1
```

Recursion stack growth

Each call adds a frame to stack memory.

Space complexity = $O(n)$ for n recursive calls.

6. Tail Recursion

Definition

A recursion where recursive call is the last operation.

This allows compiler to optimize (tail call optimization), but Java does not optimize tail recursion.

Example of tail recursion:

```
int sum(int n, int accumulator) {  
    if (n == 0) return accumulator;  
    return sum(n - 1, accumulator + n);  
}
```

Example of non-tail recursion:

```
return n + sum(n - 1);
```

Because addition happens after recursion returns.

7. Master Theorem (Divide and Conquer Analysis)

Master theorem helps analyze recurrences of form:

$$T(n) = aT(n/b) + f(n)$$

Where:

- a = number of subproblems
- b = size reduction
- $f(n)$ = additional work outside recursion

Three Cases

Case 1:

If $f(n) = O(n^k)$ where $k < \log_b a$

Then $T(n) = O(n^{\log_b a})$

Case 2:

If $f(n) = \Theta(n^{\log_b a} \log^p n)$, $p \geq 0$

Then $T(n) = O(n^{\log_b a} \log^{p+1} n)$

Case 3:

If $f(n) = \Omega(n^k)$, $k > \log_b a$, and the regularity condition holds

Then $T(n) = O(f(n))$

Example: Merge Sort

Recurrence:

$$T(n) = 2T(n/2) + O(n)$$

Here:

- $a = 2$
- $b = 2$
- $f(n) = O(n)$

Compute $\log_b a = \log_2 2 = 1$

$$f(n) = n^1$$

Matches case 2

Thus $T(n) = O(n \log n)$

8. Common Mistakes Students Make

1. Forgetting base case or having incorrect base case
 2. Not reducing the problem size (infinite recursion)
 3. Assuming recursion is always slower
 4. Ignoring recursion stack space
 5. Thinking $O(\log n)$ is faster than $O(1)$ (only usually true, not always)
 6. Misinterpreting nested loops as always $O(n^2)$
 7. Confusing worst case with average case
 8. Assuming hash maps are always $O(1)$
 9. Ignoring constant factors when they matter in practice
-

9. Interview Insights

1. Always explain algorithm with Big-O notation
2. Draw recursion trees for recursive solutions
3. Interviewers test clarity, not code memorization
4. Mention trade-offs between time and space
5. Common interview trick: ask candidate to convert recursion to iteration

6. Always check base case, boundaries, and off-by-one errors
 7. Memory constraints are as important as time constraints
-

10. Exercises (with Java Skeletons)

1. Fibonacci with memoization

```
int fib(int n, int[] memo) { ... }
```

2. Reverse a number using recursion

```
int reverse(int n) { ... }
```

3. Count digits using recursion

4. Find power(a, b) using divide and conquer

5. Recursively check if an array is sorted

I can provide full solutions on request.

1. Introduction to Sorting

Definition

Sorting means arranging elements in a specific order, usually **ascending** or **descending**.

Why sorting matters

Sorting is the foundation of many advanced algorithms:

- Binary search requires sorted data
- Greedy algorithms depend on sorted sequences
- Efficient duplicate checking becomes easier
- Many interview problems reduce to sorting first

Real-world analogy

Sorting is like organizing books on a shelf alphabetically. Once sorted, searching for a book becomes very fast.

2. Classification of Sorting Algorithms

Sorting algorithms are broadly classified into:

Based on Time Complexity

1. **Quadratic:** $O(n^2)$
Bubble, Selection, Insertion
2. **Linearithmic:** $O(n \log n)$
Merge, Quick, Heap
3. **Linear:** $O(n)$
Counting, Bucket, Radix

Based on stability

Stable sorting preserves the relative order of equal elements.

Examples: Merge, Insertion, Bubble

Not stable: Quick, Selection, Heap

Based on in-place or not

In-place: uses $O(1)$ or $O(\log n)$ space

Example: Quick sort, Insertion sort

Not in-place: uses extra space

Example: Merge sort (uses $O(n)$ extra memory)

3. Bubble Sort

Concept

Repeatedly swap adjacent elements if they are in the wrong order.

Real-world analogy

Imagine bubbles rising in water. Larger ones rise slowly.

Similarly, largest elements "bubble up" to the end.

Algorithm

Repeat for n times: compare $arr[j]$ and $arr[j+1]$, swap if needed.

ASCII diagram

Unsorted array:

[5, 1, 4, 2]

First pass:

[1, 5, 4, 2]

[1, 4, 5, 2]

[1, 4, 2, 5]

Java Code

```
void bubbleSort(int[] arr) {  
    int n = arr.length;  
    for (int i = 0; i < n - 1; i++) {  
        boolean swapped = false;  
        for (int j = 0; j < n - i - 1; j++) {  
            if (arr[j] > arr[j + 1]) {  
                int temp = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = temp;  
                swapped = true;  
            }  
        }  
        if (!swapped) break; // Optimization  
    }  
}
```

Time Complexity

Worst: $O(n^2)$

Best: $O(n)$ (already sorted)

Space: $O(1)$

Advantages

Simple, easy to understand

Stable

Disadvantages

Very slow

Not preferred in real systems

Common mistakes

Forgetting the swapped flag

Not reducing inner loop with $(n - i - 1)$

4. Selection Sort

Concept

Find the minimum element and place it in its correct position.

Real-world analogy

Selecting the smallest card from a pile and placing it at the front.

ASCII diagram

Initial:

[5, 2, 1, 4]

After iteration 1:

[1, 2, 5, 4]

Java Code

```
void selectionSort(int[] arr) {  
    int n = arr.length;  
    for (int i = 0; i < n - 1; i++) {  
        int minIdx = i;  
        for (int j = i + 1; j < n; j++) {  
            if (arr[j] < arr[minIdx]) minIdx = j;  
        }  
        int temp = arr[i];  
        arr[i] = arr[minIdx];  
        arr[minIdx] = temp;  
    }  
}
```

Complexity

Time: $O(n^2)$

Space: $O(1)$

Stable?

No. Swaps break relative order.

Uses

When memory writes are expensive (minimum number of swaps = $n-1$)

5. Insertion Sort

Concept

Insert each element into its correct position in the sorted part.

Real-world analogy

Arranging playing cards in hand.

Diagram

Start: [5, 3, 4, 1]

Take 3 → Insert left → [3, 5, 4, 1]

Take 4 → Insert → [3, 4, 5, 1]

Take 1 → Insert → [1, 3, 4, 5]

Java Code

```
void insertionSort(int[] arr) {  
    for (int i = 1; i < arr.length; i++) {  
        int key = arr[i];  
        int j = i - 1;  
        while (j >= 0 && arr[j] > key) {  
            arr[j + 1] = arr[j];  
            j--;  
        }  
        arr[j + 1] = key;  
    }  
}
```

Complexity

Worst: $O(n^2)$

Best: $O(n)$

Space: $O(1)$

When used

For small datasets

For partially sorted arrays

6. Merge Sort

Concept

Divide array into halves, sort each recursively, merge.

Real-world analogy

Sorting exam papers by dividing into bundles, sorting each, then merging.

Diagram

[5,1,4,2]

/ \

[5,1] [4,2]

/\ /\

5 1 4 2

Java Code

```
void mergeSort(int[] arr, int left, int right) {
```

```
    if (left < right) {
```

```
        int mid = left + (right - left) / 2;
```

```
        mergeSort(arr, left, mid);
```

```
        mergeSort(arr, mid + 1, right);
```

```
        merge(arr, left, mid, right);
```

```
    }
```

```
}
```

```
void merge(int[] arr, int left, int mid, int right) {
```

```

int n1 = mid - left + 1;
int n2 = right - mid;

int[] L = new int[n1];
int[] R = new int[n2];

for (int i = 0; i < n1; i++) L[i] = arr[left + i];
for (int i = 0; i < n2; i++) R[i] = arr[mid + 1 + i];

int i = 0, j = 0, k = left;

while (i < n1 && j < n2) {
    if (L[i] <= R[j]) arr[k++] = L[i++];
    else arr[k++] = R[j++];
}

while (i < n1) arr[k++] = L[i++];
while (j < n2) arr[k++] = R[j++];
}

```

Complexity

Time: $O(n \log n)$

Space: $O(n)$

Stable: Yes

Uses

Large data that needs guaranteed time

External sorting

7. Quick Sort

Concept

Select a pivot, partition array into elements < pivot, > pivot, then recursively sort.

Real-world analogy

Choosing someone as a pivot in a line and asking everyone shorter to stand left, taller to stand right.

Partition Diagram

Array: [4, 1, 5, 2, 3]

Pivot = 3

After partition:

[1,2] [3] [4,5]

Java Code

```
void quickSort(int[] arr, int low, int high) {  
    if (low < high) {  
        int pi = partition(arr, low, high);  
        quickSort(arr, low, pi - 1);  
        quickSort(arr, pi + 1, high);  
    }  
}
```

```
int partition(int[] arr, int low, int high) {  
    int pivot = arr[high];  
    int i = low - 1;  
    for (int j = low; j < high; j++) {  
        if (arr[j] < pivot) {  
            i++;  
            int temp = arr[i];  
            arr[i] = arr[j];  
            arr[j] = temp;  
        }  
    }
```

```

    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

```

Complexity

Average: $O(n \log n)$

Worst: $O(n^2)$

Space: $O(\log n)$ recursion

When used

When average performance matters

When in-place sorting is needed

Common mistake

Choosing largest or smallest element as pivot gives worst case.

8. Heap Sort

Concept

Convert array to max-heap, repeatedly extract max and rebuild heap.

Real-world analogy

A priority queue where you repeatedly remove the highest priority item.

Diagram

Max heap of [4,1,3,2]

```

    4
  /  \
 2    3
 /
1

```

Java Code


```
void heapSort(int[] arr) {  
    int n = arr.length;  
  
    for (int i = n / 2 - 1; i >= 0; i--)  
        heapify(arr, n, i);  
  
    for (int i = n - 1; i >= 0; i--) {  
        int temp = arr[0];  
        arr[0] = arr[i];  
        arr[i] = temp;  
        heapify(arr, i, 0);  
    }  
}
```

```
void heapify(int[] arr, int n, int i) {  
    int largest = i;  
    int left = 2 * i + 1;  
    int right = 2 * i + 2;  
  
    if (left < n && arr[left] > arr[largest]) largest = left;  
    if (right < n && arr[right] > arr[largest]) largest = right;  
  
    if (largest != i) {  
        int temp = arr[i];  
        arr[i] = arr[largest];  
        arr[largest] = temp;  
        heapify(arr, n, largest);  
    }  
}
```

```
}
```

Complexity

Time: $O(n \log n)$

Space: $O(1)$

Not stable

Uses

Used in priority queues

Good when memory is minimal

9. Counting Sort

Concept

Count occurrences of each number and compute positions.

Works for

- Small range of integers
- Non-negative integers

Java Code

```
void countingSort(int[] arr, int k) {  
    int[] count = new int[k + 1];  
    for (int num : arr) count[num]++;  
    int index = 0;  
    for (int i = 0; i <= k; i++) {  
        while (count[i]-- > 0) arr[index++] = i;  
    }  
}
```

Complexity

Time: $O(n + k)$

Space: $O(k)$

10. Radix Sort

Sort digits one position at a time using counting sort.

Example

Sort [170, 45, 75, 90]

Process units, tens, hundreds.

Complexity

$O(d * (n + k))$ where d = number of digits

11. Searching Algorithms

11.1 Linear Search

```
int linearSearch(int[] arr, int target) {  
    for (int i = 0; i < arr.length; i++)  
        if (arr[i] == target) return i;  
    return -1;  
}
```

Time: $O(n)$

Space: $O(1)$

Used when data is unsorted.

11.2 Binary Search

Works only on sorted arrays.

Java Code

```
int binarySearch(int[] arr, int target) {  
    int low = 0, high = arr.length - 1;  
    while (low <= high) {  
        int mid = low + (high - low) / 2;  
        if (arr[mid] == target) return mid;  
        if (arr[mid] < target) low = mid + 1;  
        else high = mid - 1;  
    }  
}
```

```

    }
    return -1;
}

```

Complexity

Time: $O(\log n)$

Space: $O(1)$

11.3 Ternary Search

Splits into three parts instead of two.

Used in unimodal functions for optimization.

12. Important Variants of Binary Search (interview focused)

1. First occurrence of a number
2. Last occurrence
3. Find smallest element in rotated array
4. Search in rotated sorted array
5. Square root using binary search
6. Peak element search
7. Find missing number
8. Binary search on answer (very important)

I can generate full solutions for these if you request.

13. Comparison Table of Sorting Algorithms

Algorithm	Best	Avg	Worst	Space	Stable	In-Place
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes

Algorithm	Best	Avg	Worst	Space	Stable	In-Place
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes
Counting	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	Yes	No
Radix	$O(d(n+k))$	$O(d(n+k))$	$O(d(n+k))$	$O(n+k)$	Yes	No

14. Common Mistakes Students Make

1. Forgetting pivot edge cases in quick sort
 2. Using merge sort recursion without thinking about extra memory
 3. Assuming quicksort is always $O(n \log n)$
 4. Forgetting that counting sort is only for limited range
 5. Binary search mistakes:
 - infinite loop
 - mid overflow
 - wrong conditions
 6. Not checking boundary cases in searching
-

15. Interview Insights

1. If asked: Which sorting is best?
Answer: Depends on scenario
 - Merge: stable, guaranteed $O(n \log n)$
 - Quick: fastest average in practice
 - Heap: in-place $O(n \log n)$
2. Binary search is one of the most frequently asked topics
3. You must know how to write quicksort and mergesort from scratch
4. Always explain time and space complexity

5. Interviewers expect discussion, not just code

1. Introduction to Linked Lists

Definition

A linked list is a linear data structure where elements are stored in **nodes**, and each node contains:

1. **Data**
2. **Pointer/reference** to the next node
3. (For doubly linked lists) reference to previous node

Why linked lists exist

Arrays have drawbacks:

- Fixed size
- Expensive insertions
- Expensive deletions
- Shifting elements is costly

Linked lists solve these problems.

Real-world analogy

Train coaches connected through couplers.

Each coach knows only the next coach.

You can attach or detach coaches easily.

Representation

[10 | next] -> [20 | next] -> [30 | next] -> null

Each block is a node.

2. Node Structure in Java

```
class Node {
```

```
    int data;
```

```
    Node next;
```

```
    Node(int data) {
```

```
    this.data = data;

    this.next = null;
}
}
```

You can adapt this class for any data type.

3. Types of Linked Lists

1. **Singly Linked List (SLL)**
2. **Doubly Linked List (DLL)**
3. **Circular Linked List (CLL)**
4. **Circular Doubly Linked List**
5. **Skip Lists** (advanced)
6. **Unrolled Linked Lists** (advanced)

We focus on SLL, DLL, and CLL for DSA interviews.

4. Singly Linked List (SLL)

Structure

Each node points to next node only.

head

|

v

[A] -> [B] -> [C] -> null

Operations to cover

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete head
5. Delete tail

6. Delete at position
7. Search for value
8. Count length
9. Reverse a linked list (iterative + recursive)
10. Middle element
11. Detect loop (Floyd cycle detection)
12. Remove loop
13. Merge two sorted linked lists
14. Intersection of two lists

We will implement each.

5. Insert Operations in SLL

5.1 Insert at Beginning

head = newNode -> oldHead

Java Code

```
Node insertAtHead(Node head, int data) {  
    Node newNode = new Node(data);  
    newNode.next = head;  
    return newNode;  
}
```

5.2 Insert at End

[A] -> [B] -> null

Insert: [C]

Java Code

```
Node insertAtEnd(Node head, int data) {  
    Node newNode = new Node(data);  
    if (head == null) return newNode;
```



```

Node temp = head;
while (temp.next != null) {
    temp = temp.next;
}
temp.next = newNode;
return head;
}

```

5.3 Insert at Position (1-based index)

Position 3:

[A] -> [B] -> [C] -> [D]

Insert X at pos 3:

[A] -> [B] -> [X] -> [C] -> [D]

Java Code

```

Node insertAtPos(Node head, int data, int pos) {
    if (pos == 1) return insertAtHead(head, data);

    Node newNode = new Node(data);
    Node temp = head;
    for (int i = 1; i < pos - 1 && temp != null; i++) {
        temp = temp.next;
    }
    if (temp == null) return head;

    newNode.next = temp.next;
    temp.next = newNode;
}

```

```
    return head;
}
```

6. Delete Operations

6.1 Delete at Head

head = head.next

Java Code

```
Node deleteHead(Node head) {
    if (head == null) return null;
    return head.next;
}
```

6.2 Delete at End

[A] -> [B] -> [C] -> null

Remove C

Java Code

```
Node deleteEnd(Node head) {
    if (head == null || head.next == null) return null;

    Node temp = head;
    while (temp.next.next != null) {
        temp = temp.next;
    }
    temp.next = null;
    return head;
}
```

6.3 Delete at Position

```
Node deleteAtPos(Node head, int pos) {  
    if (pos == 1) return deleteHead(head);  
  
    Node temp = head;  
    for (int i = 1; i < pos - 1 && temp != null; i++)  
        temp = temp.next;  
  
    if (temp == null || temp.next == null)  
        return head;  
  
    temp.next = temp.next.next;  
    return head;  
}
```

7. Searching in Linked List

```
boolean search(Node head, int key) {  
    Node temp = head;  
    while (temp != null) {  
        if (temp.data == key) return true;  
        temp = temp.next;  
    }  
    return false;  
}
```

8. Length of Linked List

```
int length(Node head) {  
    int count = 0;
```

```
while (head != null) {  
    count++;  
    head = head.next;  
}  
return count;  
}
```

9. Reverse a Linked List

9.1 Iterative Method

Diagram

Initial: 1 -> 2 -> 3 -> 4 -> null

prev=null, curr=1

Step:

1 -> null

prev=1, curr=2

2 -> 1 -> null

prev=2, curr=3

Java Code

```
Node reverse(Node head) {  
    Node prev = null;  
    Node curr = head;  
  
    while (curr != null) {  
        Node next = curr.next;  
        curr.next = prev;  
        prev = curr;
```

```
    curr = next;
}
return prev;
}
```

9.2 Recursive Method

```
Node reverseRec(Node head) {
    if (head == null || head.next == null)
        return head;

    Node newHead = reverseRec(head.next);
    head.next.next = head;
    head.next = null;

    return newHead;
}
```

10. Find Middle of Linked List (Floyd)

Logic

Use two pointers:

- Slow moves 1 step
- Fast moves 2 steps

When fast reaches end, slow is at middle.

Java Code

```
Node middle(Node head) {
    if (head == null) return null;

    Node slow = head;
```

```

Node fast = head;

while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
}
return slow;
}

```

11. Detect Loop in Linked List (Floyd Cycle Detection)

ASCII diagram

```

+-----+
|       |
A -> B -> C -> D -> E --

```

Java Code

```

boolean hasCycle(Node head) {
    Node slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}

```

12. Remove Loop in Linked List

Logic

Detect meeting point using Floyd.
Then reset slow to head.
Move slow and fast one step at a time.
The point before they meet is where loop ends.

13. Merge Two Sorted Linked Lists

Java Code

```
Node merge(Node a, Node b) {  
    if (a == null) return b;  
    if (b == null) return a;  
  
    if (a.data <= b.data) {  
        a.next = merge(a.next, b);  
        return a;  
    } else {  
        b.next = merge(a, b.next);  
        return b;  
    }  
}
```

14. Intersection Point of Two Linked Lists

Key concept

The intersection is based on reference, not value.

Optimized Approach

Use two pointers:

- p1 starts at head1
- p2 starts at head2
- When p1 reaches null → redirect to head2
- When p2 reaches null → redirect to head1

Eventually, they meet at intersection.

Java Code

```
Node getIntersection(Node headA, Node headB) {  
    Node p1 = headA, p2 = headB;  
  
    while (p1 != p2) {  
        p1 = (p1 == null) ? headB : p1.next;  
        p2 = (p2 == null) ? headA : p2.next;  
    }  
    return p1;  
}
```

15. Doubly Linked List (DLL)

Structure

null <- [A] <-> [B] <-> [C] -> null

Node structure

```
class Node {  
    int data;  
    Node prev;  
    Node next;  
  
    Node(int data) {  
        this.data = data;  
    }  
}
```

Advantages

Insert/delete at both ends efficient
Backward traversal possible

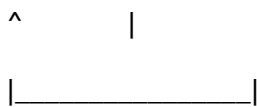
Disadvantages

Extra memory for prev pointer
More complex code

16. Circular Linked List (CLL)

Structure

[A] -> [B] -> [C]



Use cases

Round-robin scheduling
Audio/video playlist
Implementation of queues

17. Time Complexities of Linked List Operations

Operation	SLL	DLL	CLL
Insert head	$O(1)$	$O(1)$	$O(1)$
Insert tail	$O(n)$	$O(1)$	$O(1)$
Delete head	$O(1)$	$O(1)$	$O(1)$
Delete tail	$O(n)$	$O(1)$	$O(n)$
Search	$O(n)$	$O(n)$	$O(n)$

18. Common Mistakes Students Make

1. Forgetting to update tail in CLL
2. Forgetting prev pointers in DLL
3. Incorrect handling of head=null cases
4. Reversing linked list and losing references
5. Infinite loops in cyclic lists

6. Writing recursive reverse without a base case
 7. Confusing intersection by value vs reference
-

19. Interview Insights

1. Reverse a linked list is one of the top 3 most asked linked list problems
2. Detect loop using Floyd is almost mandatory for interviews
3. Merge sorted lists tests recursion skills
4. Intersection problem checks pointer understanding
5. Always talk about why linked lists allow $O(1)$ insert/delete
6. Interviewers expect you to draw diagrams
7. They check boundary case handling (head null, one node, etc.)

SECTION A: STACKS

1. Introduction to Stacks

Definition

A stack is a linear data structure that follows **LIFO** (Last In, First Out).

Real-world analogy

- Stack of plates
- Books piled on top of each other
- Browser history (Back button)

You can only:

- Push: add plate on top
- Pop: remove the plate on top
- Peek: see the top plate

Diagram

Top

|

[9] <-- last pushed

[7]

[3]

[1] <-- bottom

2. Operations on Stack

1. **push(x)** - insert element
 2. **pop()** - remove top element
 3. **peek()** - return top element
 4. **isEmpty()**
 5. **size()**
-

3. Stack Implementation Methods

1. Using arrays (static)
2. Using ArrayList (dynamic resizing)
3. Using Linked List (no size limit)
4. Using Java Stack class (not recommended—legacy)
5. Using Deque (recommended for production)

We will implement array and linked list versions (important for interviews).

4. Stack Using Array

Structure

```
class Stack {  
    int top;  
    int[] arr;  
}
```

Java Code

```
class Stack {  
    private int[] arr;
```

```
private int top;
```

```
private int capacity;
```

```
Stack(int size) {
```

```
    arr = new int[size];
```

```
    capacity = size;
```

```
    top = -1;
```

```
}
```

```
void push(int x) {
```

```
    if (top == capacity - 1)
```

```
        throw new RuntimeException("Stack Overflow");
```

```
    arr[++top] = x;
```

```
}
```

```
int pop() {
```

```
    if (top == -1)
```

```
        throw new RuntimeException("Stack Underflow");
```

```
    return arr[top--];
```

```
}
```

```
int peek() {
```

```
    if (top == -1)
```

```
        throw new RuntimeException("Stack Empty");
```

```
    return arr[top];
```

```
}
```

```
boolean isEmpty() {
```

```
        return top == -1;
    }
}
```

5. Stack Using Linked List

Structure

Top -> [data|next] -> [data|next] -> null

Java Code

```
class StackLL {
    private Node head;
    private int size;

    private static class Node {
        int data;
        Node next;
        Node(int d) { data = d; }
    }

    void push(int x) {
        Node newNode = new Node(x);
        newNode.next = head;
        head = newNode;
        size++;
    }

    int pop() {
        if (head == null)
            throw new RuntimeException("Stack Underflow");
    }
}
```

```

    int val = head.data;

    head = head.next;

    size--;

    return val;
}

int peek() {
    if (head == null)
        throw new RuntimeException("Stack Empty");

    return head.data;
}

boolean isEmpty() {
    return head == null;
}
}

```

6. Time and Space Complexity of Stack

Operation Time Space

Push	O(1)	O(1)
Pop	O(1)	O(1)
Peek	O(1)	O(1)

7. Applications of Stack

1. Expression evaluation
2. Backtracking
3. Undo operations in editors

4. Function call stack
5. DFS traversal
6. Problems like:
 - Balanced parentheses
 - Stock span
 - Next greater element
 - Reverse string
 - Infix to postfix

8. Important Stack Problems

8.1 Reverse a String Using Stack

```
String reverse(String s) {  
    Stack<Character> st = new Stack<>();  
    for (char c : s.toCharArray()) st.push(c);  
  
    StringBuilder sb = new StringBuilder();  
    while (!st.isEmpty()) sb.append(st.pop());  
    return sb.toString();  
}
```

8.2 Balanced Parentheses

Input:

{[()]}

Java Code

```
boolean isValid(String s) {  
    Stack<Character> st = new Stack<>();  
    for (char c : s.toCharArray()) {
```

```
if (c=='(' || c=='{' || c=='[') st.push(c);
else {
    if (st.isEmpty()) return false;
    char top = st.pop();
    if ((c==')' && top!='(') ||
        (c=='}' && top!='{') ||
        (c==']' && top!='['))
        return false;
}
}
return st.isEmpty();
}
```

8.3 Next Greater Element

Input: [4, 5, 2, 25]

Output: [5, 25, 25, -1]

Use a stack to track decreasing elements.

8.4 Infix to Postfix Conversion

Expression:

A + B * C

Postfix:

ABC*+

9. Common Mistakes in Stacks

1. Using top incorrectly in array
2. Not checking underflow
3. Using Java Stack class (slow, legacy)

4. Mixing char and string comparisons
 5. Forgetting to push previous operators in infix-postfix
-

10. Interview Insights: Stack

1. Balanced parentheses is almost guaranteed in interviews
 2. Next greater element tests stack mastery
 3. Infix to postfix checks operator precedence understanding
 4. Candidates often implement stacks using arrays incorrectly
 5. DFS stack vs recursion stack questions appear often
 6. LinkedList-based stack expected for memory efficiency tasks
-

SECTION B: QUEUES

1. Introduction to Queue

Definition

Queue is a linear data structure following **FIFO** (First In, First Out).

Real-world analogy

- Ticket counter queue
- Printer queue
- Task scheduling

Diagram

Front Rear

| |

[10] -> [20] -> [30] -> [40]

2. Operations

1. **enqueue(x)** – add element at rear
2. **dequeue()** – remove element from front

3. **peek()** – return front element
 4. **isEmpty()**
 5. **isFull()** (array implementation)
-

3. Queue Using Array

Java Code

```
class Queue {  
    private int[] arr;  
    private int front, rear, size, capacity;  
  
    Queue(int capacity) {  
        this.capacity = capacity;  
        arr = new int[capacity];  
        front = 0;  
        rear = -1;  
        size = 0;  
    }  
  
    void enqueue(int x) {  
        if (size == capacity)  
            throw new RuntimeException("Queue Full");  
        rear = (rear + 1) % capacity;  
        arr[rear] = x;  
        size++;  
    }  
  
    int dequeue() {  
        if (size == 0)
```

```

        throw new RuntimeException("Queue Empty");

        int val = arr[front];

        front = (front + 1) % capacity;

        size--;

        return val;
    }

    int peek() {
        if (size == 0)
            throw new RuntimeException("Queue Empty");

        return arr[front];
    }
}

```

This is a circular queue implementation.

4. Queue Using Linked List

Diagram

Front -> [A] -> [B] -> [C] -> null <- Rear

Java code

```

class QueueLL {
    private Node front, rear;

    private static class Node {
        int data;

        Node next;

        Node(int d) { data = d; }
    }
}

```

```

void enqueue(int x) {
    Node newNode = new Node(x);
    if (rear == null) {
        front = rear = newNode;
        return;
    }
    rear.next = newNode;
    rear = newNode;
}

int dequeue() {
    if (front == null)
        throw new RuntimeException("Queue Empty");
    int val = front.data;
    front = front.next;
    if (front == null) rear = null;
    return val;
}

int peek() {
    if (front == null)
        throw new RuntimeException("Queue Empty");
    return front.data;
}
}

```

5. Types of Queues

1. Simple Queue

2. Circular Queue
 3. Doubly Ended Queue (Deque)
 4. Priority Queue
 5. Double-ended priority queue
 6. Blocking queue (OS level)
-

6. Circular Queue

Why needed?

Normal array queue wastes space after dequeue.

Circular queue reuses space efficiently.

Diagram

Indexes: 0 1 2 3 4

Queue: [10][20][30][40][50]

front=3, rear=2

The rear wraps around using modulo arithmetic.

7. Deque (Double-Ended Queue)

Insert/delete from **both ends**.

Applications

Sliding window

Palindrome checking

Implementing Monotonic queue

In Java, use:

```
Deque<Integer> dq = new ArrayDeque<>();
```

8. Priority Queue

Concept

Highest priority element is removed first.

Implemented internally using **binary heap**.

Java Code

```
PriorityQueue<Integer> pq = new PriorityQueue<>(); // min-heap  
PriorityQueue<Integer> max = new PriorityQueue<>(Collections.reverseOrder());
```

9. Queue Using Two Stacks

Concept

Make enqueue expensive or dequeue expensive.

Java Code

```
class QueueUsingStacks {  
    Stack<Integer> s1 = new Stack<>();  
    Stack<Integer> s2 = new Stack<>();  
  
    void enqueue(int x) {  
        s1.push(x);  
    }  
  
    int dequeue() {  
        if (s2.isEmpty()) {  
            while (!s1.isEmpty()) s2.push(s1.pop());  
        }  
        if (s2.isEmpty())  
            throw new RuntimeException("Empty Queue");  
        return s2.pop();  
    }  
}
```

10. Stack Using Queues

Java Code (push efficient)

```
class StackUsingQueues {  
    Queue<Integer> q = new LinkedList<>();  
    Queue<Integer> temp = new LinkedList<>();  
  
    void push(int x) {  
        temp.add(x);  
        while (!q.isEmpty())  
            temp.add(q.remove());  
        Queue<Integer> swap = q;  
        q = temp;  
        temp = swap;  
    }  
  
    int pop() {  
        if (q.isEmpty())  
            throw new RuntimeException("Empty Stack");  
        return q.remove();  
    }  
}
```

11. Applications of Queue

1. BFS (Breadth First Search)
 2. CPU Scheduling
 3. Disk Scheduling
 4. Printer Spooler
 5. Call Center Systems
 6. Page Replacement Algorithms (LRU, FIFO)
-

12. Time Complexity of Queue

Operation Time

Enqueue $O(1)$

Dequeue $O(1)$

Peek $O(1)$

13. Common Mistakes in Queues

1. Using simple array instead of circular array
 2. Not resetting rear when queue becomes empty
 3. Failing to handle underflow
 4. Confusing size with indexes
 5. Losing reference of front in LinkedList queue
-

14. Interview Insights: Queue

1. Queue is essential for BFS
2. Circular queue often asked in DSA rounds
3. Queue using two stacks is a classic interview problem
4. Deque-based sliding window is important in medium-hard problems
5. Candidates often confuse stack vs queue in recursion/DFS/BFS discussion
6. Implementation correctness is tested more than theory

SECTION A: INTRODUCTION TO TREES

1. What is a Tree?

Definition

A tree is a **non-linear, hierarchical** data structure consisting of nodes connected by edges.

Components

1. **Node** – contains data

2. **Root** – starting node
3. **Parent**
4. **Child**
5. **Leaf** – node with no children
6. **Edge** – connection
7. **Height** – longest path from node to leaf
8. **Depth** – distance of node from root
9. **Subtree** – part of tree starting from a node

ASCII Diagram

A (root)

/ \

B C

/ \ \

D E F

- Height of tree = 2
- Depth of D = 2

2. Why Trees?

Real-world analogies

- Company hierarchy
- File system folders
- Family tree
- Computer directories
- Organizational charts

Why trees are used

- Faster lookup than linked list
- Dynamic structure
- Good for hierarchical relationships

- Form basis of BST, heaps, tries, segment trees
-

3. Tree Representation in Java

Binary tree node structure

```
class Node {  
    int data;  
    Node left, right;  
  
    Node(int data) {  
        this.data = data;  
        this.left = null;  
        this.right = null;  
    }  
}
```

SECTION B: BINARY TREES

4. What is a Binary Tree?

A tree where every node has **at most two children**.

A
/
B C
\

Types of Binary Trees

1. Full Binary Tree

Every node has 0 or 2 children.

2. Complete Binary Tree

All levels filled except last; nodes appear left to right.

3. Perfect Binary Tree

All internal nodes have 2 children and all leaves are at same depth.

4. **Skewed Binary Tree**

All nodes are left or right children.

5. **Balanced Binary Tree**

Height = $O(\log n)$

5. **Tree Traversal**

Traversal = visiting all nodes.

There are two categories:

1. **Depth-First Traversal (DFS)**

- Inorder
- Preorder
- Postorder

2. **Breadth-First Traversal (BFS)**

- Level Order Traversal
-

5.1 **Preorder Traversal (Root, Left, Right)**

Steps

1. Visit root
2. Traverse left subtree
3. Traverse right subtree

Example Tree

```

  A
 / \
B   C
/\
D E
```

Preorder: A B D E C

Java Code

```
void preorder(Node root) {
```

```
    if (root == null) return;

    System.out.print(root.data + " ");

    preorder(root.left);

    preorder(root.right);
}
```

5.2 Inorder Traversal (Left, Root, Right)

In BST, inorder gives sorted order.

Inorder for above tree: D B E A C

Java Code

```
void inorder(Node root) {
    if (root == null) return;

    inorder(root.left);

    System.out.print(root.data + " ");

    inorder(root.right);
}
```

5.3 Postorder Traversal (Left, Right, Root)

Order: D E B C A

Java Code

```
void postorder(Node root) {
    if (root == null) return;

    postorder(root.left);

    postorder(root.right);

    System.out.print(root.data + " ");
}
```

5.4 Level Order Traversal (Breadth-First Search)

Use queue.

Java Code

```
void levelOrder(Node root) {  
    if (root == null) return;  
  
    Queue<Node> q = new LinkedList<>();  
    q.add(root);  
  
    while (!q.isEmpty()) {  
        Node curr = q.poll();  
        System.out.print(curr.data + " ");  
        if (curr.left != null) q.add(curr.left);  
        if (curr.right != null) q.add(curr.right);  
    }  
}
```

6. Height of a Binary Tree

```
int height(Node root) {  
    if (root == null) return -1;  
    return 1 + Math.max(height(root.left), height(root.right));  
}
```

SECTION C: BINARY SEARCH TREE (BST)

7. What is a BST?

Binary Search Tree is a binary tree where:

1. Left subtree < root
2. Right subtree > root

3. Both subtrees are BST

Example

```
      8
     / \
    3   10
   /\  \
  1 6  14
```

Inorder gives sorted output: 1 3 6 8 10 14

8. Insert in BST

Algorithm

1. If tree empty → new node is root
2. If $\text{key} < \text{root}$ → go left
3. Else → go right

Java Code

```
Node insert(Node root, int key) {
    if (root == null) return new Node(key);

    if (key < root.data)
        root.left = insert(root.left, key);
    else if (key > root.data)
        root.right = insert(root.right, key);

    return root;
}
```

9. Search in BST

```
Node search(Node root, int key) {
```

```

    if (root == null || root.data == key) return root;

    if (key < root.data) return search(root.left, key);
    return search(root.right, key);
}

```

Time: $O(h)$

Worst for skewed tree: $O(n)$

Best/avg for balanced BST: $O(\log n)$

10. Minimum & Maximum in BST

```

Node min(Node root) {
    while (root.left != null)
        root = root.left;
    return root;
}

Node max(Node root) {
    while (root.right != null)
        root = root.right;
    return root;
}

```

11. Delete in BST (important)

Three cases:

Case 1: Node is leaf

Just delete.

Case 2: Node has one child

Replace with child.

Case 3: Node has two children

Replace node value with inorder successor (min in right subtree), then delete successor.

Java Code

```
Node delete(Node root, int key) {  
    if (root == null) return null;  
  
    if (key < root.data)  
        root.left = delete(root.left, key);  
    else if (key > root.data)  
        root.right = delete(root.right, key);  
    else {  
        if (root.left == null) return root.right;  
        if (root.right == null) return root.left;  
  
        Node temp = min(root.right);  
        root.data = temp.data;  
        root.right = delete(root.right, temp.data);  
    }  
    return root;  
}
```

12. Why BST Fails (Skewed Trees)

```
1  
 \  
2  
 \  
3  
 \  

```


4

Time: $O(n)$

Space: $O(n)$

Solution

Use self-balancing trees

- AVL Tree
- Red-Black Tree

SECTION D: AVL TREE

13. What is an AVL Tree?

A BST that self-balances using rotations.

Invented by Adelson-Velsky and Landis.

Property

For every node:

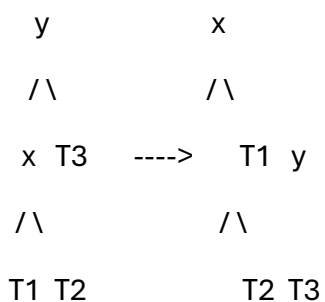
balance factor = $\text{height}(\text{left}) - \text{height}(\text{right})$

Allowed values: -1, 0, +1

If not $\rightarrow$ rotation is performed.

14. Rotations in AVL

1. Right Rotation (LL Case)



2. Left Rotation (RR Case)



$$\begin{array}{ccc}
 T1 & x & \text{---->} & y & T3 \\
 / \backslash & & & & / \backslash \\
 T2 & T3 & & & T1 & T2
 \end{array}$$

3. Left-Right Rotation (LR Case) = Left rotate child → Right rotate parent

4. Right-Left Rotation (RL Case) = Right rotate child → Left rotate parent

15. AVL Insert (complete Java Code)

```

class AVLNode {
    int data, height;
    AVLNode left, right;

    AVLNode(int d) {
        data = d;
        height = 1;
    }
}

class AVLTree {
    int height(AVLNode n) {
        return n == null ? 0 : n.height;
    }

    int getBalance(AVLNode n) {
        return n == null ? 0 : height(n.left) - height(n.right);
    }

    AVLNode rightRotate(AVLNode y) {
        AVLNode x = y.left;
    }
}

```

```
AVLNode T2 = x.right;
```

```
x.right = y;
```

```
y.left = T2;
```

```
y.height = Math.max(height(y.left), height(y.right)) + 1;
```

```
x.height = Math.max(height(x.left), height(x.right)) + 1;
```

```
return x;
```

```
}
```

```
AVLNode leftRotate(AVLNode x) {
```

```
    AVLNode y = x.right;
```

```
    AVLNode T2 = y.left;
```

```
    y.left = x;
```

```
    x.right = T2;
```

```
    x.height = Math.max(height(x.left), height(x.right)) + 1;
```

```
    y.height = Math.max(height(y.left), height(y.right)) + 1;
```

```
    return y;
```

```
}
```

```
AVLNode insert(AVLNode root, int key) {
```

```
    if (root == null) return new AVLNode(key);
```

```
    if (key < root.data)
```

```
        root.left = insert(root.left, key);
```

```
    else if (key > root.data)
        root.right = insert(root.right, key);
    else
        return root;

    root.height = 1 + Math.max(height(root.left), height(root.right));

    int balance = getBalance(root);

    if (balance > 1 && key < root.left.data)
        return rightRotate(root);

    if (balance < -1 && key > root.right.data)
        return leftRotate(root);

    if (balance > 1 && key > root.left.data) {
        root.left = leftRotate(root.left);
        return rightRotate(root);
    }

    if (balance < -1 && key < root.right.data) {
        root.right = rightRotate(root.right);
        return leftRotate(root);
    }
    return root;
}
}
```

16. Time Complexity of AVL Tree

Operation Time

Search $O(\log n)$

Insert $O(\log n)$

Delete $O(\log n)$

SECTION E: RED-BLACK TREE

17. What is a Red-Black Tree?

A Red-Black Tree is a balanced BST with color rules:

1. Each node is red or black
2. Root is always black
3. No two consecutive red nodes
4. Every path from root to leaf has same number of black nodes
5. Null children are black

Purpose

Balances height without as many rotations as AVL.

Performance

Insert/Delete: $O(\log n)$

Very used in real-world systems:

- Java TreeMap
- Java TreeSet
- Linux kernel

Structure Example

10(B)
/ \
5(R) 20(B)

18. Why Red-Black Trees are Preferred in Practice

1. Fewer rotations
 2. Good balance between performance and complexity
 3. Used in real libraries
-

19. BST vs AVL vs Red-Black

Feature	BST	AVL	Red-Black
Height	$O(n)$	$O(\log n)$	$O(\log n)$
Insert	$O(h)$	Slower	Faster
Search	Slow	Fastest	Fast
Balance	No	Strict	Mild
Use case	Rare	Where reads dominate	Where insert/delete dominate

20. Common Mistakes in Tree Problems

1. Confusing height and depth
 2. Incorrect base case in recursion
 3. Returning wrong subtree in delete
 4. Forgetting to update height in AVL
 5. Infinite recursion from incorrect pointer updates
 6. Using inorder for tree without BST property
 7. Mixing level-order and BFS definitions
-

21. Interview Insights

1. Tree traversals are asked extremely often
2. BST deletion is the most frequently failed topic
3. Interviewers test height, diameter, and path sum problems
4. Red-Black Tree theory questions appear in system design

5. Must know AVL rotations and their diagrams
6. Recursion clarity is crucial for tree problems

SECTION A: HEAPS

1. What is a Heap?

A **heap** is a special complete binary tree that satisfies the **heap property**.

Two types:

1. **Min Heap:**
Parent $\leq$ children
Root contains smallest element
2. **Max Heap:**
Parent $\geq$ children
Root contains largest element

Key properties

- **Complete Binary Tree**
All levels except last must be filled, and last level must be filled from left to right.
 - Stored usually in **arrays**, not pointers
 - Allows efficient extraction of min/max
-

2. Array Representation of Heap

For a node at index i :

- Left child = $2*i + 1$
- Right child = $2*i + 2$
- Parent = $(i - 1) / 2$

Example Min Heap Array

Index: 0 1 2 3 4 5

Array: [10, 20, 30, 40, 50, 60]

Tree form:

10

```

    /  \
   20   30
  / \  /
 40 50 60

```

3. Why Use Heap?

Real-world applications

- CPU scheduling
- Dijkstra's algorithm
- Prim's MST
- Priority queues
- Finding k largest/smallest numbers
- Data stream processing

Performance

- Insert: $O(\log n)$
 - Delete root: $O(\log n)$
 - Get min/max: $O(1)$
-

4. Insert into Heap (Min Heap)

Process:

1. Insert element at last
2. Bubble up (heapify up) until parent is smaller

Example

Insert 5 into min heap:

Initial: Insert 5: Bubble up:

```

    10      10      5
   / \    / \    / \
  20 30  20 30  20 10

```


/ \
40 40

Java Code for Insert

```
void insert(int[] heap, int n, int value) {  
    heap[n] = value;  
    int i = n;  
  
    while (i != 0 && heap[(i - 1) / 2] > heap[i]) {  
        int temp = heap[i];  
        heap[i] = heap[(i - 1) / 2];  
        heap[(i - 1) / 2] = temp;  
        i = (i - 1) / 2;  
    }  
}
```

5. Delete Min/Max from Heap

Removing root (min in min heap):

1. Replace root with last element
2. Delete last element
3. Heapify down

Example (Delete Min)

Remove 10 from [10,20,30,40,50]

Replace 10 with 50:

[50,20,30,40]

Heapify down:

Swap 50 with 20:

[20,50,30,40]

Swap 50 with 40:

[20,40,30,50]

Java Code

```
int deleteMin(int[] heap, int n) {  
    int min = heap[0];  
    heap[0] = heap[n - 1];  
    heapify(heap, n - 1, 0);  
    return min;  
}
```

6. Heapify Operation (Most Important)

Purpose:

Fix heap property from index i downward.

Java Code

```
void heapify(int[] heap, int n, int i) {  
    int smallest = i;  
    int left = 2 * i + 1;  
    int right = 2 * i + 2;  
  
    if (left < n && heap[left] < heap[smallest])  
        smallest = left;  
    if (right < n && heap[right] < heap[smallest])  
        smallest = right;
```

```
    if (smallest != i) {  
        int temp = heap[i];  
        heap[i] = heap[smallest];  
        heap[smallest] = temp;  
        heapify(heap, n, smallest);  
    }  
}
```

7. Build Heap from Array (Heap Construction)

Algorithm

Call heapify on all non-leaf nodes from bottom up.

Complexity

$O(n)$

Java Code

```
void buildHeap(int[] arr, int n) {  
    for (int i = n/2 - 1; i >= 0; i--) {  
        heapify(arr, n, i);  
    }  
}
```

8. Heap Sort (Uses Max Heap)

Steps:

1. Build max heap
2. Swap root with last
3. Reduce heap size
4. Heapify
5. Repeat

Complexity

$O(n \log n)$

Java Code: Heap Sort

```
void heapSort(int[] arr) {  
    int n = arr.length;  
  
    for (int i = n/2 - 1; i >= 0; i--)  
        heapifyMax(arr, n, i);  
  
    for (int i = n - 1; i > 0; i--) {  
        int temp = arr[0];  
        arr[0] = arr[i];  
        arr[i] = temp;  
  
        heapifyMax(arr, i, 0);  
    }  
}  
  
void heapifyMax(int[] arr, int n, int i) {  
    int largest = i;  
    int left = 2 * i + 1;  
    int right = 2 * i + 2;  
  
    if (left < n && arr[left] > arr[largest]) largest = left;  
    if (right < n && arr[right] > arr[largest]) largest = right;  
  
    if (largest != i) {  
        int temp = arr[i];
```

```

arr[i] = arr[largest];
arr[largest] = temp;

heapifyMax(arr, n, largest);
}
}

```

9. Time and Space Complexities of Heap

Operation	Time Complexity	Space
Insert	$O(\log n)$	$O(1)$
Delete min/max	$O(\log n)$	$O(1)$
Get min/max	$O(1)$	$O(1)$
Build heap	$O(n)$	$O(1)$
Heap sort	$O(n \log n)$	$O(1)$

10. Applications of Heaps

1. Priority queues
 2. Dijkstra's algorithm
 3. A* algorithm
 4. Prim's MST
 5. Median of a stream
 - Use max-heap and min-heap
 6. Merge k sorted lists
 7. Top k frequent elements
 8. Scheduler, event simulation
-

SECTION B: PRIORITY QUEUE

11. What is a Priority Queue?

A priority queue is an abstract data type where:

- Each element has a **priority**
- Higher priority items are removed first

Implemented using:

- Binary heap (most common)
 - Fibonacci heap
 - Balanced BST
-

12. Java PriorityQueue

Java provides a PriorityQueue class which implements a **min-heap** by default.

Create Min Heap

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
```

Create Max Heap

```
PriorityQueue<Integer> pq = new PriorityQueue<>(Collections.reverseOrder());
```

13. PriorityQueue Operations

13.1 Insert

```
pq.add(10);
```

```
pq.add(5);
```

```
pq.add(20);
```

13.2 Peek (min or max)

```
pq.peek();
```

13.3 Remove

```
pq.poll();
```

13.4 Size

```
pq.size();
```

13.5 Check empty

```
pq.isEmpty();
```

14. PriorityQueue: Internal Working

- Stored as complete binary tree
 - Represented as array
 - heapify up/down on insert/delete
 - Guaranteed $O(\log n)$ insert and delete
-

15. Important Problems Using PriorityQueue

15.1 Kth Largest Element

```
int findKthLargest(int[] arr, int k) {  
    PriorityQueue<Integer> pq = new PriorityQueue<>();  
    for (int num : arr) {  
        pq.add(num);  
        if (pq.size() > k) {  
            pq.poll();  
        }  
    }  
    return pq.peek();  
}
```

15.2 K Largest Elements in Array

Use min-heap of size k.

15.3 Merge K Sorted Lists

Create min-heap storing:

- node value
- list index

Time: $O(n \log k)$

15.4 Top K Frequent Elements

Use HashMap + Max Heap / Min Heap.

15.5 Median of Data Stream

Use two heaps:

- Max heap for left half
 - Min heap for right half
-

16. Max Heap vs Min Heap: Comparison

Criteria	Min Heap	Max Heap
Root	Smallest	Largest
Used for	Dijkstra, Huffman Priority scheduling	
Delete root	Smallest first	Largest first

17. Common Mistakes in Heaps

1. Incorrect parent/child indexing
 2. Forgetting to update heap size
 3. Calling heapify only once
 4. Not choosing correct heap type for problem
 5. Using PriorityQueue incorrectly with custom comparator
 6. Confusing max-heap behavior with min-heap default
-

18. Interview Insights: Heaps

1. Top k problems are extremely common
2. Dijkstra and Prim always use min-heap
3. Heapify method is a common interview coding test
4. Heap sort is frequently asked theoretically
5. Time complexity of building heap = $O(n)$ is a common trick question
6. Many system design topics depend on priority queues

SECTION A: FUNDAMENTALS OF HASHING

1. What is Hashing?

Hashing is a technique used to **map data to a fixed-size table** using a mathematical function called a **hash function**.

Purpose

Fast insertion, deletion, and search in expected **$O(1)$** time.

Real-world analogy

Imagine 1000 people attending an event. Instead of searching each badge sequentially, the organizers assign a desk number using a formula based on their ID. Each person directly goes to their assigned desk.

This eliminates searching time.

2. Hash Function

A hash function $h(\text{key})$ transforms a key into an index of an array (called hash table).

Example:

$h(\text{key}) = \text{key} \% \text{tableSize}$

Good hash function properties

1. Deterministic
 2. Uniform distribution
 3. Efficient to compute
 4. Minimizes collisions
-

3. Hash Table

A hash table is an array that stores values using the hash of the key.

ASCII representation

Index Bucket

0 []

1 [A]

2 []

3 [C -> K -> M]

4 []

5 [B]

Bucket 3 stores multiple values due to collisions.

SECTION B: COLLISIONS

4. What is a Collision?

A collision occurs when **two different keys** produce the **same hash index**.

Example:

$$h(25) = 25 \% 10 = 5$$

$$h(35) = 35 \% 10 = 5$$

Both elements want to be placed at index 5.

Collision handling strategies:

1. Chaining
2. Open Addressing
3. Double Hashing

5. Chaining (Separate Chaining)

Each bucket in the hash table stores a **linked list** of values.

Example

Index: 3

List: [C] -> [K] -> [T]

Advantages

- Simple to implement
- Works well even when load factor is high
- No clustering problems

Disadvantages

- Pointers increase memory
 - Worst-case search = $O(n)$
-

6. Open Addressing

Instead of using linked lists, all keys are placed in the table itself.

Types:

1. **Linear Probing**
 2. **Quadratic Probing**
 3. **Double Hashing**
-

6.1 Linear Probing

If collision occurs at index i , try:

$i, i+1, i+2, i+3, \dots$ until empty slot is found

Example table size = 10

Insert order: 7, 17, 27

$h(7) = 7$

$h(17) = 7 \rightarrow \text{collision} \rightarrow 8$

$h(27) = 7 \rightarrow \text{collision} \rightarrow 8 \rightarrow \text{collision} \rightarrow 9$

Problem

Primary clustering: large blocks of consecutive filled buckets.

6.2 Quadratic Probing

Instead of stepping by 1:

$i + 1^2, i + 2^2, i + 3^2, \dots$

This reduces clustering.

6.3 Double Hashing

Two hash functions are used:

$\text{index} = (h1(\text{key}) + j * h2(\text{key})) \% \text{tableSize}$

This produces best distribution.

SECTION C: LOAD FACTOR & REHASHING

7. Load Factor

Load Factor = (number of elements) / (table size)

Interpretation

- Low load factor → better performance
- High load factor → collisions increase

Typical thresholds

Java HashMap triggers resizing when LF > **0.75**.

8. Rehashing (Resizing)

When load factor exceeds threshold:

1. Table size is doubled
2. All entries are rehashed

Complexity

$O(n)$ for resizing

But happens infrequently → amortized cost is still $O(1)$.

SECTION D: HASHMAP INTERNALS (JAVA)

9. How HashMap Works in Java (Very Important for Interviews)

Java HashMap is implemented using:

1. **Array of buckets**
2. Each bucket is:
 - Linked list (for few collisions)
 - Balanced tree (if too many collisions)

Since Java 8

If a bucket has **more than 8 nodes**, LinkedList becomes a **Red-Black Tree** for faster lookup ($O(\log n)$).

ASCII view:

`hash = key.hashCode() % tableSize`

`table[index]` = either:

- Node (for linked list chain)
- TreeNode (for red-black tree)

10. Steps in HashMap.put(key, value)

1. Compute hash = hash(key)
2. Convert hash to index = hash % tableSize
3. If bucket empty: create new node
4. Else: collision → move through chain/tree
5. If key exists: replace value
6. If chain size > 8: convert to tree
7. If size > load factor threshold: resize

11. Steps in HashMap.get(key)

1. Compute hash
2. Go to index
3. Compare keys in chain/tree
4. If found → return value
5. Else → return null

Time complexity (average): $O(1)$

Worst case: $O(\log n)$ due to red-black tree

12. Custom HashMap Implementation (Interview-Level)

```
class HashNode<K,V> {
```

```
    K key;
```

```
    V value;
```

```
    HashNode<K,V> next;
```

```
    HashNode(K key, V value) {
```

```
        this.key = key;
```

```
        this.value = value;
```

```
    }
```

```
}
```

```
class MyHashMap<K,V> {
```

```
    private final int TABLE_SIZE = 16;
```

```
    private HashNode<K,V>[] table;
```

```
    MyHashMap() {
```

```
        table = new HashNode[TABLE_SIZE];
```

```
    }
```

```
int hash(K key) {  
    return Math.abs(key.hashCode() % TABLE_SIZE);  
}
```

```
void put(K key, V value) {  
    int index = hash(key);  
    HashNode<K,V> head = table[index];
```

```
    while (head != null) {  
        if (head.key.equals(key)) {  
            head.value = value;  
            return;  
        }  
        head = head.next;  
    }
```

```
    HashNode<K,V> newNode = new HashNode<>(key, value);  
    newNode.next = table[index];  
    table[index] = newNode;  
}
```

```
V get(K key) {  
    int index = hash(key);  
    HashNode<K,V> head = table[index];
```

```
    while (head != null) {  
        if (head.key.equals(key))  
            return head.value;
```

```
        head = head.next;
    }
    return null;
}
}
```

SECTION E: IMPORTANT HASHING PROBLEMS (INTERVIEW)

13. Find First Non-Repeating Character

```
char firstNonRepeat(String s) {
    Map<Character,Integer> map = new HashMap<>();
    for (char c : s.toCharArray())
        map.put(c, map.getDefault(c, 0) + 1);

    for (char c : s.toCharArray())
        if (map.get(c) == 1) return c;

    return '\0';
}
```

14. Two Sum Problem

```
int[] twoSum(int[] arr, int target) {
    Map<Integer, Integer> map = new HashMap<>();

    for (int i = 0; i < arr.length; i++) {
        int complement = target - arr[i];
        if (map.containsKey(complement))
            return new int[]{map.get(complement), i};
    }
}
```



```
        map.put(arr[i], i);
    }
    return new int[]{-1, -1};
}
```

Time: $O(n)$

15. Longest Subarray with Zero Sum

```
int longestZeroSum(int[] arr) {
    Map<Integer, Integer> map = new HashMap<>();
    int sum = 0, max = 0;

    for (int i = 0; i < arr.length; i++) {
        sum += arr[i];

        if (sum == 0) max = i + 1;

        if (map.containsKey(sum))
            max = Math.max(max, i - map.get(sum));
        else
            map.put(sum, i);
    }
    return max;
}
```

16. Count Distinct Elements in Each Window

Sliding window + hash map.

17. Group Anagrams

Use hash map of string => list.

SECTION F: COMPLEXITY OF HASHING

18. Time Complexity of Hash Table

Operation	Avg	Worst
-----------	-----	-------

Insert	$O(1)$	$O(n)$
--------	--------	--------

Search	$O(1)$	$O(n)$
--------	--------	--------

Delete	$O(1)$	$O(n)$
--------	--------	--------

Worst case happens when all keys land in same bucket.

But in Java 8+, worst-case search becomes **$O(\log n)$** due to treeification.

SECTION G: COMMON MISTAKES

1. Using poor hash function → collisions
 2. Forgetting to override equals() and hashCode()
 3. Modifying key after insertion
 4. Assuming HashMap keeps order (it does not)
 5. Using HashMap instead of LinkedHashMap when order matters
 6. Load factor misunderstandings
 7. Forgetting rehashing cost
-

SECTION H: INTERVIEW INSIGHTS

1. HashMap functionality is extremely important in interviews
2. Expect questions on:
 - Collision handling
 - Resizing
 - Treeifying
 - hashCode() and equals()

3. Many sliding window problems rely on HashMap
4. HashSet internally uses HashMap
5. Custom HashMap implementation is a favourite interview task
6. Difference between HashMap, Hashtable, LinkedHashMap must be known
7. Collisions + load factor + resizing questions appear frequently

SECTION A: GRAPH FUNDAMENTALS

1. What is a Graph?

A **graph** is a non-linear data structure consisting of:

1. **Vertices (nodes)**
2. **Edges connecting vertices**

A graph is represented as **$G = (V, E)$** .

Real-world examples

- Social networks
- Google Maps roads
- Web pages with hyperlinks
- Network routing
- Course prerequisites

2. Types of Graphs

Based on edges

1. **Undirected Graph**
2. $A \text{ --- } B$
3. **Directed Graph (DiGraph)**
4. $A \rightarrow B$

Based on weights

1. Weighted
2. Unweighted

Based on cycles

1. Cyclic
2. Acyclic (DAG – Directed Acyclic Graph)

Based on connectivity

1. Connected
 2. Disconnected
-

3. Representation of Graphs

3.1 Adjacency Matrix

A B C D

A 0 1 1 0

B 1 0 0 1

C 1 0 0 0

D 0 1 0 0

Pros

Fast lookup $O(1)$

Cons

Uses $O(V^2)$ space

3.2 Adjacency List (Preferred)

A -> B, C

B -> A, D

C -> A

D -> B

Pros

Efficient $O(V + E)$ space

Used in most algorithms (BFS, DFS, Dijkstra, Prim)

SECTION B: BFS (Breadth First Search)

4. BFS Definition

Traverses graph **level by level**, uses **queue**.

Real-world analogy

Exploring friends in circles:

- First direct friends
 - Then friends of friends
-

5. BFS Algorithm

1. Create queue
 2. Mark source node visited
 3. While queue not empty:
 - Remove node
 - Visit neighbours
 - Mark and enqueue them
-

6. BFS Java Code

```
void bfs(int start, List<List<Integer>> adj) {  
    boolean[] visited = new boolean[adj.size()];  
    Queue<Integer> q = new LinkedList<>();  
  
    visited[start] = true;  
    q.add(start);  
  
    while (!q.isEmpty()) {  
        int node = q.poll();  
        System.out.print(node + " ");
```

```
for (int neighbor : adj.get(node)) {  
    if (!visited[neighbor]) {  
        visited[neighbor] = true;  
        q.add(neighbor);  
    }  
}  
}
```

7. BFS Time & Space Complexity

Time: $O(V + E)$

Space: $O(V)$

8. BFS Applications

- Shortest path in unweighted graph
 - Cycle detection (undirected)
 - Bipartite check
 - Web crawlers
 - Finding connected components
-

SECTION C: DFS (Depth First Search)

9. DFS Definition

Traverses graph **deep first** using **recursion or stack**.

Real-world analogy

Exploring a maze by going as deep as possible before turning back.

10. DFS Java Code (Recursive)

```
void dfs(int node, boolean[] visited, List<List<Integer>> adj) {  
    visited[node] = true;  
    System.out.print(node + " ");  
  
    for (int neighbor : adj.get(node)) {  
        if (!visited[neighbor]) {  
            dfs(neighbor, visited, adj);  
        }  
    }  
}
```

11. DFS Iterative Using Stack

```
void dfsIterative(int start, List<List<Integer>> adj) {  
    boolean[] visited = new boolean[adj.size()];  
    Stack<Integer> st = new Stack<>();  
  
    st.push(start);  
  
    while (!st.isEmpty()) {  
        int node = st.pop();  
        if (!visited[node]) {  
            visited[node] = true;  
            System.out.print(node + " ");  
        }  
  
        for (int neighbor : adj.get(node)) {  
            if (!visited[neighbor])
```

```
        st.push(neighbor);
    }
}
}
```

12. DFS Time & Space Complexity

Time: $O(V + E)$

Space: $O(V)$

SECTION D: CYCLE DETECTION

13. Cycle Detection in Undirected Graph (DFS)

If during DFS you find an adjacent node already visited and not parent $\rightarrow$ cycle.

Java Code

```
boolean dfsCycle(int node, int parent, boolean[] visited, List<List<Integer>> adj) {
    visited[node] = true;

    for (int neighbor : adj.get(node)) {
        if (!visited[neighbor]) {
            if (dfsCycle(neighbor, node, visited, adj)) return true;
        } else if (neighbor != parent) {
            return true;
        }
    }
    return false;
}
```

14. Cycle Detection in Directed Graph

Use recursion stack.

```
boolean dfsCycle(int node, boolean[] visited, boolean[] stack, List<List<Integer>> adj) {  
    visited[node] = true;  
    stack[node] = true;  
  
    for (int next : adj.get(node)) {  
        if (!visited[next] && dfsCycle(next, visited, stack, adj))  
            return true;  
        else if (stack[next])  
            return true;  
    }  
  
    stack[node] = false;  
    return false;  
}
```

SECTION E: SHORTEST PATH ALGORITHMS

15. Shortest Path in Unweighted Graph (BFS)

```
int[] shortestPath(int src, List<List<Integer>> adj) {  
    int[] dist = new int[adj.size()];  
    Arrays.fill(dist, -1);  
  
    Queue<Integer> q = new LinkedList<>();  
    dist[src] = 0;  
    q.add(src);  
  
    while (!q.isEmpty()) {
```

```

int node = q.poll();

for (int next : adj.get(node)) {
    if (dist[next] == -1) {
        dist[next] = dist[node] + 1;
        q.add(next);
    }
}

return dist;
}

```

16. Dijkstra's Algorithm (Weighted Graph, No Negative Weights)

Uses **Min Heap** (PriorityQueue).

Purpose

Shortest path from single source to all vertices.

17. Dijkstra Java Code

```

class Pair {
    int node, dist;

    Pair(int n, int d) { node = n; dist = d; }
}

int[] dijkstra(int V, List<List<Pair>> adj, int src) {
    PriorityQueue<Pair> pq =
        new PriorityQueue<>((a, b) -> a.dist - b.dist);

    int[] dist = new int[V];
}

```

```

Arrays.fill(dist, Integer.MAX_VALUE);

pq.add(new Pair(src, 0));
dist[src] = 0;

while (!pq.isEmpty()) {
    Pair p = pq.poll();
    int node = p.node;
    int d = p.dist;

    if (d != dist[node]) continue;

    for (Pair it : adj.get(node)) {
        int next = it.node;
        int w = it.dist;

        if (dist[node] + w < dist[next]) {
            dist[next] = dist[node] + w;
            pq.add(new Pair(next, dist[next]));
        }
    }
}

return dist;
}

```

18. Bellman-Ford Algorithm (Handles Negative Weights)

Time: $O(VE)$

Used for

- Graphs with negative edges
- Detecting negative cycles

Algorithm

Relax all edges V-1 times.

19. Bellman-Ford Java Code

```
boolean bellmanFord(int V, int[][] edges, int src, int[] dist) {
```

```
    Arrays.fill(dist, Integer.MAX_VALUE);
```

```
    dist[src] = 0;
```

```
    for (int i = 0; i < V - 1; i++) {
```

```
        for (int[] e : edges) {
```

```
            int u = e[0], v = e[1], w = e[2];
```

```
            if (dist[u] != Integer.MAX_VALUE &&
```

```
                dist[u] + w < dist[v]) {
```

```
                dist[v] = dist[u] + w;
```

```
            }
```

```
        }
```

```
    }
```

```
    for (int[] e : edges) {
```

```
        int u = e[0], v = e[1], w = e[2];
```

```
        if (dist[u] != Integer.MAX_VALUE &&
```

```
            dist[u] + w < dist[v]) {
```

```
            return false;
```

```
        }
```

```
    }
```

```
    return true;
```

```
}
```

20. Floyd-Warshall (All-Pairs Shortest Paths)

Time: $O(V^3)$

for i

for j

for k

Handles negative weights (but not negative cycles).

Used in small graphs.

SECTION F: MINIMUM SPANNING TREE (MST)

21. What is MST?

A spanning tree that covers all vertices with **minimum weight sum**.

Used in

- Network design
 - Clustering
 - Circuit design
-

22. Prim's Algorithm

Start at any node and continuously pick smallest edge connecting used and unused nodes.

Uses Min Heap.

Time Complexity

Adjacency list + Min Heap: $O(E \log V)$

23. Prim's Algorithm Java Code

```
int prims(int V, List<List<Pair>> adj) {  
    boolean[] used = new boolean[V];
```

```

PriorityQueue<Pair> pq =
    new PriorityQueue<>((a, b) -> a.dist - b.dist);

pq.add(new Pair(0, 0));
int cost = 0;

while (!pq.isEmpty()) {
    Pair p = pq.poll();
    int node = p.node;
    int w = p.dist;

    if (used[node]) continue;

    used[node] = true;
    cost += w;

    for (Pair it : adj.get(node)) {
        if (!used[it.node])
            pq.add(new Pair(it.node, it.dist));
    }
}

return cost;
}

```

24. Kruskal's Algorithm

Uses **Disjoint Set Union (Union-Find)**.

Steps:

1. Sort edges by weight

2. Take smallest edge
3. If adding it does not form cycle → include it
4. Continue until V-1 edges are included

Time

$O(E \log E)$

25. Kruskal Java Code

```
class Edge {  
    int u, v, w;  
    Edge(int u, int v, int w) {  
        this.u = u; this.v = v; this.w = w;  
    }  
}  
  
int find(int x, int[] parent) {  
    if (parent[x] == x) return x;  
    return parent[x] = find(parent[x], parent);  
}  
  
void union(int a, int b, int[] parent, int[] rank) {  
    a = find(a, parent);  
    b = find(b, parent);  
  
    if (rank[a] < rank[b]) parent[a] = b;  
    else if (rank[b] < rank[a]) parent[b] = a;  
    else {  
        parent[b] = a;  
        rank[a]++;  
    }  
}
```

```

    }
}

int kruskal(int V, List<Edge> edges) {
    Collections.sort(edges, (a, b) -> a.w - b.w);

    int[] parent = new int[V];
    int[] rank = new int[V];

    for (int i = 0; i < V; i++) parent[i] = i;

    int cost = 0;

    for (Edge e : edges) {
        if (find(e.u, parent) != find(e.v, parent)) {
            cost += e.w;
            union(e.u, e.v, parent, rank);
        }
    }

    return cost;
}

```

SECTION G: MORE IMPORTANT GRAPH TOPICS

26. Topological Sort (DAG Only)

Order of tasks respecting dependencies.

Example:

$A \rightarrow B \rightarrow C$

$A \rightarrow D$

Valid orders:

A B D C

A D B C

Java Code (DFS)

```
void topoDFS(int node, boolean[] visited, Stack<Integer> st,
             List<List<Integer>> adj) {
    visited[node] = true;

    for (int next : adj.get(node)) {
        if (!visited[next])
            topoDFS(next, visited, st, adj);
    }
    st.push(node);
}
```

Java Code (BFS – Kahn's Algorithm)

```
List<Integer> topoSortKahn(int V, List<List<Integer>> adj) {
    int[] indegree = new int[V];

    for (int i = 0; i < V; i++)
        for (int u : adj.get(i))
            indegree[u]++;

    Queue<Integer> q = new LinkedList<>();
    for (int i = 0; i < V; i++)
        if (indegree[i] == 0) q.add(i);
```

```

List<Integer> ans = new ArrayList<>();

while (!q.isEmpty()) {
    int node = q.poll();
    ans.add(node);

    for (int next : adj.get(node)) {
        if (--indegree[next] == 0)
            q.add(next);
    }
}

return ans;
}

```

27. Bipartite Graph Check

Use BFS coloring.

```

boolean isBipartite(int V, List<List<Integer>> adj) {
    int[] color = new int[V];
    Arrays.fill(color, -1);

    for (int start = 0; start < V; start++) {
        if (color[start] == -1) {
            Queue<Integer> q = new LinkedList<>();
            q.add(start);
            color[start] = 0;

            while (!q.isEmpty()) {
                int node = q.poll();
            }
        }
    }
}

```

```

    for (int next : adj.get(node)) {
        if (color[next] == -1) {
            color[next] = color[node] ^ 1;
            q.add(next);
        } else if (color[next] == color[node]) {
            return false;
        }
    }
}
}
}
}
return true;
}

```

28. Connected Components

For disconnected graphs, run DFS/BFS from each unvisited node.

SECTION H: COMPLEXITIES

29. Graph Complexity Summary

Algorithm	Time	Space
BFS	$O(V + E)$	$O(V)$
DFS	$O(V + E)$	$O(V)$
Dijkstra	$O(E \log V)$	$O(V)$
Bellman-Ford	$O(VE)$	$O(V)$
Floyd-Warshall	$O(V^3)$	$O(V^2)$

Algorithm	Time	Space
Prim	$O(E \log V)$	$O(V)$
Kruskal	$O(E \log E)$	$O(V)$
Topological Sort	$O(V + E)$	$O(V)$

SECTION I: COMMON MISTAKES

1. Confusing directed vs undirected edges
 2. Forgetting to mark visited in BFS/DFS
 3. Using adjacency matrix for large sparse graph
 4. Incorrectly detecting cycles
 5. Not using priority queue correctly in Dijkstra
 6. Using Dijkstra on negative weights (wrong)
 7. Incorrect union-find implementation in Kruskal
 8. Not counting indegrees correctly in Kahn's algorithm
-

SECTION J: INTERVIEW INSIGHTS

1. Graph questions are common in FAANG-level interviews
2. BFS/DFS fundamentals must be perfect
3. Dijkstra, Topological Sort, Cycle Detection are frequently asked
4. MST questions test deeper reasoning
5. Two-pass DFS (Kosaraju) for SCC may appear
6. Tricky edge cases:
 - Disconnected graph
 - Multiple edges
 - Self-loops
 - Negative weights

SECTION A: INTRODUCTION TO DYNAMIC PROGRAMMING

1. What is Dynamic Programming?

Dynamic Programming (DP) is a problem-solving technique used when:

1. The problem has **overlapping subproblems**
2. The problem has **optimal substructure**

DP avoids recomputing repeated subproblems by storing results.

Real-world analogy

Imagine you need to calculate monthly electricity bill for 12 months. Instead of recalculating each month's usage from scratch, you store previous calculations and reuse them.

2. Why DP Exists?

Example: Fibonacci

Naive recursion:

$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$$

Time complexity: **$O(2^n)$**

Because it recomputes the same values many times.

DP reduces this to **$O(n)$** .

3. Two Core Techniques in DP

1. Top-Down DP (Memoization)

- Recursion
- Cache results

2. Bottom-Up DP (Tabulation)

- Iterative
- Fill table from base to answer

SECTION B: DP ESSENTIAL TERMINOLOGY

4. Optimal Substructure

A problem exhibits optimal substructure if its optimal solution can be constructed from optimal solutions of its subproblems.

Example: Shortest path in graph, knapsack, LCS.

5. Overlapping Subproblems

A problem has overlapping subproblems when the same subproblem appears multiple times.

Example: Fibonacci, LCS.

6. DP Table Representations

1D DP

Used in: Fibonacci, LIS, Climbing stairs.

2D DP

Used in: LCS, Knapsack, grid problems.

3D DP

Used in: DP with states (i, j, k).

SECTION C: FIBONACCI – GATEWAY TO DP

7. Fibonacci – Pure Recursion

```
int fib(int n) {  
    if (n <= 1) return n;  
    return fib(n-1) + fib(n-2);  
}
```

Time: $O(2^n)$

8. Fibonacci – Top-Down Memoization

```
int fib(int n, int[] dp) {
```

```
if (n <= 1) return n;

if (dp[n] != -1) return dp[n];

return dp[n] = fib(n-1, dp) + fib(n-2, dp);
}
```

9. Fibonacci – Bottom-Up Tabulation

```
int fib(int n) {
    int[] dp = new int[n+1];
    dp[0] = 0;
    dp[1] = 1;

    for (int i = 2; i <= n; i++)
        dp[i] = dp[i-1] + dp[i-2];

    return dp[n];
}
```

10. Fibonacci – Space Optimized

```
int fib(int n) {
    if (n <= 1) return n;

    int prev2 = 0, prev1 = 1;

    for (int i = 2; i <= n; i++) {
        int cur = prev1 + prev2;
        prev2 = prev1;
```

```
    prev1 = cur;
}
return prev1;
}
```

SECTION D: CLASSIC DP PROBLEMS

11. Climbing Stairs (Ways to reach nth stair)

Like Fibonacci.

$\text{ways}(n) = \text{ways}(n-1) + \text{ways}(n-2)$

Java Code (Tabulation)

```
int climbStairs(int n) {
    int[] dp = new int[n+1];
    dp[0] = 1;
    dp[1] = 1;

    for (int i = 2; i <= n; i++)
        dp[i] = dp[i-1] + dp[i-2];

    return dp[n];
}
```

12. House Robber

Cannot rob adjacent houses.

$\text{dp}[i] = \max(\text{dp}[i-1], \text{nums}[i] + \text{dp}[i-2])$

Java Code

```
int rob(int[] nums) {
    int prev2 = 0, prev1 = 0;
```



```

for (int num : nums) {
    int curr = Math.max(prev1, num + prev2);
    prev2 = prev1;
    prev1 = curr;
}
return prev1;
}

```

SECTION E: LONGEST INCREASING SUBSEQUENCE (LIS)

13. LIS – DP with $O(n^2)$

dp[i] = length of LIS ending at i

for j in 0..i-1:

if arr[j] < arr[i]:

dp[i] = max(dp[i], dp[j] + 1)

Java Code

```

int lengthOfLIS(int[] nums) {
    int n = nums.length;
    int[] dp = new int[n];
    Arrays.fill(dp, 1);

    int ans = 1;

    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (nums[j] < nums[i])
                dp[i] = Math.max(dp[i], dp[j] + 1);
        }
    }
    return ans;
}

```

```
    }  
    ans = Math.max(ans, dp[i]);  
}  
return ans;  
}
```

14. LIS – Binary Search $O(n \log n)$

Use tails array maintaining smallest tail for each length.

SECTION F: LONGEST COMMON SUBSEQUENCE (LCS)

15. LCS Definition

Given two strings A and B, find the longest sequence that appears in both (not necessarily contiguous).

Example:

A = "abcde"

B = "ace"

LCS = "ace"

16. LCS DP Formula

if $A[i] == B[j]$:

$$dp[i][j] = 1 + dp[i-1][j-1]$$

else:

$$dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$$

17. LCS Java Code (Tabulation)

```
int lcs(String a, String b) {  
    int n = a.length(), m = b.length();  
    int[][] dp = new int[n+1][m+1];
```

```

for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        if (a.charAt(i-1) == b.charAt(j-1))
            dp[i][j] = 1 + dp[i-1][j-1];
        else
            dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
    }
}
return dp[n][m];
}

```

SECTION G: EDIT DISTANCE (LEVENSHTEIN)

18. Edit Distance Definition

Minimum operations to convert string A into string B:

Operations:

1. Insert
 2. Delete
 3. Replace
-

19. Edit Distance DP Formula

if $A[i] == B[j]$:

$$dp[i][j] = dp[i-1][j-1]$$

else:

$$dp[i][j] = 1 + \min($$

$$dp[i-1][j], \text{ // delete}$$

$$dp[i][j-1], \text{ // insert}$$

```
    dp[i-1][j-1] // replace
)
```

20. Edit Distance Java Code

```
int editDistance(String a, String b) {
    int n = a.length(), m = b.length();
    int[][] dp = new int[n+1][m+1];

    for (int i = 0; i <= n; i++) dp[i][0] = i;
    for (int j = 0; j <= m; j++) dp[0][j] = j;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (a.charAt(i-1) == b.charAt(j-1))
                dp[i][j] = dp[i-1][j-1];
            else
                dp[i][j] = 1 + Math.min(dp[i-1][j],
                    Math.min(dp[i][j-1],
                        dp[i-1][j-1]));
        }
    }
    return dp[n][m];
}
```

SECTION H: 0/1 KNAPSACK

21. Problem Definition

Given weights[] and values[], pick items such that:

- Total weight $\leq$ capacity
 - Maximize total value
 - Each item can be taken 0 or 1 time
-

22. DP Formula

if weight[i] $\leq$ W:

```
dp[i][W] = max(  
    value[i] + dp[i-1][W - weight[i]],  
    dp[i-1][W]  
)
```

else:

```
dp[i][W] = dp[i-1][W]
```

23. 0/1 Knapsack Java Code

```
int knapsack(int[] wt, int[] val, int W) {
```

```
    int n = wt.length;
```

```
    int[][] dp = new int[n+1][W+1];
```

```
    for (int i = 1; i <= n; i++) {
```

```
        for (int w = 1; w <= W; w++) {
```

```
            if (wt[i-1] <= w) {
```

```
                dp[i][w] = Math.max(  
                    val[i-1] + dp[i-1][w-wt[i-1]],  
                    dp[i-1][w]
```

```
                );
```

```
            } else {
```

```
                dp[i][w] = dp[i-1][w];
```

```
            }
```

```
        }
```

```
    }
```

```
    }  
}  
return dp[n][W];  
}
```

SECTION I: DP ON GRIDS

24. Unique Paths (Robot Grid)

$dp[i][j] = dp[i-1][j] + dp[i][j-1]$

Java Code

```
int uniquePaths(int m, int n) {  
    int[][] dp = new int[m][n];  
  
    for (int i=0; i<m; i++) dp[i][0] = 1;  
    for (int j=0; j<n; j++) dp[0][j] = 1;  
  
    for (int i=1; i<m; i++) {  
        for (int j=1; j<n; j++) {  
            dp[i][j] = dp[i-1][j] + dp[i][j-1];  
        }  
    }  
    return dp[m-1][n-1];  
}
```

SECTION J: DP ON TREES

25. Tree DP

Example: Diameter of tree, maximum sum path.

SECTION K: DP STATES & PATTERNS

26. Common DP Patterns

1. 1D DP

- Fibonacci
- Climbing stairs
- LIS

2. 2D DP

- LCS
- Edit distance
- Knapsack

3. DP on grid

- Paths
- Minimum cost path

4. DP on strings

- Palindromic subsequences
- Pattern matching

5. Bitmask DP

- Traveling Salesman Problem

6. Tree DP

- Subtree sums
- Diameter

SECTION L: COMMON MISTAKES

1. Not identifying overlapping subproblems
2. Writing recursive DP without memoization
3. Incorrect base cases

4. Filling DP table in wrong order
 5. Misunderstanding transitions
 6. Not converting recursive logic to iterative
 7. Using incorrect DP dimensions
-

SECTION M: INTERVIEW INSIGHTS

1. DP is tested heavily in FAANG, TCS Digital, and product-based interviews
2. Must know: Fibonacci, LCS, Knapsack, LIS
3. Interviewers often test bottom-up understanding
4. Space optimization is a favourite question
5. DP on grids, DP on strings come frequently
6. Always explain the recurrence before coding
7. Draw tables for 2D DP problems

SECTION A: INTRODUCTION TO GREEDY ALGORITHMS

1. What is a Greedy Algorithm?

A greedy algorithm builds a solution **step by step**, always choosing the **best immediate choice** without reconsidering past decisions.

Key idea

Make a choice → never go back to fix it → assume it leads to optimal solution.

When Greedy works

When the problem satisfies:

1. **Greedy choice property**
A global optimum can be reached by picking the best local choice.
 2. **Optimal substructure**
Optimal solution to problem contains optimal solution of subproblems.
-

2. When Greedy Fails

Greedy does not always guarantee optimal solution.

Example: 0/1 Knapsack → greedy fails

Example: Fractional Knapsack → greedy works

Therefore, correctness must be proven.

SECTION B: ACTIVITY SELECTION PROBLEM

3. Activity Selection (Interval Scheduling)

Given start[] and end[] of activities, choose maximum number of non-overlapping activities.

Greedy Strategy

Always pick activity with **earliest finishing time**.

Why earliest finish time works?

It leaves maximum room for future activities.

4. Java Code – Activity Selection

```
static class Activity {  
    int start, end;  
    Activity(int s, int e) { start = s; end = e; }  
}  
  
int maxActivities(List<Activity> arr) {  
    Collections.sort(arr, (a, b) -> a.end - b.end);  
  
    int count = 1;  
    int lastEnd = arr.get(0).end;  
  
    for (int i = 1; i < arr.size(); i++) {  
        if (arr.get(i).start >= lastEnd) {  
            count++;  
        }  
    }  
}
```

```

        lastEnd = arr.get(i).end;
    }
}
return count;
}

```

SECTION C: FRACTIONAL KNAPSACK

5. Problem Definition

Given weights[] and values[], maximize value with capacity W.
But items **can be taken fractionally**.

Greedy Strategy

Take item with **highest value/weight ratio** first.

6. Java Code – Fractional Knapsack

```

static class Item {
    int value, weight;
    Item(int v, int w) { value = v; weight = w; }
}

double fractionalKnapsack(int W, List<Item> arr) {
    Collections.sort(arr, (a, b) ->
        Double.compare((double)b.value / b.weight,
            (double)a.value / a.weight));

    double totalValue = 0.0;

    for (Item it : arr) {

```

```
    if (W == 0) break;

    if (it.weight <= W) {
        totalValue += it.value;
        W -= it.weight;
    } else {
        double fraction = (double)W / it.weight;
        totalValue += it.value * fraction;
        W = 0;
    }
}

return totalValue;
}
```

SECTION D: MINIMUM NUMBER OF PLATFORMS

7. Problem Statement

Given arrival and departure times of trains, find minimum number of platforms required.

Idea

Sort arrivals and departures separately.

Use two pointers.

Why greedy?

We try to reuse platforms whenever possible.

8. Java Code – Minimum Platforms

```
int findPlatforms(int[] arr, int[] dep) {
    Arrays.sort(arr);
    Arrays.sort(dep);
```

```
int i = 0, j = 0;

int platforms = 0, maxPlatforms = 0;

while (i < arr.length && j < dep.length) {
    if (arr[i] <= dep[j]) {
        platforms++;
        maxPlatforms = Math.max(maxPlatforms, platforms);
        i++;
    } else {
        platforms--;
        j++;
    }
}

return maxPlatforms;
}
```

SECTION E: JOB SEQUENCING WITH DEADLINES

9. Problem Definition

Each job has:

- Profit
- Deadline

Goal: Maximize profit by scheduling jobs within deadlines.

Greedy Strategy

Sort jobs by **profit descending**, schedule each to latest available slot.

10. Java Code – Job Sequencing

```
static class Job {  
    int profit, deadline;  
    Job(int p, int d) { profit = p; deadline = d; }  
}  
  
int jobSequencing(List<Job> jobs) {  
    Collections.sort(jobs, (a, b) -> b.profit - a.profit);  
  
    int maxDeadline = 0;  
    for (Job j : jobs) maxDeadline = Math.max(maxDeadline, j.deadline);  
  
    int[] slot = new int[maxDeadline + 1];  
    Arrays.fill(slot, -1);  
  
    int totalProfit = 0;  
  
    for (Job job : jobs) {  
        for (int d = job.deadline; d > 0; d--) {  
            if (slot[d] == -1) {  
                slot[d] = job.profit;  
                totalProfit += job.profit;  
                break;  
            }  
        }  
    }  
    return totalProfit;  
}
```

SECTION F: HUFFMAN CODING

11. What is Huffman Coding?

A greedy algorithm used for **data compression**.

Idea:

- Frequent characters → shorter codes
- Rare characters → longer codes

Uses

ZIP files

Multimedia compression

Text encoding

12. Huffman Tree Construction (Greedy Steps)

1. Create min heap of frequencies
 2. Extract two smallest
 3. Create new node = sum of two
 4. Insert back into heap
 5. Repeat until one tree remains
-

13. Java Code – Huffman Coding (Core Part)

```
static class Node {  
    char ch;  
    int freq;  
    Node left, right;  
  
    Node(char c, int f) {  
        ch = c; freq = f;  
    }  
}
```

```

Node buildHuffman(Map<Character,Integer> freq) {
    PriorityQueue<Node> pq =
        new PriorityQueue<>((a, b) -> a.freq - b.freq);

    for (char c : freq.keySet())
        pq.add(new Node(c, freq.get(c)));

    while (pq.size() > 1) {
        Node left = pq.poll();
        Node right = pq.poll();
        Node parent = new Node('-', left.freq + right.freq);
        parent.left = left;
        parent.right = right;
        pq.add(parent);
    }
    return pq.poll();
}

```

SECTION G: GAS STATION / CIRCULAR TOUR

14. Problem Definition

Find starting petrol pump index from which you can complete a circular tour.

Greedy solution:

Track surplus & deficit.

15. Java Code – Gas Station

```

int canCompleteCircuit(int[] gas, int[] cost) {

```

```
int total = 0, current = 0, start = 0;

for (int i = 0; i < gas.length; i++) {
    total += gas[i] - cost[i];
    current += gas[i] - cost[i];

    if (current < 0) {
        start = i + 1;
        current = 0;
    }
}

return total >= 0 ? start : -1;
}
```

SECTION H: GREEDY VS DP

Where Greedy Fails

- 0/1 Knapsack
- Coin change (non-canonical denominations)
- Minimum jumps (some cases)

Where Greedy Works

- Fractional knapsack
 - Activity selection
 - Scheduling
 - Huffman coding
-

SECTION I: COMPLEXITIES

Problem	Time	Space
Activity selection	$O(n \log n)$	$O(1)$
Fractional knapsack	$O(n \log n)$	$O(1)$
Huffman coding	$O(n \log n)$	$O(n)$
Job sequencing	$O(n \log n)$	$O(n)$
Minimum platforms	$O(n \log n)$	$O(1)$

SECTION J: COMMON MISTAKES

1. Choosing greedy where DP is needed
 2. Not sorting correctly
 3. Forgetting constraints (deadlines start at 1)
 4. Greedy on values instead of ratios
 5. Wrong tie-breaking
-

SECTION K: INTERVIEW INSIGHTS

1. Greedy proofs are important in product-based interviews
2. Frequently asked problems:
 - Activity selection
 - Fractional knapsack
 - Job sequencing
 - Minimum platforms
 - Gas station
3. Huffman coding may appear in theory rounds
4. Sorting-based greedy questions are common

Backtracking

A complete, professor-level, deeply detailed chapter.

SECTION A: INTRODUCTION TO BACKTRACKING

1. What is Backtracking?

Backtracking is a **systematic trial-and-error technique** where we build solutions step-by-step and **undo (backtrack)** the last step when it leads to invalid or incomplete solutions.

Real-world analogy

Solving a maze:

- Move forward
 - If stuck → return to previous junction
 - Try next path
-

2. When is Backtracking Used?

- Searching all valid configurations
- When recursion + decision making are involved
- Generating subsets/permutations
- Constraint satisfaction problems

Common backtracking problems

- N-Queens
 - Sudoku
 - Rat in a Maze
 - Subsets
 - Permutations
 - Combination Sum
 - Word Search
 - Palindrome Partitioning
-

3. Backtracking Template (Very Important)

Almost all backtracking code follows this pattern:

```
void solve(State state) {  
    if (solution found) {  
        save solution;  
        return;  
    }  
  
    for each choice ch in valid choices:  
        choose(ch);  
        solve(updated state);  
        unchoose(ch); // backtrack  
}
```

SECTION B: SUBSETS (POWER SET)

4. Problem: Print All Subsets of an Array

Example:

Input: [1, 2]

Output:

[]

[1]

[2]

[1,2]

5. Java Code – Subsets

```
void subsets(int[] nums, int index, List<Integer> path, List<List<Integer>> ans) {  
    if (index == nums.length) {  
        ans.add(new ArrayList<>(path));  
        return;  
    }  
}
```

```
subsets(nums, index + 1, path, ans);

path.add(nums[index]);

subsets(nums, index + 1, path, ans);

path.remove(path.size() - 1);
}
```

SECTION C: PERMUTATIONS

6. Problem: Generate All Permutations

Example:

Input: [1, 2, 3]

Output:

[1,2,3]

[1,3,2]

[2,1,3]

...

7. Java Code – Permutations

```
void permute(int[] nums, boolean[] used, List<Integer> path, List<List<Integer>> ans) {
    if (path.size() == nums.length) {
        ans.add(new ArrayList<>(path));
        return;
    }

    for (int i = 0; i < nums.length; i++) {
```

```
        if (used[i]) continue;

        used[i] = true;
        path.add(nums[i]);

        permute(nums, used, path, ans);

        used[i] = false;
        path.remove(path.size() - 1);
    }
}
```

SECTION D: COMBINATION SUM

8. Problem: Combination Sum (Repeated Use Allowed)

Input: [2,3,6,7], target=7

Output:

[2,2,3]

[7]

9. Java Code – Combination Sum

```
void combinationSum(int[] arr, int index, int target,
                    List<Integer> path, List<List<Integer>> ans) {
    if (target == 0) {
        ans.add(new ArrayList<>(path));
        return;
    }
```

```
if (index == arr.length || target < 0) return;

path.add(arr[index]);
combinationSum(arr, index, target - arr[index], path, ans);
path.remove(path.size() - 1);

combinationSum(arr, index + 1, target, path, ans);
}
```

SECTION E: N-QUEENS (Most Important Problem)

10. N-Queens Problem

Place N queens on an $N \times N$ chessboard such that:

- No two queens share the same row
- No two share the same column
- No two share same diagonal

Example (n = 4)

```
. Q . .
. . . Q
Q . . .
. . Q .
```

11. Attack Conditions

For queen at (r, c):

- Same column: `col[c] == true`
 - 45° diagonal: `diag1[r - c + n] == true`
 - 135° diagonal: `diag2[r + c] == true`
-

12. N-Queens Java Code

```
void solveNQueen(int row, int n, boolean[] col,
                boolean[] diag1, boolean[] diag2,
                char[][] board, List<List<String>> ans) {

    if (row == n) {
        List<String> res = new ArrayList<>();
        for (char[] r : board)
            res.add(new String(r));
        ans.add(res);
        return;
    }

    for (int c = 0; c < n; c++) {
        if (col[c] || diag1[row - c + n] || diag2[row + c])
            continue;

        col[c] = diag1[row - c + n] = diag2[row + c] = true;
        board[row][c] = 'Q';

        solveNQueen(row + 1, n, col, diag1, diag2, board, ans);

        col[c] = diag1[row - c + n] = diag2[row + c] = false;
        board[row][c] = '.';
    }
}
```

SECTION F: SUDOKU SOLVER

13. Sudoku Validation Conditions

A cell (r,c) is valid if:

1. Value not present in row
 2. Value not present in column
 3. Value not present in 3×3 box
-

14. Sudoku Solver Java Code

```
boolean solveSudoku(char[][] board) {  
    for (int r = 0; r < 9; r++) {  
        for (int c = 0; c < 9; c++) {  
            if (board[r][c] == '.') {  
  
                for (char d = '1'; d <= '9'; d++) {  
                    if (isValid(board, r, c, d)) {  
                        board[r][c] = d;  
  
                        if (solveSudoku(board))  
                            return true;  
  
                        board[r][c] = '.';  
                    }  
                }  
                return false;  
            }  
        }  
    }  
    return true;  
}
```



```
}
```

```
boolean isValid(char[][] board, int r, int c, char d) {
```

```
    for (int i = 0; i < 9; i++)
```

```
        if (board[r][i] == d || board[i][c] == d)
```

```
            return false;
```

```
    int boxR = (r / 3) * 3;
```

```
    int boxC = (c / 3) * 3;
```

```
    for (int i = 0; i < 3; i++)
```

```
        for (int j = 0; j < 3; j++)
```

```
            if (board[boxR+i][boxC+j] == d)
```

```
                return false;
```

```
    return true;
```

```
}
```

SECTION G: RAT IN A MAZE

15. Problem Definition

Find all paths from (0,0) to (n-1,n-1), moving Down, Left, Right, Up.

16. Java Code – Rat in a Maze

```
void solve(int r, int c, int[][] maze, boolean[][] visited,
```

```
    String path, List<String> ans) {
```

```
    int n = maze.length;
```

```

if (r == n - 1 && c == n - 1) {
    ans.add(path);
    return;
}

if (r < 0 || c < 0 || r >= n || c >= n ||
    maze[r][c] == 0 || visited[r][c])
    return;

visited[r][c] = true;

solve(r+1, c, maze, visited, path + "D", ans);
solve(r, c-1, maze, visited, path + "L", ans);
solve(r, c+1, maze, visited, path + "R", ans);
solve(r-1, c, maze, visited, path + "U", ans);

visited[r][c] = false;
}

```

SECTION H: WORD SEARCH

17. Problem Definition

Check if a word exists in a grid by moving UP/DOWN/LEFT/RIGHT.

18. Java Code – Word Search

```

boolean exists(char[][] board, String word) {
    int m = board.length, n = board[0].length;

```

```

for (int r = 0; r < m; r++) {
    for (int c = 0; c < n; c++) {
        if (dfs(board, word, 0, r, c))
            return true;
    }
}
return false;
}

```

```

boolean dfs(char[][] board, String word, int idx, int r, int c) {
    if (idx == word.length()) return true;

    if (r < 0 || c < 0 || r >= board.length ||
        c >= board[0].length || board[r][c] != word.charAt(idx))
        return false;

    char temp = board[r][c];
    board[r][c] = '#'; // mark visited

    boolean found = dfs(board, word, idx + 1, r+1, c)
        || dfs(board, word, idx + 1, r-1, c)
        || dfs(board, word, idx + 1, r, c+1)
        || dfs(board, word, idx + 1, r, c-1);

    board[r][c] = temp;
    return found;
}

```

SECTION I: PALINDROME PARTITIONING

19. Java Code – Palindrome Partitioning

```
void partition(int start, String s,
               List<String> path, List<List<String>> ans) {

    if (start == s.length()) {
        ans.add(new ArrayList<>(path));
        return;
    }

    for (int end = start; end < s.length(); end++) {
        if (isPal(s, start, end)) {
            path.add(s.substring(start, end+1));
            partition(end+1, s, path, ans);
            path.remove(path.size() - 1);
        }
    }
}

boolean isPal(String s, int l, int r) {
    while (l < r)
        if (s.charAt(l++) != s.charAt(r--))
            return false;
    return true;
}
```

SECTION J: TIME COMPLEXITY ANALYSIS OF BACKTRACKING

Problem	Time Complexity
Subsets	$O(2^n)$
Permutations	$O(n \times n!)$
N-Queens	$O(n!)$
Sudoku	Exponential
Combination Sum	Typically $O(2^n)$
Word Search	$O(4^{(m \times n)})$ worst case

Backtracking is inevitably exponential.

SECTION K: COMMON MISTAKES

1. Not undoing changes (forgetting to backtrack)
 2. Using global/shared data incorrectly
 3. Incorrect base conditions
 4. Recomputing states → use visited array
 5. Modifying input grid/string without restoring previous state
 6. Wrong recursion termination
-

SECTION L: INTERVIEW INSIGHTS

1. Backtracking questions are extremely common
2. N-Queens, Subsets, Permutations, and Sudoku are standard
3. Interviewers test understanding of:
 - Recursion
 - State tracking
 - Backtracking vs brute force
 - Pruning unnecessary paths
4. Code must be clean, with correct backtracking steps

5. Always draw recursion tree in interviews
6. Show that you know how to prune (skip invalid states early)

SECTION A: INTRODUCTION

1. What is the Sliding Window Technique?

Sliding Window is an optimization over brute force used when a problem involves:

- Contiguous subarrays / substrings
- Searching for maximum/minimum/longest/shortest window satisfying a condition
- Dynamic window expansion and shrinkage

Why sliding window?

Brute force $O(n^2)$ → sliding window reduces to $O(n)$.

Real-world analogy

Imagine looking through a small window on a moving train.

- The window shows only a part of the scene
- As the train moves, the window slides over the landscape
Same idea: we maintain a window over an array/string.

2. Types of Sliding Windows

1. Fixed Size Window

- Window size is known/fixed
- Example: Maximum sum of size k

2. Variable Size Window

- Window expands/contracts based on a condition
- Example: Longest substring without repeating characters

SECTION B: FIXED SIZE WINDOW

3. Maximum Sum Subarray of Size k

Input:

arr = [2,1,5,1,3,2], k = 3

Brute force: $O(n \times k)$

Sliding window: $O(n)$

4. Java Code – Maximum Sum Subarray (Fixed Window)

```
int maxSum(int[] arr, int k) {  
    int sum = 0;  
    for (int i = 0; i < k; i++)  
        sum += arr[i];  
  
    int max = sum;  
  
    for (int i = k; i < arr.length; i++) {  
        sum += arr[i] - arr[i - k];  
        max = Math.max(max, sum);  
    }  
    return max;  
}
```

Time: $O(n)$

Space: $O(1)$

SECTION C: VARIABLE SIZE WINDOW

5. Longest Substring Without Repeating Characters

Classic interview question.

Input: "abcabcbb"

Answer: "abc" → length 3

6. Java Code – Longest Substring Without Repeating Characters

```
int longestUnique(String s) {  
    int[] freq = new int[256];  
    int left = 0, max = 0;  
  
    for (int right = 0; right < s.length(); right++) {  
        freq[s.charAt(right)]++;  
  
        while (freq[s.charAt(right)] > 1) {  
            freq[s.charAt(left)]--;  
            left++;  
        }  
  
        max = Math.max(max, right - left + 1);  
    }  
    return max;  
}
```

Explanation:

- Expand window rightward until violation (duplicate)
- Shrink window from left until valid

7. Longest Substring with At Most K Distinct Characters

Input: "eceba", k=2

Answer: "ece" → length 3

8. Java Code – At Most K Distinct

```
int longestKDistinct(String s, int k) {  
    int[] freq = new int[256];
```



```

int left = 0, count = 0, max = 0;

for (int right = 0; right < s.length(); right++) {
    if (freq[s.charAt(right)] == 0) count++;
    freq[s.charAt(right)]++;

    while (count > k) {
        freq[s.charAt(left)]--;
        if (freq[s.charAt(left)] == 0) count--;
        left++;
    }

    max = Math.max(max, right - left + 1);
}

return max;
}

```

SECTION D: ANAGRAM & FREQUENCY MAP PROBLEMS

9. Find All Anagrams of Pattern in Text

Input:

s = "cbaebabacd"

p = "abc"

Output indices: [0,6]

10. Java Code – Find Anagrams

```

List<Integer> findAnagrams(String s, String p) {
    List<Integer> ans = new ArrayList<>();

```

```

if (s.length() < p.length()) return ans;

int[] freqP = new int[26];
for (char c : p.toCharArray())
    freqP[c - 'a']++;

int[] freqS = new int[26];
int k = p.length();

for (int i = 0; i < s.length(); i++) {
    freqS[s.charAt(i) - 'a']++;

    if (i >= k)
        freqS[s.charAt(i - k) - 'a']--;

    if (Arrays.equals(freqS, freqP))
        ans.add(i - k + 1);
}
return ans;
}

```

11. Minimum Window Substring (Very Important)

Given string *s* and pattern *t*, find the minimum window in *s* which contains all characters of *t*.

This is a classic variable window problem.

12. Java Code – Minimum Window Substring

```
String minWindow(String s, String t) {
```

```
int[] freq = new int[256];

for (char c : t.toCharArray())
    freq[c]++;

int required = t.length();
int left = 0, minLen = Integer.MAX_VALUE, start = 0;

for (int right = 0; right < s.length(); right++) {
    if (freq[s.charAt(right)] > 0)
        required--;

    freq[s.charAt(right)]--;

    while (required == 0) {
        if (right - left + 1 < minLen) {
            minLen = right - left + 1;
            start = left;
        }

        freq[s.charAt(left)]++;

        if (freq[s.charAt(left)] > 0)
            required++;

        left++;
    }
}
```

```
return minLen == Integer.MAX_VALUE ? ""  
    : s.substring(start, start + minLen);  
}
```

SECTION E: SUBARRAY PROBLEMS USING SLIDING WINDOW

13. Longest Subarray with Sum $\leq K$ (Variable Window)

Works only for **non-negative** numbers.

```
int longestSumLEK(int[] arr, int k) {  
    int left = 0, sum = 0, max = 0;  
  
    for (int right = 0; right < arr.length; right++) {  
        sum += arr[right];  
  
        while (sum > k) {  
            sum -= arr[left];  
            left++;  
        }  
  
        max = Math.max(max, right - left + 1);  
    }  
  
    return max;  
}
```

14. Smallest Subarray with Sum $\geq$ target

```
int minSubArrayLen(int target, int[] arr) {  
    int left = 0, sum = 0, min = Integer.MAX_VALUE;
```

```
for (int right = 0; right < arr.length; right++) {  
    sum += arr[right];  
  
    while (sum >= target) {  
        min = Math.min(min, right - left + 1);  
        sum -= arr[left];  
        left++;  
    }  
}  
  
return min == Integer.MAX_VALUE ? 0 : min;  
}
```

SECTION F: SLIDING WINDOW TEMPLATES (MASTER TEMPLATES)

15. Fixed Window Template

```
for (right in array) {  
    // add arr[right]  
  
    if (window size > k)  
        // remove arr[left++]  
  
    // update answer  
}
```

16. Variable Window Template

```
for (right in array) {  
    // include arr[right]
```

```
while (window invalid)

    // remove arr[left++]

// if window valid → update answer
}
```

17. Frequency Map Template (Most Important for Strings)

```
int[] freq = new int[256];
```

```
for (char c : pattern)
```

```
    freq[c]++;
```

```
int required = pattern length;
```

```
for (right in string) {
```

```
    if (freq[s[right]] > 0)
```

```
        required--;
```

```
    freq[s[right]]--;
```

```
while (required == 0) {
```

```
    // update answer
```

```
    freq[s[left]]++;
```

```
    if (freq[s[left]] > 0)
```

```
        required++;
```

```
    left++;  
}  
}
```

SECTION G: TIME COMPLEXITY

Most sliding window problems: **$O(n)$**

No nested loops over full input.

Space:

- If frequency array used: $O(1)$
 - If HashMap: $O(k)$
-

SECTION H: COMMON MISTAKES

1. Not shrinking window on invalid condition
 2. Off-by-one errors when calculating window size
 3. Confusing fixed vs variable windows
 4. Forgetting to maintain frequency maps
 5. Misinterpreting when window becomes valid
-

SECTION I: INTERVIEW INSIGHTS

1. This is one of the most important interview techniques
2. Frequently asked problems:
 - Longest substring without repeating
 - Minimum window substring
 - Count anagrams
 - Subarray sum constraints
3. Always mention $O(n)$ advantage
4. Clearly explain left/right pointer movement

SECTION A: INTRODUCTION

1. What is the Two-Pointers Technique?

Two-pointers is a pattern where you keep **two pointers (indexes)** that move through the array/string strategically to reduce complexity from $O(n^2)$ to $O(n)$.

Common pointer strategies:

1. **Left & Right pointers** on the same array
2. **Slow & Fast pointers** (linked lists)
3. **Two independent pointers** used in comparisons
4. **Meet in the middle** pointers

2. When to use Two-Pointers?

Use it when:

- You need to find pairs/triplets in sorted arrays
- Problems involve merging sorted arrays
- Removing duplicates
- Reversing arrays/strings
- Palindrome checks
- Linked list cycle detection (Floyd's algorithm)
- Middle of linked list
- Partitioning arrays

3. Real-World Analogy

Imagine two people walking on a track:

- One from the left end
- One from the right end

They move inward to meet depending on conditions.
This is exactly what two-pointers do over an array.

SECTION B: BASIC PATTERNS

PATTERN 1: OPPOSING POINTERS (BEGIN & END)

Used when:

- Array is sorted
 - Need to find a pair that satisfies some condition
 - Problems like Two Sum (sorted), Triplets, 3-Sum, 4-Sum
-

4. Example: Two Sum (Sorted Array)

Given sorted array, find two elements that sum to a target.

Java Code

```
int[] twoSum(int[] arr, int target) {  
    int left = 0, right = arr.length - 1;  
  
    while (left < right) {  
        int sum = arr[left] + arr[right];  
  
        if (sum == target)  
            return new int[]{left, right};  
  
        if (sum < target)  
            left++;  
        else  
            right--;  
    }  
    return new int[]{-1, -1};  
}
```

Time: $O(n)$

Space: $O(1)$

5. ASCII Diagram

arr: [1, 2, 3, 4, 6]

^ ^

left right

Move pointers inward depending on sum.

PATTERN 2: FAST & SLOW POINTERS

Used mainly in linked list problems:

- Detect cycle
 - Find cycle length
 - Find cycle starting point
 - Find middle of linked list
 - Remove nth node from end
-

6. Linked List Cycle Detection (Floyd's Algorithm)

Slow moves 1 step

Fast moves 2 steps

If they meet → cycle exists.

Java Code

```
boolean hasCycle(ListNode head) {  
  
    ListNode slow = head, fast = head;  
  
    while (fast != null && fast.next != null) {  
        slow = slow.next;  
        fast = fast.next.next;  
  
        if (slow == fast)
```

```
        return true;
    }
    return false;
}
```

7. Middle of Linked List

```
ListNode middle(ListNode head) {
    ListNode slow = head, fast = head;

    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}
```

Time: $O(n)$

Space: $O(1)$

PATTERN 3: TWO INDEPENDENT POINTERS

Useful when comparing two arrays/strings or merging them.

Examples:

- Merge two sorted arrays
 - Check if one string is subsequence of another
-

8. Check if s2 is a subsequence of s1

```
boolean isSubsequence(String s1, String s2) {
    int i = 0, j = 0;
```

```
while (i < s1.length() && j < s2.length()) {  
    if (s1.charAt(i) == s2.charAt(j))  
        j++;  
    i++;  
}  
return j == s2.length();  
}
```

PATTERN 4: WINDOW-BASED TWO POINTERS

This correlates strongly with **Sliding Window Technique**.
We maintain left and right pointers to define the window.

SECTION C: TWO POINTERS ON SORTED ARRAYS

9. Remove Duplicates from Sorted Array

Given sorted array, remove duplicates **in-place**.

Java Code

```
int removeDuplicates(int[] arr) {  
    int slow = 0;  
  
    for (int fast = 1; fast < arr.length; fast++) {  
        if (arr[fast] != arr[slow]) {  
            slow++;  
            arr[slow] = arr[fast];  
        }  
    }  
  
    return slow + 1; // new length  
}
```

10. Three Sum Problem (Very Important)

Given array, find all unique triplets that sum to 0.

Strategy:

1. Sort array
 2. Fix index i
 3. Use left & right pointers
-

Java Code – 3Sum

```
List<List<Integer>> threeSum(int[] arr) {  
    List<List<Integer>> res = new ArrayList<>();  
    Arrays.sort(arr);  
  
    for (int i = 0; i < arr.length - 2; i++) {  
        if (i > 0 && arr[i] == arr[i - 1])  
            continue;  
  
        int left = i + 1, right = arr.length - 1;  
  
        while (left < right) {  
            int sum = arr[i] + arr[left] + arr[right];  
  
            if (sum == 0) {  
                res.add(Arrays.asList(arr[i], arr[left], arr[right]));  
                left++;  
                right--;  
  
                while (left < right && arr[left] == arr[left - 1])  
                    left++;  
                while (left < right && arr[right] == arr[right + 1])  
                    right--;  
            }  
            else if (sum < 0)  
                left++;  
            else  
                right--;  
        }  
    }  
    return res;  
}
```

```

        left++;

        while (left < right && arr[right] == arr[right + 1])
            right--;
    }

    else if (sum < 0)
        left++;

    else
        right--;

    }

}

return res;

}

```

Time: $O(n^2)$

Space: $O(1)$

11. Four Sum Problem

Same idea as 3-sum, but more loops + two pointers.

SECTION D: TWO POINTERS ON STRINGS

12. Reverse a String Using Two Pointers

```

String reverse(String s) {

    char[] arr = s.toCharArray();

    int left = 0, right = arr.length - 1;

    while (left < right) {

        char temp = arr[left];

        arr[left] = arr[right];

```

```
    arr[right] = temp;

    left++;

    right--;

}

return new String(arr);

}
```

13. Check if String is Palindrome

```
boolean isPalindrome(String s) {

    int left = 0, right = s.length() - 1;


    while (left < right) {

        if (s.charAt(left) != s.charAt(right))

            return false;

        left++;

        right--;

    }

    return true;

}
```

SECTION E: TWO POINTERS ON LINKED LISTS

14. Find Start of Cycle

```
ListNode detectCycle(ListNode head) {

    ListNode slow = head, fast = head;


    while (fast != null && fast.next != null) {

        slow = slow.next;
```

```
fast = fast.next.next;

if (slow == fast) {
    slow = head;
    while (slow != fast) {
        slow = slow.next;
        fast = fast.next;
    }
    return slow;
}

return null;
}
```

15. Remove Nth Node from End

```
ListNode removeNth(ListNode head, int n) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;

    ListNode fast = dummy, slow = dummy;

    for (int i = 0; i < n; i++)
        fast = fast.next;

    while (fast.next != null) {
        fast = fast.next;
        slow = slow.next;
    }
}
```



```
slow.next = slow.next.next;

return dummy.next;
}
```

SECTION F: TWO POINTERS + SORTING = POWERFUL COMBINATION

Used for:

- Closest sum
 - Pair difference problems
 - Counting pairs
 - Container with most water
-

16. Container With Most Water (LeetCode 11)

Classic two-pointer problem.

```
int maxArea(int[] height) {
    int left = 0, right = height.length - 1;
    int max = 0;

    while (left < right) {
        int h = Math.min(height[left], height[right]);
        max = Math.max(max, h * (right - left));

        if (height[left] < height[right])
            left++;
        else
            right++;
    }
}
```

```
    return max;
}
```

Time: $O(n)$

SECTION G: TWO POINTERS + PREFIX CONDITION

Used in problems like:

- Largest subarray with 0 sum
 - K-sum variations
-

SECTION H: TIME COMPLEXITY

Most two-pointer problems: $O(n)$ or $O(n \log n)$ with sorting.

Space: $O(1)$

SECTION I: COMMON MISTAKES

1. Forgetting array must be sorted
 2. Infinite loops (wrong pointer updates)
 3. Overlapping pointers incorrectly
 4. Not handling duplicates for 3-sum, 4-sum
 5. Assuming two-pointers works on unsorted array (it does not)
-

SECTION J: INTERVIEW INSIGHTS

1. Two pointers is one of the highest-used techniques
2. Many FAANG interview problems reduce to two pointers
3. Always mention time complexity improvement
4. Sort + two pointers solves many pair/triplet problems
5. Cycle detection (linked lists) is a guaranteed interview topic

SECTION A: INTRODUCTION TO TRIES

1. What is a Trie?

A Trie (also called **prefix tree**) is a tree-like data structure used for fast retrieval of strings, especially when dealing with prefixes.

It stores characters in a hierarchical form so that all strings sharing prefixes use the same path.

2. Why Trie Exists?

(Understanding the pain point)

Suppose you store words:

cat
car
cart
dog
doll

Using a HashMap:

- Lookup is fast
- But prefix search (like searching all words starting with "ca") is slow
- Autocomplete and dictionary operations become inefficient

Trie solves this efficiently.

3. Real-world analogy

Imagine a dictionary:

- Words arranged alphabetically
- All words starting with "ca" grouped together
- All words starting with "car" in a subsection

A Trie organizes words in a similar spirit at the character level.

SECTION B: TRIE STRUCTURE

Each node contains:

1. Array or HashMap of children (size 26 for lowercase English)

2. Boolean flag indicating end of word

ASCII diagram for inserting words:

cat, car, dog

(root)

|

c ---- d

| |

a o

| |

t g

|

r

4. Node Structure in Java

```
class TrieNode {  
    TrieNode[] children = new TrieNode[26];  
    boolean isEnd;  
}
```

SECTION C: CORE TRIE OPERATIONS

We cover:

1. Insert
2. Search
3. StartsWith
4. Delete
5. Count words
6. Count prefixes
7. Autocomplete

5. Insert a Word

Steps:

1. Start from root
2. For every character:
 - If node doesn't exist → create new
3. Mark last node as end of word

Java code:

```
class Trie {  
    private TrieNode root;  
  
    public Trie() {  
        root = new TrieNode();  
    }  
  
    public void insert(String word) {  
        TrieNode curr = root;  
  
        for (char c : word.toCharArray()) {  
            int idx = c - 'a';  
  
            if (curr.children[idx] == null)  
                curr.children[idx] = new TrieNode();  
  
            curr = curr.children[idx];  
        }  
        curr.isEnd = true;  
    }  
}
```

```
}
```

6. Search a Word

```
public boolean search(String word) {  
    TrieNode curr = root;  
  
    for (char c : word.toCharArray()) {  
        int idx = c - 'a';  
  
        if (curr.children[idx] == null)  
            return false;  
  
        curr = curr.children[idx];  
    }  
    return curr.isEnd;  
}
```

7. Starts With (Prefix Search)

```
public boolean startsWith(String prefix) {  
    TrieNode curr = root;  
  
    for (char c : prefix.toCharArray()) {  
        int idx = c - 'a';  
  
        if (curr.children[idx] == null)  
            return false;  
  
        curr = curr.children[idx];  
    }  
}
```

```

    }

    return true;
}

```

This is where Trie shines.

Checking prefix in a HashMap is inefficient but $O(\text{length of prefix})$ in Trie.

SECTION D: TRIE DELETION

Deleting a word from Trie is tricky because:

- Other words may share nodes
- Cannot simply remove nodes blindly
- Must delete only if the node is not required by other strings

Concept: Use recursion

8. Java Code – Delete Word from Trie

```

public boolean delete(String word) {
    return deleteHelper(root, word, 0);
}

```

```

private boolean deleteHelper(TrieNode curr, String word, int index) {
    if (index == word.length()) {
        if (!curr.isEnd) return false;
        curr.isEnd = false;
        return isEmpty(curr);
    }
}

```

```

int idx = word.charAt(index) - 'a';

```

```

TrieNode node = curr.children[idx];

```

```

if (node == null) return false;

boolean shouldDelete = deleteHelper(node, word, index + 1);

if (shouldDelete) {
    curr.children[idx] = null;
    return !curr.isEnd && isEmpty(curr);
}

return false;
}

private boolean isEmpty(TrieNode node) {
    for (TrieNode child : node.children)
        if (child != null) return false;
    return true;
}

```

SECTION E: TRIE COMPLEXITIES

Operation	Time Complexity	Space
Insert	$O(\text{length of word})$	$O(N * \text{alphabet})$
Search	$O(\text{length of word})$	$O(1)$
Delete	$O(\text{length of word})$	$O(1)$
StartsWith	$O(\text{length of prefix})$	$O(1)$
Autocomplete	$O(\text{prefix} + \text{results})$	$O(1)$

Trie is extremely fast because operations depend only on the length of the string, not number of words.

SECTION F: AUTOCOMPLETE USING TRIE

Autocomplete requires:

1. Go to prefix node
2. DFS to fetch all words from there

9. Retrieve All Words With a Given Prefix

```
public List<String> autoComplete(String prefix) {  
    TrieNode curr = root;  
    List<String> result = new ArrayList<>();  
  
    for (char c : prefix.toCharArray()) {  
        int idx = c - 'a';  
        if (curr.children[idx] == null)  
            return result; // empty list  
        curr = curr.children[idx];  
    }  
  
    dfs(curr, prefix, result);  
    return result;  
}  
  
private void dfs(TrieNode node, String path, List<String> result) {  
    if (node.isEnd) result.add(path);  
  
    for (int i = 0; i < 26; i++) {  
        if (node.children[i] != null) {  
            dfs(node.children[i], path + (char)(i + 'a'), result);  
        }  
    }  
}
```

```
}  
  
}
```

This builds a full autocomplete system.

SECTION G: TRIE VARIANTS

10. Compressed Trie (Radix Tree)

Compress nodes that have a single child.
Space efficient version.

Example:

normal trie for "cat" and "car":

$c \rightarrow a \rightarrow t$

$c \rightarrow a \rightarrow r$

compressed trie:

$c \rightarrow a \rightarrow ("t", "r")$

11. Suffix Trie

Store all suffixes of a string.

Used for:

- Fast substring search
- Pattern matching
- Longest repeated substring

Example for “banana”:

banana

anana

nana

ana

na

a

12. Ternary Search Tree (TST)

Each node has 3 children:

- left
- equal
- right

Works like character-level BST.

Space efficient compared to full Trie.

SECTION H: TRIE USE CASES

13. Applications of Trie

1. Autocomplete
 2. Spell Check
 3. Word Suggestions
 4. IP Routing (binary trie)
 5. Searching contacts
 6. Prefix matching
 7. dictionary implementation
 8. DNA sequence search
-

SECTION I: COMMON MISTAKES

1. Using HashMap instead of array unnecessarily
2. Forgetting to mark end-of-word

3. Not handling delete properly
 4. Using too much memory for sparse alphabets
 5. Confusing prefix search with full search
-

SECTION J: INTERVIEW INSIGHTS

1. Tries are highly likely in system design and product-based interviews
 2. Frequent coding rounds:
 - Autocomplete
 - Prefix matching
 - Word search in grid (Trie + DFS)
 - Count distinct substrings using Trie
 3. Mention complexity advantage over HashMap for prefix queries
 4. Always explain memory trade-offs
-

SECTION K: FULL TRIE CLASS (COMPLETE IMPLEMENTATION)

```
class TrieNode {  
    TrieNode[] children = new TrieNode[26];  
    boolean isEnd;  
}
```

```
class Trie {  
    private TrieNode root = new TrieNode();  
  
    public void insert(String word) {  
        TrieNode curr = root;  
        for (char c : word.toCharArray()) {  
            int idx = c - 'a';  
            if (curr.children[idx] == null)
```

```
        curr.children[idx] = new TrieNode();  
        curr = curr.children[idx];  
    }  
    curr.isEnd = true;  
}
```

```
public boolean search(String word) {  
    TrieNode curr = root;  
    for (char c : word.toCharArray()) {  
        int idx = c - 'a';  
        if (curr.children[idx] == null)  
            return false;  
        curr = curr.children[idx];  
    }  
    return curr.isEnd;  
}
```

```
public boolean startsWith(String prefix) {  
    TrieNode curr = root;  
    for (char c : prefix.toCharArray()) {  
        int idx = c - 'a';  
        if (curr.children[idx] == null)  
            return false;  
        curr = curr.children[idx];  
    }  
    return true;  
}
```

```

public List<String> autoComplete(String prefix) {
    TrieNode curr = root;
    List<String> result = new ArrayList<>();

    for (char c : prefix.toCharArray()) {
        int idx = c - 'a';
        if (curr.children[idx] == null)
            return result;
        curr = curr.children[idx];
    }

    dfs(curr, prefix, result);
    return result;
}

private void dfs(TrieNode node, String path, List<String> result) {
    if (node.isEnd) result.add(path);
    for (int i = 0; i < 26; i++) {
        if (node.children[i] != null) {
            dfs(node.children[i], path + (char)(i + 'a'), result);
        }
    }
}

```

SECTION A: INTRODUCTION

1. What is a Segment Tree?

A Segment Tree is a binary tree used for answering **range queries** and performing **updates** efficiently.

Range queries include:

- Range Sum
- Range Minimum (RMQ)
- Range Maximum
- Range GCD
- Range XOR

Segment Trees support:

- Query: $O(\log n)$
- Update: $O(\log n)$

This is a huge improvement over naive $O(n)$ per query.

2. Why Segment Tree?

Real-world problem:

You have an array and you need to frequently:

- Get sum of values from L to R
- Update $\text{arr}[i]$ to new value
- Repeat many times

If you use prefix sums:

- Query = $O(1)$
- Update = $O(n)$ (not acceptable for large arrays)

Segment tree solves both in **$O(\log n)$** .

3. Real-World Analogy

Imagine you manage a large company with many departments.

- To get total budget of departments 10–30, you don't add every one manually
- You refer to a hierarchy where groups of departments are pre-summed

The company hierarchy acts like a Segment Tree.

SECTION B: BASIC CONCEPT

4. Structure of Segment Tree

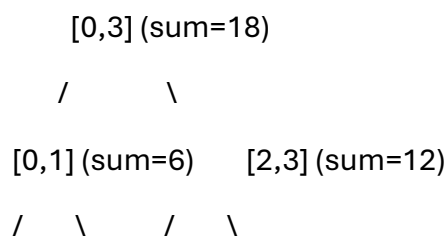
A segment tree is a complete binary tree where:

- Each node stores information about a segment of the array
- Root = entire array
- Leaves = individual elements

Example array:

arr = [2, 4, 5, 7]

Tree representation (sum tree):



5. Storage of Segment Tree

We typically store the tree in an array tree[] of size $4 * n$.

Indexing:

- root $\rightarrow$ index 1
 - left child $\rightarrow$ index $* 2$
 - right child $\rightarrow$ index $* 2 + 1$
-

SECTION C: BUILDING THE SEGMENT TREE

6. Java Code – Build Segment Tree (Sum Tree)


```
class SegmentTree {  
    int[] tree;  
    int n;  
  
    public SegmentTree(int[] arr) {  
        n = arr.length;  
        tree = new int[4 * n];  
        build(arr, 0, n - 1, 1);  
    }  
  
    private void build(int[] arr, int start, int end, int index) {  
        if (start == end) {  
            tree[index] = arr[start];  
            return;  
        }  
  
        int mid = (start + end) / 2;  
  
        build(arr, start, mid, index * 2);  
        build(arr, mid + 1, end, index * 2 + 1);  
  
        tree[index] = tree[index * 2] + tree[index * 2 + 1];  
    }  
}
```

Complexity

Build: $O(n)$

Space: $O(4n)$

SECTION D: RANGE QUERY (SUM)

7. Query Range Sum (L, R)

Three cases:

1. **Complete Overlap**
Query covers node segment
Return value
2. **No Overlap**
Outside segment
Return neutral value (0 for sum)
3. **Partial Overlap**
Split into children

8. Java Code – Query Range Sum

```
public int query(int qs, int qe) {  
    return queryUtil(0, n - 1, qs, qe, 1);  
}  
  
private int queryUtil(int start, int end, int qs, int qe, int index) {  
    if (qs > end || qe < start)  
        return 0;  
  
    if (qs <= start && end <= qe)  
        return tree[index];  
  
    int mid = (start + end) / 2;  
  
    int left = queryUtil(start, mid, qs, qe, index * 2);
```

```
int right = queryUtil(mid + 1, end, qs, qe, index * 2 + 1);

return left + right;
}
```

SECTION E: POINT UPDATE

9. Update arr[i] = newValue

Propagate the update along the path.

```
public void update(int pos, int val) {
    updateUtil(0, n - 1, pos, val, 1);
}

private void updateUtil(int start, int end, int pos, int val, int index) {
    if (start == end) {
        tree[index] = val;
        return;
    }

    int mid = (start + end) / 2;

    if (pos <= mid)
        updateUtil(start, mid, pos, val, index * 2);
    else
        updateUtil(mid + 1, end, pos, val, index * 2 + 1);

    tree[index] = tree[index * 2] + tree[index * 2 + 1];
}
```

Time: $O(\log n)$

SECTION F: RANGE MINIMUM QUERY (RMQ)

Segment tree can also store:

- Minimum
- Maximum
- GCD
- XOR

Only difference: node stores different info.

Example: replace sum merge with min merge:

```
tree[index] = Math.min(tree[index * 2], tree[index * 2 + 1]);
```

SECTION G: LAZY PROPAGATION

(For Range Updates)

10. Why Lazy Propagation?

Problem:

You want to update a whole range:

```
arr[L..R] += value
```

If you propagate immediately, complexity becomes $O(n \log n)$.

Lazy propagation allows:

- Range updates in $O(\log n)$
 - Range queries in $O(\log n)$
-

11. Concept of Lazy Tree

We maintain a lazy[] array.

If update cannot be pushed down immediately, store it in lazy child nodes.

ASCII Diagram:

Parent has pending update

Lazy stores pending update to children

Propagate only when needed

12. Java Implementation – Lazy Propagation

```
int[] lazy;
```

```
private void updateRange(int start, int end, int l, int r, int val, int index) {
```

```
    if (lazy[index] != 0) {  
        tree[index] += (end - start + 1) * lazy[index];
```

```
        if (start != end) {  
            lazy[index * 2] += lazy[index];  
            lazy[index * 2 + 1] += lazy[index];  
        }
```

```
        lazy[index] = 0;
```

```
    }
```

```
    if (start > r || end < l)
```

```
        return;
```

```
    if (l <= start && end <= r) {
```

```
        tree[index] += (end - start + 1) * val;
```

```
        if (start != end) {
```

```
            lazy[index * 2] += val;
```

```
            lazy[index * 2 + 1] += val;
```

```

    }

    return;
}

int mid = (start + end) / 2;

updateRange(start, mid, l, r, val, index * 2);
updateRange(mid + 1, end, l, r, val, index * 2 + 1);

tree[index] = tree[index * 2] + tree[index * 2 + 1];
}

```

SECTION H: USE CASES OF SEGMENT TREE

1. Range Sum Query (RSQ)
 2. Range Minimum/Maximum Query (RMQ)
 3. Range XOR
 4. Range GCD
 5. Range updates (lazy propagation)
 6. Stock prices time-series
 7. Competitive programming
 8. Frequency tables
 9. 2D segment trees for matrices
-

SECTION I: COMPLEXITIES

Operation	Time	Space
Build	$O(n)$	$O(4n)$
Query	$O(\log n)$	$O(1)$

Operation	Time	Space
Update	$O(\log n)$	$O(1)$
Lazy Range Update	$O(\log n)$	$O(n)$

SECTION J: COMMON MISTAKES

1. Wrong mid calculation
 2. Forgetting to return 0 for no-overlap in sum tree
 3. Not using neutral element properly
 4. Mixing 1-indexed and 0-indexed segments
 5. Incorrect lazy propagation cleanup
-

SECTION K: FULL SEGMENT TREE IMPLEMENTATION (SUM + POINT UPDATE + RANGE QUERY)

```
class SegmentTree {
    int[] tree;
    int n;

    SegmentTree(int[] arr) {
        n = arr.length;
        tree = new int[4 * n];
        build(arr, 0, n - 1, 1);
    }

    void build(int[] arr, int start, int end, int index) {
        if (start == end) {
            tree[index] = arr[start];
            return;
        }
    }
```

```

int mid = (start + end) / 2;

build(arr, start, mid, index * 2);
build(arr, mid + 1, end, index * 2 + 1);

tree[index] = tree[index * 2] + tree[index * 2 + 1];
}

int query(int qs, int qe) {
    return queryUtil(0, n - 1, qs, qe, 1);
}

int queryUtil(int start, int end, int qs, int qe, int index) {
    if (qs > end || qe < start)
        return 0;

    if (qs <= start && end <= qe)
        return tree[index];

    int mid = (start + end) / 2;

    int left = queryUtil(start, mid, qs, qe, index * 2);
    int right = queryUtil(mid + 1, end, qs, qe, index * 2 + 1);

    return left + right;
}

```



```

void update(int pos, int val) {
    updateUtil(0, n - 1, pos, val, 1);
}

void updateUtil(int start, int end, int pos, int val, int index) {
    if (start == end) {
        tree[index] = val;
        return;
    }

    int mid = (start + end) / 2;

    if (pos <= mid)
        updateUtil(start, mid, pos, val, index * 2);
    else
        updateUtil(mid + 1, end, pos, val, index * 2 + 1);

    tree[index] = tree[index * 2] + tree[index * 2 + 1];
}
}

```

SECTION A: INTRODUCTION

1. What is a Fenwick Tree?

A Fenwick Tree (also called Binary Indexed Tree or BIT) is a data structure that efficiently supports:

- Prefix sum queries
- Point updates

Everything in **$O(\log n)$** .

Fenwick Tree is a simpler and more memory-efficient alternative to Segment Tree for prefix-based operations.

2. Why Fenwick Tree Exists?

Problem:

You need to maintain an array that supports:

1. Update $\text{arr}[i] += x$
2. Query sum of $\text{arr}[1..i]$

Naive approach: $O(n)$ per query.

Fenwick Tree reduces this to **$O(\log n)$** .

3. Real-World Analogy

Think of a Fenwick Tree as a system where:

- Some buckets store small segments
- Some store bigger segments
- You combine buckets to get prefix sum
- Updates ripple through specific buckets based on binary representation

This structure uses binary properties to jump efficiently.

SECTION B: CORE CONCEPT

4. Understanding BIT with Binary Representation

BIT works using the **least significant bit (LSB)**.

Definition:

$$\text{LSB}(x) = x \& (-x)$$

Examples:

x binary LSB

4 100 100 (4)

x binary LSB

6 110 010 (2)

12 1100 0100 (4)

BIT stores partial sums in a clever way:

- bit[i] stores sum of elements from $i - \text{LSB}(i) + 1$ to i

5. ASCII Illustration of BIT Ranges

For array indices 1..8:

Index: 1 2 3 4 5 6 7 8

Range:

1	[1]
2	[1..2]
3	[3]
4	[1..4]
5	[5]
6	[5..6]
7	[7]
8	[1..8]

SECTION C: OPERATIONS

6. Update Operation

To update index i by $+val$:

1. Add val to $\text{bit}[i]$
2. Move to next index: $i += \text{LSB}(i)$
3. Repeat until $i > n$

Java:

```
void update(int i, int val) {
    while (i <= n) {
        bit[i] += val;
        i += i & -i;
    }
}
```

7. Prefix Sum Query

To compute sum of arr[1..i]:

1. Add bit[i]
2. Move to parent node: $i -= \text{LSB}(i)$
3. Repeat until $i = 0$

Java:

```
int prefixSum(int i) {
    int sum = 0;
    while (i > 0) {
        sum += bit[i];
        i -= i & -i;
    }
    return sum;
}
```

8. Range Sum Query (L..R)

Using prefix sums:

$\text{sum}(L..R) = \text{prefixSum}(R) - \text{prefixSum}(L-1)$

SECTION D: BUILDING THE BIT

There are two ways:

1. Insert one-by-one: $O(n \log n)$

2. Efficient $O(n)$ construction

Efficient construction:

```
for (int i = 1; i <= n; i++) {  
    int j = i + (i & -i);  
    if (j <= n)  
        bit[j] += bit[i];  
}
```

SECTION E: FULL JAVA IMPLEMENTATION

9. Fenwick Tree Java Class

```
class FenwickTree {  
    int[] bit;  
    int n;  
  
    FenwickTree(int n) {  
        this.n = n;  
        bit = new int[n + 1];  
    }  
  
    FenwickTree(int[] arr) {  
        this.n = arr.length;  
        bit = new int[n + 1];  
  
        for (int i = 1; i <= n; i++)  
            bit[i] = arr[i - 1];  
    }  
}
```

```
for (int i = 1; i <= n; i++) {  
    int j = i + (i & -i);  
    if (j <= n)  
        bit[j] += bit[i];  
}  
}
```

```
void update(int i, int val) {  
    while (i <= n) {  
        bit[i] += val;  
        i += i & -i;  
    }  
}
```

```
int prefixSum(int i) {  
    int sum = 0;  
    while (i > 0) {  
        sum += bit[i];  
        i -= i & -i;  
    }  
    return sum;  
}
```

```
int rangeSum(int l, int r) {  
    return prefixSum(r) - prefixSum(l - 1);  
}  
}
```

SECTION F: USE CASES

1. Range sum queries
 2. Frequency tables
 3. Competitive programming (very common)
 4. Inversions counting
 5. Counting smaller elements to the right
 6. Updating elements in real-time
 7. Dynamic histogram or counters
-

SECTION G: INVERSION COUNT USING BIT

(Important Interview Problem)

Given array, count number of pairs (i, j) such that:

- $i < j$
- $arr[i] > arr[j]$

We compress coordinates and use BIT.

10. Inversion Count Java Code

```
int countInversions(int[] arr) {  
    int n = arr.length;  
  
    int[] temp = arr.clone();  
    Arrays.sort(temp);  
  
    Map<Integer, Integer> rank = new HashMap<>();  
    int r = 1;  
    for (int num : temp)  
        if (!rank.containsKey(num))  
            rank.put(num, r++);  
}
```

```

FenwickTree ft = new FenwickTree(r);

int invCount = 0;

for (int i = n - 1; i >= 0; i--) {
    int pos = rank.get(arr[i]);
    invCount += ft.prefixSum(pos - 1);
    ft.update(pos, 1);
}

return invCount;
}

```

SECTION H: COMPLEXITIES

Operation	Time	Space
Update	$O(\log n)$	$O(n)$
Prefix Sum	$O(\log n)$	$O(1)$
Range Sum	$O(\log n)$	$O(1)$
Build	$O(n)$	$O(n)$

Fenwick Tree is more space-efficient than Segment Tree.

SECTION I: COMMON MISTAKES

1. Forgetting Fenwick Tree is 1-indexed
2. Using 0-indexed logic incorrectly
3. Forgetting LSB logic
4. Using `arr[]` directly instead of `BIT[]`

5. Confusion with negative indices on prefixSum
-

SECTION J: INTERVIEW INSIGHTS

1. Very common in competitive programming
2. Often used in problems involving dynamic queries
3. Excellent for online systems (real-time updates)
4. Simpler than segment trees
5. If only prefix sums and point updates required → BIT is best

SECTION A: INTRODUCTION

1. What is Bit Manipulation?

Bit manipulation involves using binary operations directly on integers to achieve very fast calculations.

Computers store everything in bits (0 or 1).

Bit operations run at CPU-instruction level → extremely fast.

2. Why Learn Bit Manipulation?

Important for:

- DSA & competitive programming
 - Low-level optimizations
 - Problems requiring:
 - subsets
 - masks
 - toggling bits
 - uniqueness
 - parity
 - power of 2 checks
 - Many interview problems use bit tricks
-

3. Binary Representation Review

Number → Binary:

5 = 0101

7 = 0111

12 = 1100

Each bit position represents powers of 2.

SECTION B: BITWISE OPERATORS

4. Basic Bitwise Operators

Operator	Meaning	Example
&	AND	1010 & 1100 = 1000
	OR	1010
^	XOR	1010 ^ 1100 = 0110
~	NOT	~1010 = 0101
<<	Left Shift	x << 1 = multiply by 2
>>	Right Shift	x >> 1 = divide by 2

SECTION C: COMMON BIT TRICKS

5. Check if a number is even or odd

if ((n & 1) == 0) → even

else odd

6. Multiply/Divide by 2 using shifts

n << 1 = n * 2

n >> 1 = n / 2

7. Clear the lowest set bit

$n = n \& (n - 1)$

Example:

$n = 12 = 1100$

$n-1 = 11 = 1011$

$n \& (n-1) = 1000$

8. Check if number is a power of two

$\text{if } (n > 0 \&\& (n \& (n - 1)) == 0)$

power of 2

9. Count number of set bits (Brian Kernighan's Algorithm)

```
int countBits(int n) {
```

```
    int count = 0;
```

```
    while (n > 0) {
```

```
        n &= (n - 1);
```

```
        count++;
```

```
    }
```

```
    return count;
```

```
}
```

Time: $O(\text{number of set bits})$

SECTION D: GET, SET, CLEAR, TOGGLE BITS

10. Get a bit at position i

```
int bit = (n >> i) & 1;
```

11. Set a bit at position i

```
n = n | (1 << i);
```

12. Clear a bit at position i

```
n = n & ~(1 << i);
```

13. Toggle a bit at position i

```
n = n ^ (1 << i);
```

SECTION E: RANGE CLEARING

14. Clear bits from LSB to i

```
n = n & ~( (1 << (i + 1)) - 1 );
```

15. Clear bits in a specific range (i to j)

```
int mask1 = (~0) << (j + 1);
```

```
int mask2 = (1 << i) - 1;
```

```
n = n & (mask1 | mask2);
```

SECTION F: SWAPPING USING XOR

16. Swap two numbers without temp

```
a = a ^ b;
```

```
b = a ^ b;
```

```
a = a ^ b;
```

SECTION G: SUBSETS USING BITMASKS

Given array:

```
arr = [1,2,3]
```

All subsets can be represented using a mask from 0 to 7.

Example mask = 5 = (101)

include arr[0]

skip arr[1]

include arr[2]

Java code:

```
void subsets(int[] arr) {  
    int n = arr.length;  
    for (int mask = 0; mask < (1 << n); mask++) {  
        List<Integer> subset = new ArrayList<>();  
        for (int i = 0; i < n; i++)  
            if ((mask & (1 << i)) != 0)  
                subset.add(arr[i]);  
        System.out.println(subset);  
    }  
}
```

SECTION H: BIT TRICKS FOR INTERVIEW

17. Single Number (Every element appears twice except one)

```
int singleNumber(int[] arr) {  
    int xor = 0;  
    for (int x : arr)  
        xor ^= x;  
    return xor;  
}
```

Why XOR works:

$x \oplus x = 0$

$x \oplus 0 = x$

18. Single Number II (Every element appears 3 times except one)

Track each bit's count mod 3.

```
int singleNumber(int[] nums) {  
    int ones = 0, twos = 0;  
    for (int x : nums) {  
        ones = (ones ^ x) & ~twos;  
        twos = (twos ^ x) & ~ones;  
    }  
    return ones;  
}
```

19. Find Two Unique Numbers (Every element twice except two)

```
int[] twoSingle(int[] arr) {  
    int xor = 0;  
    for (int x : arr) xor ^= x;  
  
    int lsb = xor & -xor;  
  
    int a = 0, b = 0;  
    for (int x : arr) {  
        if ((x & lsb) == 0) a ^= x;  
        else b ^= x;  
    }  
  
    return new int[]{a, b};  
}
```

```
}
```

SECTION I: ADVANCED BIT TRICKS

20. Reverse Bits

```
int reverseBits(int n) {  
    int rev = 0;  
    for (int i = 0; i < 32; i++) {  
        rev <<= 1;  
        if ((n & 1) == 1)  
            rev |= 1;  
        n >>= 1;  
    }  
    return rev;  
}
```

21. Hamming Distance Between Two Numbers

```
int hammingDistance(int a, int b) {  
    return countBits(a ^ b);  
}
```

22. Find Next Power of Two

```
int nextPowerOfTwo(int n) {  
    if (n == 0) return 1;  
    n--;  
    n |= n >> 1;  
    n |= n >> 2;  
    n |= n >> 4;
```

```

n |= n >> 8;
n |= n >> 16;
return n + 1;
}

```

23. Check if k-th bit is set

```

boolean isSet(int n, int k) {
    return (n & (1 << k)) != 0;
}

```

24. Turn off the rightmost set bit

```

n = n & (n - 1);

```

SECTION J: SUMMARY TABLE OF OPERATORS

Operation	Expression	Purpose
is even	$n \& 1$	check last bit
set bit	$n \mid (1 \ll i)$	
clear bit	$n \& \sim(1 \ll i)$	turn bit off
toggle bit	$n \wedge (1 \ll i)$	flip bit
LSB	$n \& -n$	isolate lowest set bit
remove LSB	$n \& (n - 1)$	clear lowest set bit
is power of 2 $n > 0 \&\& (n \& (n - 1)) == 0$ check P.O.T		

SECTION K: TIME COMPLEXITY

Bit operations are $O(1)$.

Bitmasking across all subsets: $O(2^n)$.

Counting bits (Brian Kernighan): $O(\text{bits set})$.

SECTION L: COMMON MISTAKES

1. Forgetting operator precedence
 2. Incorrect use of negative numbers
 3. Using signed shift ($\gg$) instead of unsigned shift ($\ggg$)
 4. Overflow when shifting too much
 5. Incorrect mask formation
-

SECTION M: INTERVIEW INSIGHTS

1. Bit problems appear frequently in FAANG
2. Many DP optimizations use bitmasking
3. Bit tricks drastically reduce solution complexity
4. XOR-based problems are interview favorites
5. Counting bits, power of 2 checks are standard screening questions

SECTION A: INTRODUCTION

String algorithms deal with:

- Finding patterns in text
- Searching occurrences of a substring
- Processing string prefixes and suffixes
- Comparing large strings efficiently

A naive pattern search takes $O(n \times m)$.

Advanced string algorithms reduce this to $O(n + m)$.

SECTION B: KMP ALGORITHM (Knuth-Morris-Pratt)

1. What is KMP?

KMP is a string pattern-matching algorithm that searches for a pattern P in text T in $O(n + m)$ time.

KMP uses preprocessing to avoid re-checking characters.

2. Key Concept: LPS Array (Longest Prefix Suffix)

LPS[i] = longest proper prefix of pattern[0..i] which is also a suffix.

Example pattern: ababaca

Compute LPS:

i	P[i]	LPS
---	------	-----

0	a	0
---	---	---

1	b	0
---	---	---

2	a	1
---	---	---

3	b	2
---	---	---

4	a	3
---	---	---

5	c	0
---	---	---

6	a	1
---	---	---

3. Real-world analogy

Imagine reading a book and encountering a mismatch.

Instead of going back to page 1, you jump to the last known matching section.

KMP uses the same idea using LPS.

4. ASCII Diagram – Pattern Sliding

T: a b a b a c a b a b a a c

P: a b a b a c a

Mismatch at index → shift pattern using LPS

5. Building the LPS Array

```
int[] buildLPS(String pat) {
```

```
    int m = pat.length();
```

```

int[] lps = new int[m];

int len = 0;
int i = 1;

while (i < m) {
    if (pat.charAt(i) == pat.charAt(len)) {
        len++;
        lps[i] = len;
        i++;
    } else {
        if (len != 0)
            len = lps[len - 1];
        else {
            lps[i] = 0;
            i++;
        }
    }
}

return lps;
}

```

6. KMP Search Algorithm

```

List<Integer> KMPSearch(String text, String pat) {

    int n = text.length();

    int m = pat.length();

    int[] lps = buildLPS(pat);

```

```

int i = 0, j = 0;

List<Integer> result = new ArrayList<>();

while (i < n) {
    if (text.charAt(i) == pat.charAt(j)) {
        i++;
        j++;
    }

    if (j == m) {
        result.add(i - j);
        j = lps[j - 1];
    } else if (i < n && text.charAt(i) != pat.charAt(j)) {
        if (j != 0)
            j = lps[j - 1];
        else
            i++;
    }
}

return result;
}

```

7. Complexity

Time: $O(n + m)$

Space: $O(m)$

KMP is optimal for pattern matching in worst-case scenarios.

SECTION C: Z-ALGORITHM

8. What is Z-Algorithm?

For a string S, Z-array stores:

$Z[i]$ = length of longest substring starting at i that matches prefix of S.

Example:

S = "aabxaayaab"

Z array:

i: 0 1 2 3 4 5 6 7 8 9

S: a a b x a a y a a b

Z: - 1 0 0 3 1 0 2 1 0

9. How Z Helps in Pattern Matching?

Concatenate pattern + '\$' + text

Compute Z array.

Wherever $Z[i] == \text{length}(\text{pattern})$, match found.

10. Z-Function Construction

```
int[] Z(String s) {  
    int n = s.length();  
    int[] z = new int[n];  
  
    int l = 0, r = 0;  
  
    for (int i = 1; i < n; i++) {  
        if (i <= r)  
            z[i] = Math.min(r - i + 1, z[i - l]);  
  
        while (i + z[i] < n && s.charAt(z[i]) == s.charAt(i + z[i]))  
            z[i]++;  
    }  
}
```

```
        if (i + z[i] - 1 > r) {  
            l = i;  
            r = i + z[i] - 1;  
        }  
    }  
    return z;  
}
```

11. Pattern Search Using Z

```
List<Integer> ZSearch(String text, String pat) {  
    String s = pat + "$" + text;  
    int[] z = Z(s);  
  
    List<Integer> result = new ArrayList<>();  
    int m = pat.length();  
  
    for (int i = 0; i < z.length; i++)  
        if (z[i] == m)  
            result.add(i - m - 1);  
  
    return result;  
}
```

12. Complexity

Time: $O(n + m)$

Space: $O(n + m)$

SECTION D: RABIN-KARP ALGORITHM

13. What is Rabin-Karp?

Rabin-Karp uses **hashing** to find pattern occurrences.

Key ideas:

- Compute hash of pattern
 - Compute rolling hash of text windows
 - Compare hashes
 - When hashes match, validate (to avoid collision)
-

14. Rolling Hash Concept

We treat string as base-d number ($d = 256$ or alphabet size).

Example:

For pattern length m :

$\text{hash}(\text{"abc"}) = a \cdot d^2 + b \cdot d^1 + c \cdot d^0$

For next window:

remove left char

shift left

add new right char

15. Java Code – Rabin-Karp

```
List<Integer> rabinKarp(String text, String pat) {  
    int n = text.length();  
    int m = pat.length();  
  
    int base = 256;  
    int mod = 1000000007;
```

```

long patHash = 0, textHash = 0;

long power = 1;

// power = base^(m-1)
for (int i = 0; i < m - 1; i++)
    power = (power * base) % mod;

// Compute initial hash
for (int i = 0; i < m; i++) {
    patHash = (patHash * base + pat.charAt(i)) % mod;
    textHash = (textHash * base + text.charAt(i)) % mod;
}

List<Integer> result = new ArrayList<>();

// Slide the window
for (int i = 0; i <= n - m; i++) {
    if (patHash == textHash) {
        if (text.substring(i, i + m).equals(pat))
            result.add(i);
    }

    if (i < n - m) {
        textHash = (textHash - text.charAt(i) * power) % mod;
        if (textHash < 0) textHash += mod;

        textHash = (textHash * base + text.charAt(i + m)) % mod;
    }
}

```



```
}  
  
return result;  
  
}
```

16. Complexity

Average: $O(n + m)$

Worst-case: $O(n \times m)$ (due to collisions)

But typically efficient.

SECTION E: COMPARISON TABLE

Algorithm	Time	Space	Strength
KMP	$O(n + m)$	$O(m)$	No backtracking
Z-Algorithm	$O(n + m)$	$O(n+m)$	Useful for multiple patterns
Rabin-Karp	Avg $O(n+m)$	$O(1)$	Hash-based: ideal for multi-search

SECTION F: REAL INTERVIEW USAGE

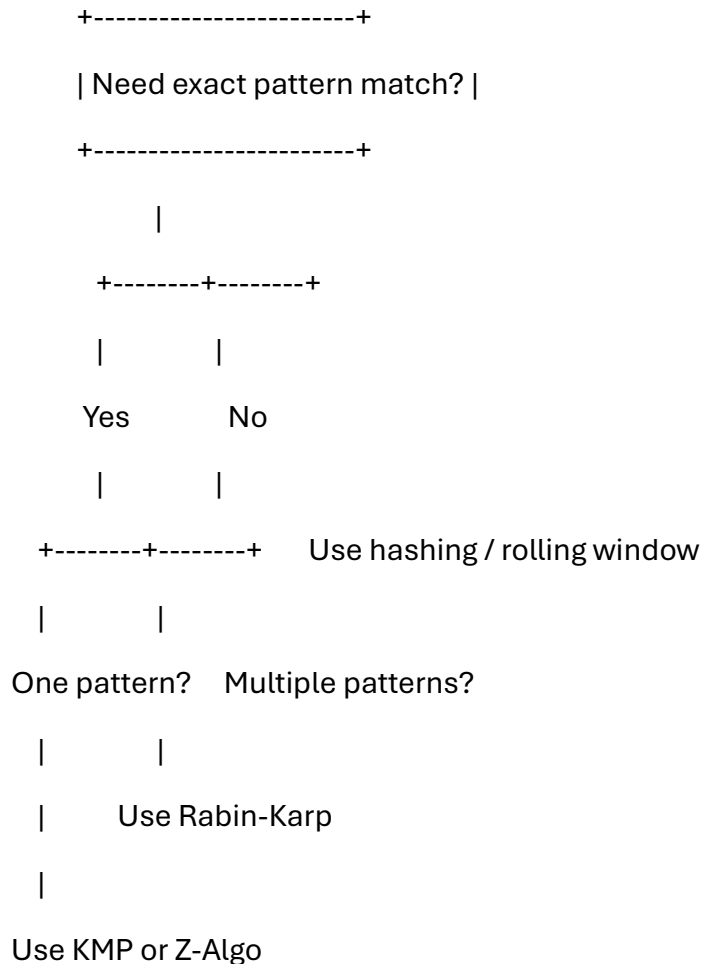
1. KMP is the most frequently asked
 2. Z Algorithm often used in advanced problems
 3. Rabin-Karp used for:
 - plagiarism detection
 - substring search in huge texts
 - multiple pattern matching
-

SECTION G: COMMON MISTAKES

1. Incorrect handling of LPS backtracking
2. Not managing hash overflow in Rabin-Karp
3. Incorrect window shifts in Z algorithm
4. Forgetting the separator ('\$') in Z-search

5. Using substring in loops causing $O(n^2)$ performance in naive solutions

SECTION H: SUMMARY FLOWCHART FOR CHOOSING ALGORITHM



SECTION A: INTRODUCTION TO GRAPHS

1. What is a Graph?

A graph is a collection of:

- Nodes (Vertices)
- Edges connecting nodes

Useful for modeling:

- Roads
- Networks
- Social connections

- Dependencies
 - Maps
 - Electricity grids
 - Flight routes
-

2. Types of Graphs

Type	Description
Directed	Edge has direction ($u \rightarrow v$)
Undirected	No direction ($u - v$)
Weighted	Edge has a cost/weight
Unweighted	No weights
Cyclic	Contains cycles
Acyclic	No cycles
DAG	Directed Acyclic Graph
Connected	Path exists between every pair
Disconnected	Otherwise

3. Graph Representations

1. Adjacency List

Most common, efficient for sparse graphs.

$0 \rightarrow [1,3]$

$1 \rightarrow [0,2]$

$2 \rightarrow [1]$

$3 \rightarrow [0]$

2. Adjacency Matrix

2D matrix of size $V \times V$.

Useful for dense graphs.

0 1 2 3

0 0 1 0 1

1 1 0 1 0

2 0 1 0 0

3 1 0 0 0

3. Edge List

List of edges:

(0,1), (1,2), (0,3)

SECTION B: BFS (BREADTH-FIRST SEARCH)

4. What is BFS?

BFS explores nodes level by level using a queue.

Used for:

- Shortest path in unweighted graphs
 - Level order traversal
 - Detecting connected components
 - Cycle detection in undirected graphs
-

5. BFS Algorithm (Java)

```
void bfs(int start, List<List<Integer>> graph) {  
    boolean[] visited = new boolean[graph.size()];  
    Queue<Integer> q = new LinkedList<>();  
  
    visited[start] = true;  
    q.add(start);  
  
    while (!q.isEmpty()) {
```

```
int node = q.poll();

System.out.print(node + " ");

for (int neighbor : graph.get(node)) {
    if (!visited[neighbor]) {
        visited[neighbor] = true;
        q.add(neighbor);
    }
}
}
```

6. BFS Time Complexity

$O(V + E)$

SECTION C: DFS (DEPTH-FIRST SEARCH)

7. What is DFS?

DFS explores deep before wide.

Uses a stack or recursion.

Applications:

- Path finding
 - Cycle detection
 - Topological sort
 - Strongly connected components
 - Detecting bridges and articulation points
-

8. DFS Algorithm (Java)

```
void dfs(int node, boolean[] visited, List<List<Integer>> graph) {  
    visited[node] = true;  
    System.out.print(node + " ");  
  
    for (int neighbor : graph.get(node)) {  
        if (!visited[neighbor])  
            dfs(neighbor, visited, graph);  
    }  
}
```

Time Complexity

$O(V + E)$

SECTION D: CYCLE DETECTION

9. Cycle Detection in Undirected Graph (DFS)

```
boolean dfsCycle(int node, int parent, boolean[] visited, List<List<Integer>> g) {  
    visited[node] = true;  
  
    for (int nei : g.get(node)) {  
        if (!visited[nei]) {  
            if (dfsCycle(nei, node, visited, g))  
                return true;  
        } else if (nei != parent)  
            return true;  
    }  
    return false;  
}
```

10. Cycle Detection in Directed Graph (DFS + recursion stack)

```
boolean dfsDirected(int node, boolean[] visited, boolean[] rec, List<List<Integer>> g) {  
    visited[node] = true;  
    rec[node] = true;  
  
    for (int nei : g.get(node)) {  
        if (!visited[nei] && dfsDirected(nei, visited, rec, g))  
            return true;  
        else if (rec[nei])  
            return true;  
    }  
  
    rec[node] = false;  
    return false;  
}
```

SECTION E: TOPOLOGICAL SORT

11. What is Topological Sorting?

A linear ordering of nodes such that:

If there is an edge $u \rightarrow v$, then u appears before v .

Only possible for DAGs.

12. Topological Sort Using DFS

```
void topoDFS(int node, boolean[] visited, Stack<Integer> st, List<List<Integer>> g) {  
    visited[node] = true;
```

```

    for (int nei : g.get(node))
        if (!visited[nei])
            topoDFS(nei, visited, st, g);

    st.push(node);
}

```

13. Topological Sort Using BFS (Kahn's Algorithm)

```

List<Integer> topoSort(int V, List<List<Integer>> g) {
    int[] indegree = new int[V];
    for (int i = 0; i < V; i++)
        for (int nei : g.get(i))
            indegree[nei]++;

    Queue<Integer> q = new LinkedList<>();
    for (int i = 0; i < V; i++)
        if (indegree[i] == 0)
            q.add(i);

    List<Integer> ans = new ArrayList<>();

    while (!q.isEmpty()) {
        int node = q.poll();
        ans.add(node);

        for (int nei : g.get(node)) {
            indegree[nei]--;
            if (indegree[nei] == 0)

```



```

        q.add(nei);
    }
}
return ans;
}

```

SECTION F: MINIMUM SPANNING TREE (MST)

Two famous algorithms:

- Kruskal's
 - Prim's
-

14. KRUSKAL'S ALGORITHM

Kruskal's uses **sorting + DSU**.

Steps:

1. Sort edges by weight
 2. Use Disjoint Set Union (Union-Find)
 3. Add edges that don't create cycles
-

Java Code – Kruskal

```

class Edge {
    int u, v, w;

    Edge(int u, int v, int w) {
        this.u = u; this.v = v; this.w = w;
    }
}

int kruskal(int V, List<Edge> edges) {
    Collections.sort(edges, (a, b) -> a.w - b.w);
}

```

```

DSU dsu = new DSU(V);

int cost = 0;

for (Edge e : edges) {
    if (dsu.find(e.u) != dsu.find(e.v)) {
        cost += e.w;
        dsu.union(e.u, e.v);
    }
}

return cost;
}

```

15. PRIM'S ALGORITHM

Works like Dijkstra but only for MST.

```

int prim(int V, List<List<int[]>> graph) {
    boolean[] visited = new boolean[V];

    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);

    pq.add(new int[]{0, 0}); // node, weight

    int cost = 0;

    while (!pq.isEmpty()) {
        int[] top = pq.poll();

        int node = top[0], wt = top[1];

        if (visited[node]) continue;

        visited[node] = true;
    }
}

```

```
cost += wt;

for (int[] nei : graph.get(node)) {
    pq.add(nei);
}
}
return cost;
}
```

SECTION G: SHORTEST PATH ALGORITHMS

16. BFS Shortest Path (Unweighted Graph)

```
int[] shortestPath(int src, int V, List<List<Integer>> g) {
    int[] dist = new int[V];
    Arrays.fill(dist, -1);
    Queue<Integer> q = new LinkedList<>();

    q.add(src);
    dist[src] = 0;

    while (!q.isEmpty()) {
        int node = q.poll();
        for (int nei : g.get(node)) {
            if (dist[nei] == -1) {
                dist[nei] = dist[node] + 1;
                q.add(nei);
            }
        }
    }
}
```

```
}  
return dist;  
}
```

17. DIJKSTRA'S ALGORITHM (Weighted Graph, No Negative Weights)

```
int[] dijkstra(int V, List<List<int[]>> g, int src) {  
    int[] dist = new int[V];  
    Arrays.fill(dist, Integer.MAX_VALUE);  
    dist[src] = 0;  
  
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);  
    pq.add(new int[]{src, 0});  
  
    while (!pq.isEmpty()) {  
        int[] curr = pq.poll();  
        int node = curr[0], d = curr[1];  
  
        if (d > dist[node]) continue;  
  
        for (int[] nei : g.get(node)) {  
            int v = nei[0], w = nei[1];  
  
            if (dist[node] + w < dist[v]) {  
                dist[v] = dist[node] + w;  
                pq.add(new int[]{v, dist[v]});  
            }  
        }  
    }  
}
```

```
    return dist;
}
```

18. BELLMAN-FORD (Handles Negative Weights)

Time: $O(V \times E)$

```
int[] bellmanFord(int V, List<Edge> edges, int src) {
    int[] dist = new int[V];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;

    for (int i = 1; i < V; i++) {
        for (Edge e : edges) {
            if (dist[e.u] != Integer.MAX_VALUE &&
                dist[e.u] + e.w < dist[e.v]) {
                dist[e.v] = dist[e.u] + e.w;
            }
        }
    }

    return dist;
}
```

Detecting negative cycles:

Run one more iteration. If distance improves $\rightarrow$ cycle exists.

19. FLOYD-WARSHALL (All-Pairs Shortest Paths)

Dynamic Programming algorithm for all pairs shortest path.

Time: $O(n^3)$

```
void floydWarshall(int[][] dist) {
    int V = dist.length;
```

```

for (int k = 0; k < V; k++)
    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            if (dist[i][k] + dist[k][j] < dist[i][j])
                dist[i][j] = dist[i][k] + dist[k][j];
}

```

SECTION H: GRAPH CONNECTIVITY

20. Connected Components in Undirected Graph (DFS)

```

int countComponents(int V, List<List<Integer>> g) {
    boolean[] visited = new boolean[V];
    int count = 0;

    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            dfs(i, visited, g);
            count++;
        }
    }

    return count;
}

```

SECTION I: DIRECTED GRAPH COMPONENTS

21. KOSARAJU'S ALGORITHM (Strongly Connected Components)

Three steps:

1. DFS to compute finishing times
2. Reverse the graph
3. DFS in finishing time order

SECTION J: GRAPH COMPLEXITIES

Algorithm	Complexity
BFS	$O(V + E)$
DFS	$O(V + E)$
Dijkstra	$O((V+E) \log V)$
Bellman-Ford	$O(V \times E)$
Floyd-Warshall	$O(V^3)$
Kruskal	$O(E \log E)$
Prim	$O(E \log V)$

SECTION K: INTERVIEW INSIGHTS

1. BFS and DFS are the most fundamental
2. Dijkstra is asked very frequently
3. Topological sort important for dependencies
4. MST problems come up often in system design
5. Bellman-Ford tests negative cycle understanding
6. Floyd-Warshall is asked in DP + Graph combo
7. Cycle detection is a classic question

SECTION A: INTRODUCTION TO DYNAMIC PROGRAMMING

Dynamic Programming (DP) is a technique for solving problems with:

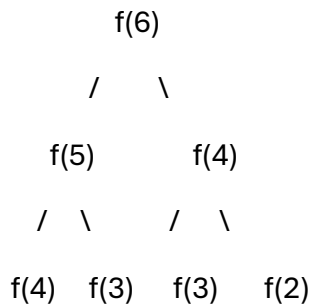
1. **Overlapping subproblems**
2. **Optimal substructure**

DP improves brute force recursion by storing already computed results.

Why DP is Necessary

Consider Fibonacci:

Brute force recursion calls the same values many times:



Nodes repeat. This wastes exponential time.

DP eliminates this repeated work.

SECTION B: PROPERTIES OF DP

1. Overlapping Subproblems

A problem can be broken into subproblems that repeat.

Example: Fibonacci, Knapsack, LIS, Coin Change

2. Optimal Substructure

The optimal solution depends on optimal solutions of subproblems.

Example: Shortest path, DP on trees, DP on grids

SECTION C: DP TECHNIQUES

There are **three** DP styles:

1. **Memoization (Top-Down)**

Recursion + cache

2. **Tabulation (Bottom-Up)**

Iterative DP table

3. **Space Optimization**

Convert 2D → 1D when only previous row/column matters

Example Problem: Fibonacci

1. Recursion (Exponential)

```
int fib(int n) {  
    if (n <= 1) return n;  
    return fib(n-1) + fib(n-2);  
}
```

Time: $O(2^n)$

2. Memoization (Top-Down)

```
int fib(int n, int[] dp) {  
    if (n <= 1) return n;  
    if (dp[n] != -1) return dp[n];  
  
    dp[n] = fib(n-1, dp) + fib(n-2, dp);  
    return dp[n];  
}
```

Time: $O(n)$

Space: $O(n)$

3. Tabulation (Bottom-Up)

```
int fib(int n) {  
    int[] dp = new int[n+1];  
    dp[0] = 0;  
    dp[1] = 1;  
  
    for (int i = 2; i <= n; i++)  
        dp[i] = dp[i-1] + dp[i-2];  
}
```

```
    return dp[n];  
}
```

4. Space-Optimized DP

```
int fib(int n) {  
    int a = 0, b = 1;  
    for (int i = 2; i <= n; i++) {  
        int c = a + b;  
        a = b;  
        b = c;  
    }  
    return b;  
}
```

SECTION D: DP ON GRIDS

Classic category.

Problems:

1. Unique paths
 2. Minimum path sum
 3. Grid with obstacles
 4. Robot paths
-

1. Unique Paths ($m \times n$ grid)

A robot starts at (0,0) and can move only:

- Right
 - Down
-

Recurrence

$$dp[i][j] = dp[i-1][j] + dp[i][j-1]$$

Java (Tabulation)

```
int uniquePaths(int m, int n) {  
    int[][] dp = new int[m][n];  
  
    for (int i = 0; i < m; i++)  
        dp[i][0] = 1;  
  
    for (int j = 0; j < n; j++)  
        dp[0][j] = 1;  
  
    for (int i = 1; i < m; i++)  
        for (int j = 1; j < n; j++)  
            dp[i][j] = dp[i-1][j] + dp[i][j-1];  
  
    return dp[m-1][n-1];  
}
```

2. Minimum Path Sum

Given a grid with costs, find minimum sum path (only right/down).

$$dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1])$$

SECTION E: 0/1 KNAPSACK

Most famous DP problem.

Problem

Given:

- Weight array $w[]$
- Value array $v[]$
- Capacity C

Choose items such that:

- Total weight $\leq C$
- Value maximized

Choices:

1. Include
2. Exclude

Recurrence

if ($w[i] \leq C$):

```
dp[i][C] = max(  
    v[i] + dp[i-1][C-w[i]], // take  
    dp[i-1][C]             // skip  
)
```

else:

```
dp[i][C] = dp[i-1][C]    // cannot take
```

Java Implementation

```
int knapsack(int[] wt, int[] val, int W, int n) {
```

```
    int[][] dp = new int[n+1][W+1];
```

```
    for (int i = 1; i <= n; i++) {
```

```
        for (int w = 1; w <= W; w++) {
```

```

        if (wt[i-1] <= w) {
            dp[i][w] = Math.max(
                val[i-1] + dp[i-1][w - wt[i-1]],
                dp[i-1][w]
            );
        } else {
            dp[i][w] = dp[i-1][w];
        }
    }
}

return dp[n][W];
}

```

Time: $O(nW)$
 Space: $O(nW)$

SECTION F: UNBOUNDED KNAPSACK

Similar to knapsack but items can repeat unlimited times.

Recurrence:

```

dp[i][w] = max(
    val[i] + dp[i][w - wt[i]], // take (stay on same i)
    dp[i-1][w]                // skip
)

```

SECTION G: LONGEST INCREASING SUBSEQUENCE (LIS)

Extremely important interview question.

1. DP Approach ($O(n^2)$)

```
int lis(int[] arr) {  
    int n = arr.length;  
    int[] dp = new int[n];  
    Arrays.fill(dp, 1);  
  
    for (int i = 1; i < n; i++) {  
        for (int j = 0; j < i; j++) {  
            if (arr[i] > arr[j])  
                dp[i] = Math.max(dp[i], dp[j] + 1);  
        }  
    }  
  
    int ans = 0;  
    for(int x : dp) ans = Math.max(ans, x);  
    return ans;  
}
```

Time: $O(n^2)$

SECTION H: LONGEST COMMON SUBSEQUENCE (LCS)

Another fundamental DP.

Recurrence

```
if (s1[i] == s2[j])  
    dp[i][j] = 1 + dp[i-1][j-1]  
else  
    dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

Java Code

```
int lcs(String a, String b) {  
    int n = a.length(), m = b.length();  
    int[][] dp = new int[n+1][m+1];  
  
    for (int i = 1; i <= n; i++) {  
        for (int j = 1; j <= m; j++) {  
  
            if (a.charAt(i-1) == b.charAt(j-1)) {  
                dp[i][j] = 1 + dp[i-1][j-1];  
            } else {  
                dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);  
            }  
  
        }  
    }  
    return dp[n][m];  
}
```

SECTION I: EDIT DISTANCE (Levenshtein Distance)

Operations:

1. Insert
2. Delete
3. Replace

Recurrence:

if same:

$$dp[i][j] = dp[i-1][j-1]$$

else:

```
dp[i][j] = 1 + min(  
    dp[i-1][j], // delete  
    dp[i][j-1], // insert  
    dp[i-1][j-1] // replace  
)
```

SECTION J: PALINDROMIC DP

Examples:

1. Longest Palindromic Substring
 2. Longest Palindromic Subsequence
 3. Minimum deletions to make palindrome
 4. Partitioning into palindromes
-

Longest Palindromic Subsequence (LPS)

LPS = LCS(s, reverse(s))

SECTION K: DP ON SUBSETS (Bitmask DP)

Important for:

- Traveling Salesman
 - Maximum compatible jobs
 - Counting subsets with property
-

SECTION L: DP ON TREES

Tree DP is used in:

- Counting nodes
- Maximum path sum
- Diameter of tree
- Matching problems

SECTION M: DP OPTIMIZATION (Advanced)

Not needed for beginners, but important for advanced interviews:

1. Space optimization
 2. Knuth optimization
 3. Divide and conquer optimization
 4. DP with monotonic queue
 5. Convex hull trick
 6. Matrix exponentiation
-

SECTION N: COMMON MISTAKES IN DP

1. Incorrect base cases
 2. Not identifying overlapping subproblems
 3. Using recursion where tabulation is easier
 4. Wrong loop ordering
 5. Not understanding state transitions
 6. Using 2D DP when 1D is enough
 7. Forgetting modulo operations
-

SECTION O: INTERVIEW INSIGHTS

1. Always write recurrence first
 2. Always identify states clearly
 3. For strings, DP table is usually $(n+1) \times (m+1)$
 4. For knapsack, loops are crucial
 5. Memorize patterns
 6. Always test with small sample inputs
 7. Use print-table to debug transitions
-

SECTION P: DP PROBLEM PATTERNS

This list is extremely important.

1. Subsequence-Based DP

- LCS
- LIS
- Edit Distance
- Minimum deletions
- Count subsequences

State: $dp[i][j]$ or $dp[i]$

2. Knapsack-Based DP

- 0/1 Knapsack
- Unbounded Knapsack
- Coin Change
- Rod Cutting

State: $dp[i][capacity]$

3. Grid-Based DP

- Unique Paths
- Minimum Path Sum
- Grid with obstacles

State: $dp[row][col]$

4. Partition Based

- Palindrome partitioning
- Equal sum subset
- Partition array

5. DP on Digits

Important for competitive programming.

6. DP on Trees

- Maximum sum
 - Diameter
-

7. Bitmask DP

- Traveling Salesman
- Maximum score assignment

SECTION A: WHAT IS THE GREEDY APPROACH?

A greedy algorithm makes a decision that looks best **at the current moment**, hoping it leads to the global optimum.

Definition:

A greedy algorithm builds a solution step by step by choosing the **locally optimal** choice at each stage, without reconsidering previous choices.

Key requirement for correctness:

The problem must satisfy:

1. **Optimal Substructure**
2. **Greedy Choice Property**

If **any** of these fail → greedy fails → must use DP.

SECTION B: DP vs GREEDY – VERY IMPORTANT

Feature	Greedy	Dynamic Programming
Decision basis	Local optimum	All possibilities considered
Backtracking	Not allowed	Allowed through recursion
Uses memory table	No	Yes

Feature	Greedy	Dynamic Programming
Works for	Special cases only	Broad class of problems
Speed	Very fast	Slower
Guarantees optimal?	Only if greedy-choice property holds	Always

Important note:
Many greedy failures come from choosing the wrong local rule.

SECTION C: STRUCTURE OF GREEDY PROBLEMS

Greedy algorithms follow a pattern:

1. Choose the best available option now
2. Update the system
3. Repeat until final solution

Example: Picking the shortest job first or picking the earliest finishing interval.

SECTION D: ACTIVITY SELECTION PROBLEM

(Maximum number of non-overlapping intervals)

This is the foundational greedy problem.

Problem

Given `start[]` and `end[]` times of activities. Select maximum number of activities that do not overlap.

Greedy Strategy

Sort activities by **end time** (earliest finishing first).

Reason:

- Choosing earliest finishing leaves room for more future activities
 - A mathematical proof exists (interval scheduling theorem)
-

Java Implementation

```
class Activity {  
    int start, end;  
    Activity(int s, int e) { start = s; end = e; }  
}  
  
int maxActivities(List<Activity> acts) {  
  
    Collections.sort(acts, (a, b) -> a.end - b.end);  
  
    int count = 1;  
    int lastEnd = acts.get(0).end;  
  
    for (int i = 1; i < acts.size(); i++) {  
        if (acts.get(i).start >= lastEnd) {  
            count++;  
            lastEnd = acts.get(i).end;  
        }  
    }  
    return count;  
}
```

Time Complexity

$O(n \log n)$ for sorting.

SECTION E: INTERVAL SCHEDULING VS INTERVAL PARTITIONING

1. Interval scheduling: Select maximum non-overlapping intervals

2. Interval partitioning: Minimum number of rooms required (e.g., meeting rooms problem)

Meeting Rooms (Minimum Rooms Required)

Use a min-heap storing end times.

```
int minRooms(int[][] meetings) {  
    Arrays.sort(meetings, (a,b)->a[0]-b[0]);  
  
    PriorityQueue<Integer> pq = new PriorityQueue<>();  
  
    for (int[] m : meetings) {  
        if (!pq.isEmpty() && pq.peek() <= m[0])  
            pq.poll();  
        pq.add(m[1]);  
    }  
    return pq.size();  
}
```

SECTION F: FRACTIONAL KNAPSACK (Greedy Works Here)

Unlike 0/1 knapsack, here we can take fractional part.

Rule:

Choose items with **highest value/weight ratio** first.

Java Implementation

```
class Item {  
    int wt, val;  
    Item(int w, int v) { wt = w; val = v; }  
}
```

```

double fractionalKS(List<Item> items, int W) {

    items.sort((a, b) ->
        Double.compare((double)b.val/b.wt, (double)a.val/a.wt)
    );

    double totalVal = 0;

    for (Item it : items) {
        if (it.wt <= W) {
            totalVal += it.val;
            W -= it.wt;
        } else {
            totalVal += ((double) it.val / it.wt) * W;
            break;
        }
    }

    return totalVal;
}

```

Time: $O(n \log n)$

SECTION G: JOB SEQUENCING WITH DEADLINES

Given:

Each job has:

- Profit
- Deadline

You must schedule jobs in slots so that profit is maximized
(one job per slot, job takes 1 unit time)

Greedy Strategy:

1. Sort jobs by **highest profit first**
 2. Use a slot array to schedule jobs before their deadlines
-

Java Implementation

```
class Job {  
    int id, profit, deadline;  
    Job(int id, int p, int d) { this.id = id; profit = p; deadline = d; }  
}  
  
int maxProfit(List<Job> jobs) {  
    Collections.sort(jobs, (a,b)->b.profit - a.profit);  
  
    int maxDeadline = 0;  
    for (Job j : jobs) maxDeadline = Math.max(maxDeadline, j.deadline);  
  
    int[] slot = new int[maxDeadline+1];  
    Arrays.fill(slot, -1);  
  
    int totalProfit = 0;  
  
    for (Job j : jobs) {  
        for (int t = j.deadline; t > 0; t--) {  
            if (slot[t] == -1) {  
                slot[t] = j.id;  
                totalProfit += j.profit;  
                break;  
            }  
        }  
    }  
}
```



```
}  
  
return totalProfit;  
  
}
```

SECTION H: GREEDY FOR MINIMUM SPANNING TREE

Two greedy algorithms:

1. Kruskal
2. Prim

We already covered MST in the Graphs chapter.
Here we focus on the greedy nature.

Why Greedy Works for MST?

MST has a property:

- The cheapest available edge that does not form a cycle is always part of the MST.

This is called **cut property**.

Both Prim's and Kruskal's use the cut property.

SECTION I: HUFFMAN CODING (Optimal Lossless Compression)

Used in:

- File compression
- JPEG
- ZIP
- GZIP
- Multimedia compression

Greedy strategy:

Always combine two smallest frequency nodes.

Use a min-heap.

Java Implementation (Key Steps Only)

```

class Node {
    char ch;
    int freq;
    Node left, right;

    Node(char c, int f) { ch = c; freq = f; }
    Node(int f, Node l, Node r) { freq = f; left = l; right = r; }
}

Node buildHuffman(Map<Character,Integer> freqMap) {
    PriorityQueue<Node> pq = new PriorityQueue<>((a,b)->a.freq - b.freq);

    for (var e : freqMap.entrySet())
        pq.add(new Node(e.getKey(), e.getValue()));

    while (pq.size() > 1) {
        Node a = pq.poll();
        Node b = pq.poll();
        Node parent = new Node(a.freq + b.freq, a, b);
        pq.add(parent);
    }
    return pq.poll();
}

```

SECTION J: GREEDY FOR PRIORITY SCHEDULING

Examples:

- Shortest Job First
- Round Robin (not greedy)

- Earliest Deadline First
-

SECTION K: GREEDY FOR MINIMUM COINS

Classic interview trap.

If coin system is canonical (like Indian or US coins):

Greedy gives correct minimum coins.

Example:

Denominations: 1, 2, 5, 10

Change: 18

Choose: $10 + 5 + 2 + 1$

But greedy **fails** for some custom denominations.

Example:

Coins: 1, 4, 6

Change: 8

Greedy chooses $6 + 1 + 1 = 3$ coins

Optimal is $4 + 4 = 2$ coins

Therefore:

Greedy does not universally work for coin change.

SECTION L: COMMON MISTAKES STUDENTS MAKE

1. Assuming greedy is always optimal
 2. Forgetting to sort
 3. Choosing wrong local rule
 4. Mixing greedy and DP incorrectly
 5. Not proving that greedy choice works
 6. Using greedy for knapsack (0/1) which fails
 7. Confusing interval scheduling with interval partitioning
-

SECTION M: INTERVIEW INSIGHTS

1. If a problem asks for:
 - Maximum tasks

- Minimum resources
 - Maximum non-overlapping intervals
→ greedy likely works
2. If problem asks for:
- Combination of choices
 - Maximum/minimum with constraints
→ DP is more likely
3. In greedy questions, interviewer checks:
- Your reasoning for the greedy rule
 - Understanding of why sorting is required
 - Whether you can detect counterexamples
4. Classic interview greedy problems:
- Activity selection
 - Job sequencing
 - Fractional knapsack
 - Meeting rooms
 - Huffman coding
 - Minimum platforms (trains)
 - Gas station problem
 - Candy distribution
 - Jump game

SECTION N: SUMMARY TABLE

Problem	Greedy Rule
Activity Selection	Choose earliest finishing interval
Fractional Knapsack	Choose by highest value/weight
Job Scheduling	Choose by highest profit

Problem	Greedy Rule
Meeting Rooms	Use min-heap of end times
Huffman Coding	Merge two smallest frequencies
MST (Kruskal)	Pick smallest non-cycling edge
MST (Prim)	Pick smallest edge from visited → unvisited
Gas Station	Track deficit and candidate start
Candy Distribution	Left-to-right then right-to-left passes

SECTION A: INTRODUCTION

Backtracking is a **systematic way of exploring all possibilities** of a problem by:

1. Choosing an option
2. Exploring deeper
3. Undoing the choice
4. Trying the next option

It is essentially:

DFS + undo operations

Backtracking = Recursion + State Reversal

A recursive decision is made.

If the decision does not lead to a solution, we reverse the decision and try another.

This is similar to exploring branches of a decision tree.

Real World Analogy

Imagine navigating a maze:

- Move forward
- If you hit a dead end, backtrack and try a different direction
- Continue until the entire maze is explored or the exit is found

This is exactly how backtracking algorithms work.

SECTION B: GENERAL FORM OF BACKTRACKING

Every backtracking problem follows this template:

```
void backtrack(params...) {  
    if (solution found) {  
        store/print solution;  
        return;  
    }  
  
    for (choice in all possible choices) {  
        make(choice);  
        backtrack(next state);  
        undo(choice); // backtrack  
    }  
}
```

SECTION C: WHY BACKTRACKING IS EXPONENTIAL?

Because it tries all combinations/permutations in the worst case.

Examples:

Problem	Time Complexity
---------	-----------------

N-Queens	$O(N!)$
----------	---------

Permutations	$O(N!)$
--------------	---------

Subsets	$O(2^n)$
---------	----------

Sudoku	$O(9^{81})$ worst case
--------	------------------------

Word search	$O(4^n)$
-------------	----------

Backtracking often provides the **only feasible structured solution** for exponential problems.

SECTION D: CLASSIC BACKTRACKING PROBLEMS

We will cover:

1. Generating subsets
 2. Generating permutations
 3. N-Queens
 4. Sudoku solver
 5. Rat in a maze
 6. Word search
 7. Palindrome partitioning
 8. Combination Sum
 9. Phone keypad combinations
 10. Letter case permutations
-

SECTION E: SUBSETS (POWER SET)

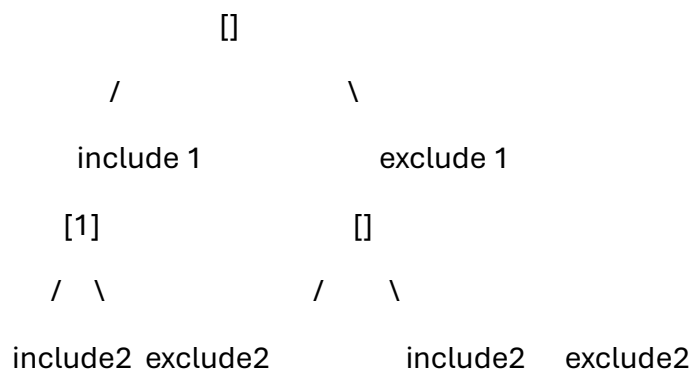
Given `nums[]`, generate all subsets.

Subset generation is classic:

At every element, we have two choices:

- Include
 - Exclude
-

Recursion Tree Example (`nums = [1,2]`)



Java Implementation

```
void subsets(int[] nums, int index, List<Integer> curr, List<List<Integer>> ans) {  
    if (index == nums.length) {  
        ans.add(new ArrayList<>(curr));  
        return;  
    }  
  
    curr.add(nums[index]);  
    subsets(nums, index+1, curr, ans);  
  
    curr.remove(curr.size()-1);  
    subsets(nums, index+1, curr, ans);  
}
```

Time: $O(2^n)$

SECTION F: PERMUTATIONS

At each position, pick an unused number.

Java Implementation

```
void permute(int[] nums, boolean[] used, List<Integer> curr, List<List<Integer>> ans) {  
    if (curr.size() == nums.length) {  
        ans.add(new ArrayList<>(curr));  
        return;  
    }  
  
    for (int i = 0; i < nums.length; i++) {  
        if (used[i]) continue;
```



```

        used[i] = true;
        curr.add(nums[i]);

        permute(nums, used, curr, ans);

        curr.remove(curr.size()-1);
        used[i] = false;
    }
}
Time: O(n!)

```

SECTION G: COMBINATION SUM

Given candidate numbers, find combinations that sum to target.
Elements can repeat.

Greedy fails. DP possible.
Backtracking is most intuitive.

Java Implementation

```

void combSum(int[] arr, int target, int index, List<Integer> curr, List<List<Integer>> ans) {
    if (target == 0) {
        ans.add(new ArrayList<>(curr));
        return;
    }
    if (target < 0) return;

    for (int i = index; i < arr.length; i++) {
        curr.add(arr[i]);
        combSum(arr, target-arr[i], i, curr, ans); // can reuse i
        curr.remove(curr.size()-1);
    }
}

```

```
}  
}
```

Time: Exponential

SECTION H: N-QUEENS

One of the most important backtracking problems.

Place N queens such that no two queens attack each other:

- Same row
 - Same column
 - Same diagonal
-

Board Diagram (4x4)

```
. Q . .  
. . . Q  
Q . . .  
. . Q .
```

Safety Check

For row r, col c:

- Check same column
 - Check 45-degree diagonal
 - Check 135-degree diagonal
-

Java Implementation

```
boolean isSafe(char[][] board, int row, int col, int n) {  
    for (int i = 0; i < row; i++)  
        if (board[i][col] == 'Q') return false;
```

```

    for (int i=row, j=col; i>=0 && j>=0; i--, j--)
        if (board[i][j] == 'Q') return false;

    for (int i=row, j=col; i>=0 && j<n; i--, j++)
        if (board[i][j] == 'Q') return false;

    return true;
}

void solve(int row, char[][] board, int n, List<List<String>> ans) {
    if (row == n) {
        List<String> sol = new ArrayList<>();
        for (char[] r : board) sol.add(new String(r));
        ans.add(sol);
        return;
    }

    for (int col = 0; col < n; col++) {
        if (isSafe(board, row, col, n)) {
            board[row][col] = 'Q';
            solve(row+1, board, n, ans);
            board[row][col] = '.';
        }
    }
}

```

SECTION I: SUDOKU SOLVER

Sudoku is a constraint satisfaction problem:

- 9×9 grid
- Fill numbers 1–9
- Each row, column, 3×3 box must be unique

Java Implementation (Core)

```
boolean solveSudoku(char[][] board) {  
    for (int r = 0; r < 9; r++) {  
        for (int c = 0; c < 9; c++) {  
  
            if (board[r][c] == '.') {  
  
                for (char d = '1'; d <= '9'; d++) {  
                    if (isValid(board, r, c, d)) {  
                        board[r][c] = d;  
  
                        if (solveSudoku(board))  
                            return true;  
  
                        board[r][c] = '.';  
                    }  
                }  
                return false;  
            }  
        }  
    }  
    return true;  
}
```

SECTION J: WORD SEARCH (Matrix)

Given a board, check if word exists by moving horizontally or vertically.

Java Implementation

```
boolean dfs(char[][] b, int i, int j, String w, int idx) {  
    if (idx == w.length()) return true;  
  
    if (i < 0 || j < 0 || i >= b.length || j >= b[0].length)  
        return false;  
  
    if (b[i][j] != w.charAt(idx))  
        return false;  
  
    char temp = b[i][j];  
    b[i][j] = '#';  
  
    boolean found =  
        dfs(b, i+1, j, w, idx+1) ||  
        dfs(b, i-1, j, w, idx+1) ||  
        dfs(b, i, j+1, w, idx+1) ||  
        dfs(b, i, j-1, w, idx+1);  
  
    b[i][j] = temp;  
    return found;  
}
```

SECTION K: PALINDROME PARTITIONING

Partition string into palindromes.

Example:

"aab"

Subsets:

- "a", "a", "b"
- "aa", "b"

Java Implementation

```
void partition(String s, int index, List<String> curr, List<List<String>> ans) {  
    if (index == s.length()) {  
        ans.add(new ArrayList<>(curr));  
        return;  
    }  
  
    for (int i = index; i < s.length(); i++) {  
        if (isPal(s, index, i)) {  
            curr.add(s.substring(index, i+1));  
            partition(s, i+1, curr, ans);  
            curr.remove(curr.size()-1);  
        }  
    }  
}
```

SECTION L: PHONE KEYPAD COMBINATIONS

Digits to letters.

Classic for recursion trees.

Java Implementation

```
String[] map = {  
    "", "", "abc", "def", "ghi", "jkl",
```

```

    "mno", "pqrs", "tuv", "wxyz"
};

void phone(String digits, int index, StringBuilder sb, List<String> ans) {
    if (index == digits.length()) {
        ans.add(sb.toString());
        return;
    }

    String letters = map[digits.charAt(index)-'0'];
    for (char ch : letters.toCharArray()) {
        sb.append(ch);
        phone(digits, index+1, sb, ans);
        sb.deleteCharAt(sb.length()-1);
    }
}

```

SECTION M: TEMPLATE FOR ALL BACKTRACKING PROBLEMS

To solve any backtracking problem:

1. Define the state
2. Define the choices
3. Define the goal condition
4. Ensure to undo the choice

GENERAL TEMPLATE

```

void backtrack(State st) {

    if (isGoal(st)) {

```

```
    save(st);  
    return;  
}  
  
for (Choice ch : choices(st)) {  
  
    apply(ch, st);  
  
    backtrack(st);  
  
    undo(ch, st);  
}  
}
```

This structure works universally.

SECTION N: OPTIMIZATIONS (PRUNING)

Backtracking becomes faster if we prune invalid branches early.

Examples:

1. If current sum > target → stop
 2. If no more possibility can beat current best → stop
 3. Fail early when constraint violates (Sudoku, N-Queens)
-

SECTION O: COMMON MISTAKES

1. Forgetting to undo state
2. Not checking bounds
3. Recursion that never stops
4. Using global variables incorrectly
5. Using list reference directly (must clone)

6. Bad ordering of choices

SECTION P: INTERVIEW INSIGHTS

1. Backtracking is mainly tested on:
 - N-Queens
 - Sudoku
 - Subsets/permutations
 - Word Search
 - Palindrome Partition
 - Combination Sum
2. Interviewer checks:
 - Recursion understanding
 - Correct pruning
 - Clean code
 - Handling adjacency (board problems)
 - Using visited/non-visited logic
3. Always discuss:
 - Why backtracking is needed
 - Why greedy or DP won't work
 - Time complexity (exponential)

SECTION A: INTRODUCTION

Sliding window is an optimization technique used for:

1. Subarray problems
2. Substring problems
3. Problems dealing with continuous segments
4. Frequency-based problems
5. Finding max/min in ranges

It eliminates recomputation by maintaining a window (range) that slides across the array/string.

Key Idea

Instead of recalculating over each subarray, reuse knowledge from the previous window.

Imagine a window frame moving from left to right.

SECTION B: TYPES OF SLIDING WINDOWS

There are **two** major categories:

1. Fixed-size window

Window size is constant (k).

Examples:

- Maximum sum subarray of size k
- First negative number in every window
- Number of occurrences of anagrams of a pattern

2. Variable-size window

Window grows or shrinks depending on conditions.

Examples:

- Longest substring without repetition
 - Longest substring with at most K distinct characters
 - Minimum window substring
 - Smallest subarray with sum $\geq S$
-

SECTION C: FIXED-SIZE SLIDING WINDOW

Example 1: Maximum Sum Subarray of Size K

Given array and k, find maximum sum of any contiguous subarray of size k.

Java Implementation

```
int maxSum(int[] arr, int k) {  
    int windowSum = 0;  
    int maxSum = Integer.MIN_VALUE;  
  
    int start = 0;  
  
    for (int end = 0; end < arr.length; end++) {  
  
        windowSum += arr[end];  
  
        if (end - start + 1 == k) {  
            maxSum = Math.max(maxSum, windowSum);  
  
            windowSum -= arr[start];  
            start++;  
        }  
    }  
    return maxSum;  
}
```

Time Complexity:

$O(n)$

Example 2: First Negative Number in Every Window

Use a queue to store negative numbers' indices.

SECTION D: VARIABLE-SIZED WINDOW

This is the more critical type for interviews.

General pattern:

```
while (end < n) {  
  
    add arr[end] to window  
  
    while (window is invalid)  
        shrink from left  
  
    update result  
  
    end++  
}
```

Example 3: Longest Substring Without Repeating Characters

Classic question.

Use a map or array to track character frequency.

Java Implementation

```
int longestUnique(String s) {  
    int[] freq = new int[256];  
    int left = 0, maxLen = 0;  
  
    for (int right = 0; right < s.length(); right++) {  
  
        freq[s.charAt(right)]++;  
  
        while (freq[s.charAt(right)] > 1) {
```

```

        freq[s.charAt(left)]--;
        left++;
    }

    maxlen = Math.max(maxlen, right - left + 1);
}

return maxlen;
}

```

Example 4: Longest Substring with At Most K Distinct Characters

Maintain count of distinct characters.

Java Implementation

```

int longestKDistinct(String s, int k) {
    int[] freq = new int[256];
    int left = 0, count = 0, maxlen = 0;

    for (int right = 0; right < s.length(); right++) {

        if (freq[s.charAt(right)] == 0)
            count++;
        freq[s.charAt(right)]++;

        while (count > k) {
            freq[s.charAt(left)]--;
            if (freq[s.charAt(left)] == 0)
                count--;
            left++;
        }
    }
}

```

```

    }

    maxLen = Math.max(maxLen, right - left + 1);
}

return maxLen;
}

```

Example 5: Smallest Subarray with Sum $\geq S$

Find shortest window.

Java Implementation

```

int smallestSubarraySum(int[] arr, int S) {
    int left = 0, windowSum = 0;
    int minLen = Integer.MAX_VALUE;

    for (int right = 0; right < arr.length; right++) {
        windowSum += arr[right];

        while (windowSum >= S) {
            minLen = Math.min(minLen, right - left + 1);
            windowSum -= arr[left];
            left++;
        }
    }

    return minLen == Integer.MAX_VALUE ? 0 : minLen;
}

```

Example 6: Minimum Window Substring (Hard)

Given strings s and t, find minimum window in s that contains all characters of t.

This is the most important window question.

Java Implementation (Core Logic)

```
String minWindow(String s, String t) {  
    int[] req = new int[256];  
    for (char c : t.toCharArray()) req[c]++;  
  
    int left = 0, count = t.length();  
    int minLen = Integer.MAX_VALUE, start = 0;  
  
    for (int right = 0; right < s.length(); right++) {  
        if (req[s.charAt(right)] > 0)  
            count--;  
        req[s.charAt(right)]--;  
  
        while (count == 0) {  
            if (right - left + 1 < minLen) {  
                minLen = right - left + 1;  
                start = left;  
            }  
  
            req[s.charAt(left)]++;  
            if (req[s.charAt(left)] > 0)  
                count++;  
            left++;  
        }  
    }  
}
```

```
return minLen == Integer.MAX_VALUE ? "" : s.substring(start, start + minLen);  
}
```

SECTION E: HOW TO IDENTIFY A SLIDING WINDOW PROBLEM

You should apply sliding window when:

1. The problem involves subarrays/substrings
 2. You're looking for longest/shortest/number of something
 3. The array/string is continuous
 4. There is a constraint you need to maintain inside a window
 5. The window expands until invalid, then shrinks
-

SECTION F: COMMON SLIDING WINDOW PATTERNS

Pattern	Example
Fixed size window	Maximum sum subarray
Variable window with constraint	Longest substring without repeating characters
Window requiring frequency check	Anagram search
Window that grows until target	Smallest subarray with sum $\geq S$
Window that shrinks on violation	At most K distinct characters

SECTION G: FIND ALL ANAGRAMS IN STRING

Given s and p, find all start indices where anagram of p appears in s.

Java Implementation

```
List<Integer> findAnagrams(String s, String p) {  
    int[] freq = new int[26];  
    for (char c : p.toCharArray()) freq[c - 'a']++;
```



```

List<Integer> ans = new ArrayList<>();
int left = 0, match = 0;

for (int right = 0; right < s.length(); right++) {

    freq[s.charAt(right) - 'a']--;

    while (freq[s.charAt(right) - 'a'] < 0) {
        freq[s.charAt(left) - 'a']++;
        left++;
    }

    if (right - left + 1 == p.length())
        ans.add(left);
    }
return ans;
}

```

SECTION H: MAXIMUM NUMBER OF VOWELS IN SUBSTRING OF LENGTH K

Example of fixed window.

Java Implementation

```

int maxVowels(String s, int k) {

    int cnt = 0, max = 0;

    int start = 0;

    Set<Character> vow = Set.of('a','e','i','o','u');

```

```

for (int end = 0; end < s.length(); end++) {
    if (vow.contains(s.charAt(end))) cnt++;

    if (end - start + 1 == k) {
        max = Math.max(max, cnt);

        if (vow.contains(s.charAt(start))) cnt--;
        start++;
    }
}
return max;
}

```

SECTION I: COMMON MISTAKES STUDENTS MAKE

1. Forgetting to shrink window for variable-case
 2. Incorrect order of update → check → shrink
 3. Confusing fixed and variable windows
 4. Not resetting frequency arrays properly
 5. Forgetting edge cases (empty string, $k > \text{length}$)
 6. Mismanaging left and right pointers
-

SECTION J: INTERVIEW INSIGHTS

1. Always explain window expansion and contraction logic
2. Talk about what makes the window invalid
3. Prefer array frequency over HashMap for speed
4. For substring problems:
window content = string[left..right]

5. For subarray problems:
windowSum is maintained dynamically
6. Write the condition clearly before coding
7. Use test examples in dry-run during interview

SECTION A: INTRODUCTION

The Two-Pointers technique uses **two indices** to efficiently process:

1. Sorted arrays
 2. Opposite-end processing
 3. Searching for pairs/triplets
 4. Removing duplicates
 5. Checking palindromes
 6. Partitioning arrays
 7. Merge-like operations
-

Why Two-Pointers Works

Because many problems only need **linear scanning** when:

- Data is sorted (array/string)
- You are matching two elements based on a relationship
- You are moving pointers in coordinated ways

Two-pointers often reduces complexity from **$O(n^2)$** to **$O(n)$** .

SECTION B: TYPES OF TWO-POINTER PATTERNS

There are **three major categories**:

1. Opposite-direction pointers

Used when array is sorted or operations begin from both ends.

Example:

Pair sum, palindrome checking.

2. Same-direction pointers

This is Sliding Window (already covered).

Example:

Longest substring without repeating characters.

3. Fast and slow pointers

Used in linked lists, cycles, and array problems.

Example:

Cycle detection, middle of linked list.

SECTION C: OPPOSITE-DIRECTION POINTERS

Example 1: Pair Sum in Sorted Array

Given sorted array `arr[]` and `target`, find if pair exists.

Pointers:

- left starts at 0
- right starts at `n-1`

Move depending on sum.

Java Implementation

```
boolean pairSum(int[] arr, int target) {
```

```
    int left = 0, right = arr.length - 1;
```

```
    while (left < right) {
```

```
        int sum = arr[left] + arr[right];
```

```
        if (sum == target) return true;
```

```
        else if (sum < target) left++;
```

```
        else right--;
```

```
    }
```

```
    return false;
```

```
}
```

Time: $O(n)$

Example 2: Check If String Is Palindrome

Pointers at start and end.

```
boolean isPalindrome(String s) {  
    int left = 0, right = s.length() - 1;  
  
    while (left < right) {  
        if (s.charAt(left) != s.charAt(right)) return false;  
        left++;  
        right--;  
    }  
    return true;  
}
```

Example 3: Reverse Array In-Place

```
void reverse(int[] arr) {  
    int left = 0, right = arr.length - 1;  
    while (left < right) {  
        int t = arr[left];  
        arr[left] = arr[right];  
        arr[right] = t;  
        left++;  
        right--;  
    }  
}
```

Example 4: 3-Sum (Triplets That Sum to Zero)

Important interview problem.

Approach:

1. Sort array
 2. Fix one index i
 3. Use two-pointers (l and r) to find remaining sum
-

Java Implementation

```
List<List<Integer>> threeSum(int[] nums) {  
    Arrays.sort(nums);  
    List<List<Integer>> ans = new ArrayList<>();  
  
    for (int i = 0; i < nums.length - 2; i++) {  
  
        if (i > 0 && nums[i] == nums[i-1]) continue;  
  
        int left = i + 1, right = nums.length - 1;  
  
        while (left < right) {  
            int sum = nums[i] + nums[left] + nums[right];  
  
            if (sum == 0) {  
                ans.add(Arrays.asList(nums[i], nums[left], nums[right]));  
                left++;  
                right--;  
  
                while (left < right && nums[left] == nums[left-1]) left++;  
                while (left < right && nums[right] == nums[right+1]) right--;
```

```
        } else if (sum < 0) left++;  
        else right--;  
    }  
}  
return ans;  
}
```

Example 5: Container With Most Water

Important LeetCode problem.

Two pointers at ends:

- Move pointer pointing to the shorter line
- Because increasing width decreases height

```
int maxArea(int[] h) {  
    int left = 0, right = h.length - 1;  
    int max = 0;  
  
    while (left < right) {  
        int area = Math.min(h[left], h[right]) * (right - left);  
        max = Math.max(max, area);  
  
        if (h[left] < h[right]) left++;  
        else right--;  
    }  
  
    return max;  
}
```

SECTION D: SAME-DIRECTION (SLIDING WINDOW)

Already covered in detail in Part 23.

Focus here is on conceptual connection:

Two-pointers + shrinking/growing = Sliding Window.

Examples:

- Longest substring without repeating characters
 - Minimum window substring
 - Smallest subarray with sum $\geq S$
-

SECTION E: FAST & SLOW POINTERS (Floyd's Cycle Method)

Used for:

1. Detecting loops in linked list
 2. Finding the starting node of loop
 3. Finding the middle of linked list
 4. Checking if number is happy (cycle detection in transformation)
 5. Intersection of linked lists
-

Example 6: Detect Cycle in Linked List

If fast == slow at some point $\rightarrow$ cycle exists.

```
boolean hasCycle(ListNode head) {  
    ListNode slow = head, fast = head;  
  
    while (fast != null && fast.next != null) {  
        slow = slow.next;  
        fast = fast.next.next;  
  
        if (slow == fast) return true;  
    }  
}
```



```
    return false;
}
```

Example 7: Find Middle of Linked List

When fast moves 2×, slow reaches the middle.

```
ListNode middle(ListNode head) {
    ListNode slow = head, fast = head;

    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}
```

Example 8: Cycle Starting Node

Once meeting point found:

- Move slow to head
- Move both one step
- Meeting point = start of cycle

```
ListNode detectCycle(ListNode head) {
    ListNode slow = head, fast = head;

    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) break;
    }
}
```

```
if (fast == null || fast.next == null) return null;

slow = head;

while (slow != fast) {
    slow = slow.next;
    fast = fast.next;
}

return slow;
}
```

SECTION F: MERGE-TYPE TWO POINTERS

Used in:

1. Merging sorted arrays
 2. Merge step in merge sort
 3. Comparing two sorted lists
-

Example 9: Merge Two Sorted Arrays

```
int[] merge(int[] a, int[] b) {
    int i = 0, j = 0, k = 0;
    int[] c = new int[a.length + b.length];

    while (i < a.length && j < b.length) {
        if (a[i] < b[j]) c[k++] = a[i++];
        else c[k++] = b[j++];
    }
}
```

```
while (i < a.length) c[k++] = a[i++];  
while (j < b.length) c[k++] = b[j++];  
  
return c;  
}
```

SECTION G: REMOVE DUPLICATES IN SORTED ARRAY

Two-pointer technique:

- Slow pointer tracks unique position
- Fast pointer scans elements

```
int removeDup(int[] nums) {  
    int slow = 1;  
  
    for (int fast = 1; fast < nums.length; fast++) {  
        if (nums[fast] != nums[fast-1]) {  
            nums[slow++] = nums[fast];  
        }  
    }  
    return slow;  
}
```

SECTION H: PARTITIONING ARRAY (DUTCH NATIONAL FLAG)

Three-way partitioning:

- < pivot
- = pivot
- pivot

Useful in quicksort.

```
void partition(int[] arr, int pivot) {
```

```
int low = 0, mid = 0, high = arr.length - 1;

while (mid <= high) {
    if (arr[mid] < pivot) swap(arr, low++, mid++);
    else if (arr[mid] > pivot) swap(arr, mid, high--);
    else mid++;
}
}
```

SECTION I: PATTERN RECOGNITION

Use two-pointers when:

1. Array or string is sorted
 2. You need pair/triplet/sum relationships
 3. You are comparing left and right ends
 4. You need to partition
 5. You are merging
 6. You are checking palindrome or symmetric property
-

SECTION J: COMMON MISTAKES

1. Forgetting to sort when required
 2. Using two-pointers in unsorted cases where it doesn't apply
 3. Missing duplicate skips in 3-sum
 4. Incorrect movement of left/right pointers
 5. Using sliding window logic incorrectly for two-pointer problems
-

SECTION K: INTERVIEW INSIGHTS

1. Interviewers expect linear-time solutions

2. Often hidden hint:
“sorted array”, “continuous subarray”, “pair sum”
3. Common questions:
 - 2-sum (sorted)
 - 3-sum
 - Container with most water
 - Remove duplicates
 - Middle of linked list
 - Cycle detection
4. Always talk through pointer movement logic
5. Always check conditions before pointer moves
6. Dry-run with small test case

SECTION A: WHAT IS A TRIE?

A **Trie** (pronounced “try”) is a tree-like data structure used to store strings for efficient:

1. Prefix searching
2. Auto-completion
3. Spell checking
4. Word dictionary queries
5. Searching large text collections

Data Structure Definition

Each node represents a **character**, and the path from the root to a node represents a **prefix** of some stored word.

Each node contains:

1. Links to next characters
2. A boolean flag `isEndOfWord` to mark the end of a valid word

ASCII Diagram of a Trie

Storing words: "cat", "car", "bat"

```

      (root)
     /   \
    c     b
   / \   \
  a  a   a
 |  |   |
t  r   t

```

SECTION B: TRIE NODE STRUCTURE

Typical implementation:

```

class TrieNode {
    TrieNode[] children = new TrieNode[26];
    boolean isEnd;
}

```

If lowercase English letters.

For Unicode or general-purpose: use HashMap instead of array.

SECTION C: BASIC OPERATIONS

Operations supported:

1. Insert word
2. Search word
3. Check if prefix exists (startsWith)
4. Delete word (optional)
5. Count words with a given prefix (advanced)
6. List all words (DFS)

Time complexity:

All operations run in **O(L)** where L = length of the word/prefix.

Trie performance doesn't depend on number of stored words directly.

SECTION D: INSERT OPERATION

Java Implementation

```
class Trie {  
    TrieNode root = new TrieNode();  
  
    void insert(String word) {  
        TrieNode curr = root;  
  
        for (char c : word.toCharArray()) {  
            int index = c - 'a';  
  
            if (curr.children[index] == null)  
                curr.children[index] = new TrieNode();  
  
            curr = curr.children[index];  
        }  
        curr.isEnd = true;  
    }  
}
```

SECTION E: SEARCH OPERATION

```
boolean search(String word) {  
    TrieNode curr = root;  
  
    for (char c : word.toCharArray()) {  
        int index = c - 'a';
```

```
        if (curr.children[index] == null)
            return false;

        curr = curr.children[index];
    }
    return curr.isEnd;
}
```

SECTION F: STARTSWITH / PREFIX CHECK

```
boolean startsWith(String prefix) {
    TrieNode curr = root;

    for (char c : prefix.toCharArray()) {
        int index = c - 'a';

        if (curr.children[index] == null)
            return false;

        curr = curr.children[index];
    }
    return true;
}
```

SECTION G: AUTOCOMPLETE FEATURE (List All Words With Prefix)

Steps:

1. Traverse to prefix node
2. DFS from that node to collect all words

Java Implementation

```
List<String> autoComplete(String prefix) {  
    TrieNode curr = root;  
  
    for (char c : prefix.toCharArray()) {  
        int i = c - 'a';  
  
        if (curr.children[i] == null)  
            return new ArrayList<>();  
  
        curr = curr.children[i];  
    }  
  
    List<String> results = new ArrayList<>();  
    dfs(prefix, curr, results);  
    return results;  
}  
  
void dfs(String prefix, TrieNode node, List<String> results) {  
    if (node.isEnd) results.add(prefix);  
  
    for (int i = 0; i < 26; i++) {  
        if (node.children[i] != null) {  
            dfs(prefix + (char)(i + 'a'), node.children[i], results);  
        }  
    }  
}
```

SECTION H: WILDCARD SEARCH ('.' matches any character)

Essential for problems like:

- Word Dictionary with '.'
- LeetCode 211: Add and Search Word

Java Implementation

```
boolean searchWithDot(String word, TrieNode node, int index) {  
    if (index == word.length())  
        return node.isEnd;  
  
    char c = word.charAt(index);  
  
    if (c == '.') {  
        for (TrieNode child : node.children) {  
            if (child != null && searchWithDot(word, child, index + 1))  
                return true;  
        }  
        return false;  
    } else {  
        int i = c - 'a';  
        return node.children[i] != null &&  
            searchWithDot(word, node.children[i], index + 1);  
    }  
}
```

SECTION I: DELETING A WORD FROM TRIE

Requires post-order deletion.

```
boolean delete(TrieNode node, String word, int index) {

    if (index == word.length()) {
        if (!node.isEnd) return false;
        node.isEnd = false;

        return noChildren(node);
    }

    int i = word.charAt(index) - 'a';

    if (node.children[i] == null) return false;

    boolean shouldDelete = delete(node.children[i], word, index+1);

    if (shouldDelete) {
        node.children[i] = null;
        return !node.isEnd && noChildren(node);
    }
    return false;
}

boolean noChildren(TrieNode node) {
    for (TrieNode ch : node.children)
        if (ch != null) return false;
    return true;
}
```

SECTION J: TRIE COMPLEXITIES

Operation	Complexity
-----------	------------

Insert	$O(L)$
--------	--------

Search	$O(L)$
--------	--------

Prefix check	$O(L)$
--------------	--------

Delete	$O(L)$
--------	--------

Memory	$O(26 \times N \times \text{average length})$
--------	---

Tries trade **time** for **space**.

SECTION K: USE CASES OF TRIE

1. Autocomplete systems
2. Predictive text
3. Search engines
4. Spell checkers
5. Dictionary word storage
6. Prefix lookup
7. Word games (Boggle, Scrabble)
8. IP routing tables
9. Phone directory systems

SECTION L: VARIANTS OF TRIE

1. Standard Trie

Fixed alphabet.

2. Compressed Trie (Radix Tree / Patricia Trie)

Compress chains of single children.

Example:

Words: "bear", "bell"

Normal Trie:

b → e → a → r

 \ → l → l

Compressed Trie:

b → e → ["ar"]

 \ → ["ll"]

3. Suffix Trie

Stores all suffixes of a word.

Used for substring queries.

4. Suffix Tree (Advanced)

Optimal for substring problems.

5. Ternary Search Tree

Used when alphabet size is huge.

SECTION M: TRIE VS HASHMAP

Feature	Trie	HashMap
Prefix search	Excellent	Poor
Autocomplete	Yes	Hard
Memory usage	High	Lower
Search time	O(L)	O(L)
Order	Lexicographical	No natural order
Wildcard search	Efficient	Hard

SECTION N: COMMON MISTAKES

1. Using arrays for large alphabets
2. Not handling Unicode or uppercase characters
3. Forgetting to mark isEndOfWord

4. Using recursion incorrectly for wildcard search
 5. Memory blow-up due to too many nodes
 6. Trying to use trie when HashMap would be simpler
-

SECTION O: INTERVIEW INSIGHTS

1. Very popular with FAANG-type companies
2. Expect questions like:
 - Word dictionary with wildcard
 - Autocomplete system
 - Count words with prefix
 - Longest common prefix
 - Replace words in sentence
3. Interviewer checks:
 - Understanding of node structure
 - Depth-first search logic
 - Efficiency of prefix-based search
 - Trade-offs between Trie and arrays/maps

Always emphasize why Trie is needed:

"HashMap is not suitable for prefix searching, but Trie allows $O(L)$ prefix lookup."

SECTION A: WHAT IS A SEGMENT TREE?

A **Segment Tree** is a binary tree designed to answer **range queries** efficiently, such as:

1. Range Sum
2. Range Minimum / Maximum
3. Range GCD
4. Range XOR
5. Range updates
6. Lazy propagation updates

It allows:

- Query in $O(\log n)$
- Update in $O(\log n)$

This is much faster than computing results in $O(n)$ for each query.

SECTION B: WHY DO WE NEED A SEGMENT TREE?

Consider array `arr[]` with:

- Many queries like `sum(l, r)`
- Many updates `arr[i] = new value`

Brute force:

- Query: $O(n)$
- Update: $O(1)$

Segment Tree:

- Query: $O(\log n)$
- Update: $O(\log n)$

Huge improvement when queries are large.

SECTION C: SEGMENT TREE STRUCTURE

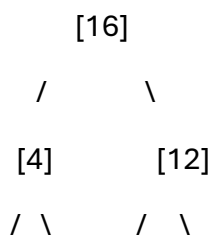
Stored as a tree where:

- Root represents the whole array
- Left child represents left half
- Right child represents right half

Example:

Array = [1, 3, 5, 7]

Tree:



[1] [3] [5] [7]

Where each node = sum of range.

SECTION D: TREE SIZE

For array of size N:

- Maximum segment tree size = $4N$

We store it in an array (like heap).

SECTION E: BUILDING A SEGMENT TREE

Time: $O(n)$

Java Implementation: Build

```
class SegmentTree {

    int[] tree;

    int n;

    SegmentTree(int[] arr) {
        n = arr.length;
        tree = new int[4 * n];
        build(arr, 0, n - 1, 1);
    }

    void build(int[] arr, int start, int end, int node) {
        if (start == end) {
            tree[node] = arr[start];
            return;
        }
    }
```



```
int mid = (start + end) / 2;

build(arr, start, mid, 2 * node);

build(arr, mid + 1, end, 2 * node + 1);

tree[node] = tree[2 * node] + tree[2 * node + 1];
}
}
```

SECTION F: RANGE QUERY (SUM)

```
int query(int start, int end, int l, int r, int node) {

    // no overlap
    if (end < l || start > r)
        return 0;

    // total overlap
    if (l <= start && end <= r)
        return tree[node];

    // partial overlap
    int mid = (start + end) / 2;

    int left = query(start, mid, l, r, 2 * node);
    int right = query(mid + 1, end, l, r, 2 * node + 1);

    return left + right;
```

```
}
```

SECTION G: POINT UPDATE

Update `arr[index] = newValue`.

```
void update(int start, int end, int index, int value, int node) {
```

```
    if (start == end) {  
        tree[node] = value;  
        return;  
    }
```

```
    int mid = (start + end) / 2;
```

```
    if (index <= mid)  
        update(start, mid, index, value, 2 * node);  
    else  
        update(mid + 1, end, index, value, 2 * node + 1);
```

```
    tree[node] = tree[2 * node] + tree[2 * node + 1];
```

```
}
```

SECTION H: LAZY PROPAGATION

(For Range Updates)

Imagine you want to do:

- Add value `X` to a whole range
OR
- Set a range to a new value

Segment Trees need Lazy Propagation to avoid $O(n)$ updates.

Concept of Lazy Propagation

Each node has a "lazy" value that is applied later.

ASCII:

Before applying lazy updates:

```

    [sum]
    lazy=0
  /   \
[sum] [sum]
lazy=5 lazy=0
```

Meaning the left subtree has an update pending.

Java Implementation: Lazy Propagation (Range Update)

Data structures:

```
int[] tree;
```

```
int[] lazy;
```

Range update:

```
void updateRange(int start, int end, int l, int r, int value, int node) {
```

```
    // pending lazy updates
```

```
    if (lazy[node] != 0) {
```

```
        tree[node] += (end - start + 1) * lazy[node];
```

```
    if (start != end) {
```

```
        lazy[2 * node] += lazy[node];
```

```
        lazy[2 * node + 1] += lazy[node];
```

```
    }
```

```

        lazy[node] = 0;
    }

    // no overlap
    if (start > r || end < l)
        return;

    // total overlap
    if (l <= start && end <= r) {
        tree[node] += (end - start + 1) * value;

        if (start != end) {
            lazy[2 * node] += value;
            lazy[2 * node + 1] += value;
        }
        return;
    }

    // partial overlap
    int mid = (start + end) / 2;

    updateRange(start, mid, l, r, value, 2 * node);
    updateRange(mid + 1, end, l, r, value, 2 * node + 1);

    tree[node] = tree[2 * node] + tree[2 * node + 1];
}

```

SECTION I: RANGE QUERY WITH LAZY

```
int queryRange(int start, int end, int l, int r, int node) {

    if (lazy[node] != 0) {
        tree[node] += (end - start + 1) * lazy[node];

        if (start != end) {
            lazy[2 * node] += lazy[node];
            lazy[2 * node + 1] += lazy[node];
        }
        lazy[node] = 0;
    }

    if (start > r || end < l)
        return 0;

    if (l <= start && end <= r)
        return tree[node];

    int mid = (start + end) / 2;

    return queryRange(start, mid, l, r, 2 * node) +
           queryRange(mid + 1, end, l, r, 2 * node + 1);
}
```

SECTION J: USE CASES OF SEGMENT TREES

1. Range Sum Query (RSQ)
2. Range Minimum Query (RMQ)

3. Range Maximum Query
 4. Range XOR
 5. Range GCD
 6. Range add updates
 7. Range set updates
 8. Histogram largest rectangle (advanced)
 9. Competitive programming (very common)
 10. Stock market range analysis
-

SECTION K: VARIANTS OF SEGMENT TREES

1. **Segment Tree with Lazy Propagation**
 2. **Persistent Segment Tree**
(Maintains history of versions)
 3. **Merge Sort Tree**
("tree of sorted lists")
 4. **Segment Tree Beats (very advanced)**
For range-min-set / range-chmax problems
-

SECTION L: COMPLEXITIES

Operation	Time	Space
-----------	------	-------

Build	$O(n)$	$O(4n)$
-------	--------	---------

Query	$O(\log n)$	$O(1)$
-------	-------------	--------

Update	$O(\log n)$	$O(1)$
--------	-------------	--------

Lazy update	$O(\log n)$	$O(1)$
-------------	-------------	--------

SECTION M: COMMON MISTAKES

1. Forgetting mid calculation
2. Mixing node indexing (1-indexed tree array)

3. Ignoring lazy updates → incorrect results
 4. Treating no-overlap cases incorrectly
 5. Forgetting to propagate lazy values
 6. Wrong merging logic for custom functions
-

SECTION N: INTERVIEW INSIGHTS

1. Segment Tree is rarely coded completely in interviews unless you are applying for high-level roles or competitive programming oriented companies.
2. Most common tasks:
 - RMQ (Range Minimum Query)
 - Range sum
 - Range updates
3. Interviewers check your understanding of:
 - Recursion
 - Tree representation
 - Efficient range calculations
 - Lazy propagation
4. If asked:

"Why use Segment Tree over Fenwick Tree?"

Answer: Segment Trees support more operations (min, max, gcd, range update) and are more flexible.

INTERVIEW TIPS

What interviewers test

1. Problem-solving
2. Clarity of thought
3. Use of optimal data structures
4. Clean code
5. Correct time/space analysis
6. Ability to optimize

Golden strategy

1. Understand the problem deeply
2. Analyze brute-force
3. Identify patterns
4. Pick correct DS/Algo
5. Write clean, structured code
6. Test with edge cases

Most commonly asked topics

1. Arrays + Two pointers
2. Strings (KMP, sliding window)
3. Trees + BST
4. Recursion + Backtracking
5. Graphs (BFS, DFS, Dijkstra)
6. DP (Knapsack, LIS, LCS)
7. HashMap logic
8. Heaps (k largest, k smallest)
9. Bit manipulation patterns
10. Segment Tree/Fenwick Tree (for advanced roles)

23. FINAL SUMMARY TABLE

Category	Most Important Algorithms / Problems
Searching	Binary search + variations
Sorting	Merge, Quick, Heap
DP	LIS, LCS, Knapsack, Grid paths
Strings	KMP, Sliding Window, Z-Algorithm
Graphs	BFS, DFS, Dijkstra, Topo sort, MST
Trees	Traversals, BST ops, LCA

Category	Most Important Algorithms / Problems
Hashing	HashMap patterns
Heaps	Priority queue problems
Sliding Window	Longest substring, window min/max
Backtracking	N-Queens, subsets, permutations
Advanced	Segment Tree, Fenwick Tree

TOP 500 MOST IMPORTANT DSA + SYSTEM QUESTIONS

SECTION 1: ARRAYS — 60 QUESTIONS

- Two Sum
- Best Time to Buy and Sell Stock
- Kadane's Algorithm (Maximum Subarray Sum)
- Move Zeroes
- Contains Duplicate
- Merge Sorted Arrays
- Rotate Array
- Plus One
- Longest Consecutive Sequence
- Product of Array Except Self
- Maximum Product Subarray
- Subarray Sum Equals K
- Minimum Size Subarray Sum
- Sliding Window Maximum
- 3Sum
- 4Sum
- Container With Most Water
- Trapping Rain Water
- Jump Game I

20. Jump Game II
21. Candy Distribution
22. Majority Element (Moore's Voting)
23. Spiral Matrix
24. Set Matrix Zeroes
25. Pascal's Triangle
26. Insert Interval
27. Merge Intervals
28. Non-overlapping Intervals
29. Summary Ranges
30. Find Minimum in Rotated Sorted Array
31. Search Rotated Sorted Array
32. Binary Search variations (lower bound, upper bound)
33. First and Last Occurrence in Sorted Array
34. Peak Element
35. Median of Two Sorted Arrays
36. Find K Closest Elements
37. Missing Number
38. Find All Numbers Disappeared
39. Count Inversions (Merge Sort)
40. Dutch National Flag (Sort Colors)
41. Wiggle Sort
42. Rearrange Array by Sign
43. Alternating Positive and Negative
44. Smallest Subarray with Sum $> X$
45. Longest Subarray with Sum Zero
46. Equilibrium Index
47. Leaders in Array

- 48. Next Greater Element
 - 49. Next Smaller Element
 - 50. Celebrity Problem
 - 51. Sliding Window: Count Distinct Elements
 - 52. Maximum Points from Cards
 - 53. Circular Subarray Sum
 - 54. Number of Subarrays with Product $< k$
 - 55. Maximum XOR of Two Numbers
 - 56. XOR Subarray Queries
 - 57. Rotate Matrix by 90°
 - 58. Flood Fill on 2D Arrays
 - 59. Positions of Large Groups
 - 60. Range Sum Query Problems
-

SECTION 2: STRINGS — 40 QUESTIONS

- 61. Valid Anagram
- 62. Valid Palindrome
- 63. Longest Palindromic Substring
- 64. Longest Palindromic Subsequence
- 65. Longest Common Prefix
- 66. Group Anagrams
- 67. Count and Say
- 68. Reverse Words in a String
- 69. Find All Anagrams in String
- 70. Minimum Window Substring
- 71. Longest Substring Without Repeating Characters
- 72. Check Isomorphic Strings
- 73. Check Subsequence

74. Implement strStr()
 75. KMP Pattern Matching
 76. Z-Algorithm Pattern Matching
 77. Rabin-Karp
 78. Word Break
 79. Word Break II
 80. Word Ladder
 81. Word Ladder II
 82. Longest Duplicate Substring
 83. Rotate String
 84. Smallest Subsequence of Distinct Characters
 85. Compare Version Numbers
 86. Reverse Only Letters
 87. Decode Ways
 88. Add Binary
 89. Multiply Strings
 90. Custom Sort String
 91. Validate IP Address
 92. Palindrome Partitioning
 93. Longest Happy Prefix
 94. Edit Distance
 95. Remove Duplicate Letters
 96. Count Palindromic Substrings
 97. String Compression
 98. Roman to Integer
 99. Integer to Roman
 100. Check if two strings are rotations
-

SECTION 3: LINKED LISTS — 35 QUESTIONS

101. Reverse Linked List
102. Middle of Linked List
103. Detect Loop (Floyd's Cycle)
104. Remove Cycle
105. Merge Two Sorted Lists
106. Merge K Sorted Lists (min-heap)
107. Add Two Numbers (Linked List)
108. Palindrome Linked List
109. Intersection of Two Linked Lists
110. Remove Nth Node from End
111. Rotate Linked List
112. Reorder Linked List
113. Partition List
114. Reverse Linked List in Groups of K
115. Linked List Cycle II (detect start point)
116. Convert Binary List to Integer
117. Swap Nodes in Pairs
118. Delete Node (without head pointer)
119. Flatten a Multilevel Linked List
120. Sort a Linked List
121. Copy List with Random Pointer
122. Insert into Sorted Circular Linked List
123. Split Circular Linked List
124. Add 1 to Linked List Number
125. Multiply Linked List numbers
126. Remove duplicates from sorted list
127. Remove duplicates from unsorted list

- 128. Segregate even/odd nodes
 - 129. Find modular node
 - 130. Reverse alternate K nodes
 - 131. Check if list is even or odd length
 - 132. Convert Sorted List to BST
 - 133. Check bipartite using adjacency list
 - 134. Kth node from end using 2 pointers
 - 135. Union & Intersection of two lists
-

SECTION 4: STACKS & QUEUES — 30 QUESTIONS

- 136. Valid Parentheses
- 137. Min Stack
- 138. Next Greater Element (Stack)
- 139. Next Smaller Element
- 140. Largest Rectangle in Histogram
- 141. Maximal Rectangle
- 142. Evaluate Reverse Polish Notation
- 143. Basic Calculator
- 144. Decode String
- 145. Implement Stack using Queues
- 146. Implement Queue using Stacks
- 147. Stack span problem
- 148. Sliding Window Maximum (Deque)
- 149. First Negative in every K window
- 150. Rotting Oranges
- 151. Circular Queue
- 152. LRU Cache
- 153. LFU Cache

- 154. Design Hit Counter
 - 155. Celebrity Problem
 - 156. Remove K digits
 - 157. Daily Temperatures
 - 158. Asteroid Collision
 - 159. Simplify Path
 - 160. Reverse Stack using Recursion
 - 161. Balanced Bracket with *, (,)
 - 162. Maximum Of Minimum for Every Window
 - 163. Queue Reconstruction by Height
 - 164. Moving Average from Data Stream
 - 165. Stack using Linked List
-

SECTION 5: TREES — 60 QUESTIONS

- 166. Inorder Traversal
- 167. Preorder Traversal
- 168. Postorder Traversal
- 169. Level Order Traversal
- 170. Zigzag Level Order
- 171. Right View of Binary Tree
- 172. Left View of Binary Tree
- 173. Top View of Binary Tree
- 174. Bottom View
- 175. Height of Tree
- 176. Diameter of Binary Tree
- 177. Balanced Binary Tree
- 178. Symmetric Tree
- 179. Path Sum

180. Path Sum II
181. Maximum Path Sum
182. Sum of Left Leaves
183. Invert Binary Tree
184. Construct Tree from Inorder & Preorder
185. Construct Tree from Inorder & Postorder
186. Build Tree from Level Order
187. Serialize and Deserialize Binary Tree
188. Lowest Common Ancestor (Binary Tree)
189. Lowest Common Ancestor (BST)
190. Convert Sorted Array to BST
191. Check if two trees are identical
192. Tree is subtree of another tree
193. Recover BST
194. Morris Traversals
195. Vertical Order Traversal
196. Boundary Traversal
197. Maximum Width of Binary Tree
198. Deepest Leaves Sum
199. Kth Smallest Element in BST
200. Validate BST
201. Insert into BST
202. Delete from BST
203. Floor and Ceil in BST
204. Inorder predecessor & successor
205. Kth largest in BST
206. Convert BST to Balanced BST
207. Count nodes of complete tree

- 208. BST Iterator
 - 209. Binary Tree Cameras
 - 210. House Robber III
 - 211. Maximum Average Subtree
 - 212. Binary Tree Pruning
 - 213. Cousins in binary tree
 - 214. Check balanced subtree
 - 215. Largest BST in binary tree
 - 216. Tree from parent array
 - 217. Count Good Nodes
 - 218. Maximum Ancestor Difference
 - 219. Sum of distances in tree
 - 220. Distance between two nodes in tree
 - 221. Red-Black Tree basic operations (concept)
 - 222. AVL Tree rotations (concept)
 - 223. Threaded binary tree (concept)
 - 224. Leaf-Similar Trees
 - 225. Tilt of Binary Tree
-

SECTION 6: GRAPHS — 70 QUESTIONS

- 226. BFS Traversal
- 227. DFS Traversal
- 228. Number of Islands
- 229. Flood Fill
- 230. Rotten Oranges
- 231. Cycle Detection (Directed)
- 232. Cycle Detection (Undirected)
- 233. Topological Sort

- 234. Kahn's Algorithm (BFS Toposort)
- 235. Check Bipartite
- 236. Course Schedule I
- 237. Course Schedule II
- 238. Clone Graph
- 239. Number of Connected Components
- 240. Word Ladder
- 241. Word Ladder II
- 242. Alien Dictionary
- 243. Snake and Ladder Problem
- 244. Knight Shortest Path
- 245. Remove Stones (Union-Find)
- 246. Redundant Connection
- 247. Redundant Connection II
- 248. Critical Connections (Bridges)
- 249. Articulation Points
- 250. SCC (Kosaraju)
- 251. Dijkstra Algorithm
- 252. Bellman-Ford
- 253. Floyd-Warshall
- 254. Prim's MST
- 255. Kruskal's MST
- 256. Union-Find by Rank + Path Compression
- 257. Find Circle Number (Union-Find)
- 258. Graph Coloring
- 259. Euler Path & Circuit
- 260. Hamiltonian Path (backtracking)
- 261. Bipartite Check using DFS

- 262. Shortest Path in DAG
 - 263. Letter Combinations (graph based)
 - 264. Walls and Gates
 - 265. Pacific Atlantic Water Flow
 - 266. Swim in Rising Water
 - 267. Trapping Rain Water II (heap + graph)
 - 268. Count unreachable pairs
 - 269. Restore array from adjacent pairs
 - 270. Graph Valid Tree
 - 271. Path with minimum effort
 - 272. Minimum cost to connect points
 - 273. Detect cycle using DSU
 - 274. Find eventual safe states
 - 275. Minimum height trees
 - 276. Reconstruct itinerary
 - 277. Town judge problem
 - 278. Network delay time
 - 279. Cheapest flights with K stops
 - 280. Shortest path visiting all nodes (bitmask BFS)
 - 281. Maze solver
 - 282. Is graph tree or not
-

SECTION 7: DYNAMIC PROGRAMMING — 70 QUESTIONS

- 283. Fibonacci (DP)
- 284. Climbing Stairs
- 285. House Robber
- 286. House Robber II
- 287. Decode Ways

- 288. Unique Paths
- 289. Unique Paths II
- 290. Min Cost Climbing Stairs
- 291. Minimum Path Sum
- 292. Coin Change
- 293. Coin Change 2
- 294. 0/1 Knapsack
- 295. Unbounded Knapsack
- 296. Rod Cutting
- 297. Target Sum
- 298. Partition Equal Subset Sum
- 299. Subset Sum
- 300. Combination Sum
- 301. Combination Sum II
- 302. Longest Increasing Subsequence
- 303. LIS Using Binary Search
- 304. Longest Common Subsequence
- 305. Longest Common Substring
- 306. Edit Distance
- 307. Shortest Common Supersequence
- 308. Palindromic Substrings
- 309. Longest Palindromic Subsequence
- 310. Palindrome Partitioning (DP)
- 311. Word Break
- 312. Word Break II
- 313. Burst Balloons
- 314. Matrix Chain Multiplication
- 315. Boolean Parenthesization

- 316. Egg Dropping
- 317. Super Egg Drop
- 318. Cherry Pickup
- 319. Minimum Falling Path
- 320. Maximum Product Subarray
- 321. Best Time to Buy and Sell Stock I
- 322. Buy and Sell Stock II
- 323. Buy and Sell Stock with Cooldown
- 324. Buy and Sell Stock with Transaction Fee
- 325. Buy and Sell Stock III
- 326. Buy and Sell Stock IV
- 327. Count ways to reach nth stair with jumps
- 328. DP on trees: House Robber III
- 329. DP on grids: obstacle grid
- 330. N-Queens (backtracking + DP)
- 331. TSP (bitmask DP)
- 332. DP on XOR subarrays
- 333. DP on subsets
- 334. Digit DP basics
- 335. Distinct Subsequences
- 336. Domino and tromino tiling
- 337. Interleaving string
- 338. LPS using DP
- 339. Maximum sum increasing subsequence
- 340. Number of longest LIS
- 341. Arithmetic slices
- 342. K Inverse Pairs
- 343. Number of dice rolls with target sum

- 344. Scramble string
 - 345. Count palindromic subsequences
 - 346. Minimum removals to make mountain array
 - 347. Maximal square
-

SECTION 8: GREEDY ALGORITHMS — 25 QUESTIONS

- 348. Activity Selection
- 349. Fractional Knapsack
- 350. Job Sequencing
- 351. Gas Station
- 352. Jump Game
- 353. Minimum Platforms
- 354. Maximum Meetings
- 355. Largest Number
- 356. Lemonade Change
- 357. Candy Distribution
- 358. Assign Cookies
- 359. Boats to Save People
- 360. Partition Labels
- 361. Reorganize String
- 362. Minimum Arrows to Burst Balloons
- 363. Task Scheduler
- 364. Queue Reconstruction
- 365. Buy and Sell Stock (Greedy)
- 366. Minimum Time to Make Ropes Colorful
- 367. Monotone Increasing Digits
- 368. Remove K Digits
- 369. Shortest Job First scheduling

- 370. Minimum absolute difference pairs
 - 371. Car pooling
-

SECTION 9: BACKTRACKING — 25 QUESTIONS

- 372. Permutations
 - 373. Permutations II
 - 374. Subsets
 - 375. Subsets II
 - 376. Combination Sum
 - 377. Combination Sum II
 - 378. Generate Parentheses
 - 379. Letter Combinations of Phone Number
 - 380. N-Queens
 - 381. Sudoku Solver
 - 382. Rat in a Maze
 - 383. Word Search
 - 384. Palindrome Partitioning
 - 385. Restore IP Addresses
 - 386. All Paths in DAG
 - 387. Combination sum with duplicates
 - 388. Balanced parentheses variations
 - 389. Tug of war problem
-

SECTION 10: BIT MANIPULATION — 20 QUESTIONS

- 390. Single Number
- 391. Single Number II
- 392. Single Number III
- 393. Counting Bits

- 394. Bitwise AND of range
 - 395. Reverse Bits
 - 396. Power of Two
 - 397. Hamming Weight
 - 398. Hamming Distance
 - 399. XOR subarray queries
 - 400. Missing Number
 - 401. Subsets using bits
 - 402. Maximum XOR Pair
 - 403. Binary watch
 - 404. Total set bits in 1 to n
 - 405. Convert integer to binary
 - 406. Next power of two
 - 407. Gray code
 - 408. Decode XORed permutation
-

SECTION 11: HEAPS & PRIORITY QUEUES — 25 QUESTIONS

- 409. Kth Largest Element
- 410. Kth Smallest Element
- 411. Merge K Sorted Lists
- 412. Merge K Sorted Arrays
- 413. Top K Frequent Elements
- 414. Find Median from Data Stream
- 415. Sliding Window Maximum
- 416. Smallest Range in K lists
- 417. Frequency Sort
- 418. Min cost to connect ropes
- 419. Reorganize String

- 420. WWE Kth largest stream
 - 421. Ugly Numbers
 - 422. Nearly Sorted Array
 - 423. Running median
 - 424. KLargest elements
 - 425. K Closest Points to Origin
 - 426. K closest numbers
 - 427. Find GCD pairs using heap
 - 428. Hands of Straights
 - 429. Cheapest flights graph using min-heap
 - 430. Huffman Encoding (concept)
-

SECTION 12: TRIE — 20 QUESTIONS

- 431. Implement Trie
- 432. Trie Insert / Search / Delete
- 433. Word Break (Trie)
- 434. Maximum XOR With Element
- 435. Replace Words
- 436. Design Add and Search Words
- 437. Palindrome Pairs using Trie
- 438. Autocomplete System
- 439. Longest Word in Dictionary
- 440. Sum of Prefix Scores
- 441. Prefix and Suffix Search
- 442. Search suggestions system
- 443. Count distinct substrings using Trie
- 444. Trie for pattern matching
- 445. Dictionary serialization

- 446. Magic Dictionary
 - 447. Folder system using Trie
 - 448. Build Suffix Trie (concept)
 - 449. Wildcard pattern using Trie
 - 450. Longest common prefix using Trie
-

SECTION 13: ADVANCED STRUCTURES — 30 QUESTIONS

- 451. Segment Tree: Range Sum
- 452. Segment Tree: Range Minimum
- 453. Segment Tree Lazy Propagation
- 454. Fenwick Tree Range Queries
- 455. Mo's Algorithm
- 456. Sqrt Decomposition
- 457. Disjoint Set Union (Union-Find)
- 458. LRU Cache
- 459. LFU Cache
- 460. Bloom Filter (concept)
- 461. Skip List (concept)
- 462. Heavy Light Decomposition
- 463. Euler Tour Technique
- 464. Sparse Table for RMQ
- 465. K-D Tree (concept)
- 466. Red-Black Tree (concept)
- 467. AVL Rotations
- 468. HashMap Internal Working
- 469. Consistent Hashing (system design)
- 470. Memory Layout of Objects
- 471. KMP Automaton

- 472. Suffix Array
 - 473. Suffix Tree (concept)
 - 474. Aho-Corasick
 - 475. Palindromic Tree (EERTREE)
 - 476. Rolling Hash collision handling
 - 477. RMQ using Sparse Table
 - 478. Persistent Segment Tree
 - 479. Treaps
 - 480. Zero-One BFS
-

SECTION 14: SYSTEM DESIGN + OOPS (INTERVIEW ESSENTIAL) — 20 QUESTIONS

- 481. Design URL Shortener
- 482. Design LRU Cache
- 483. Design Autocomplete System
- 484. Design Online Judge
- 485. Design News Feed
- 486. Design File System
- 487. Design Parking Lot
- 488. Design Elevator System
- 489. Design Logging Rate Limiter
- 490. Design WhatsApp “typing” feature
- 491. Design Chat Application
- 492. Design Memory Allocator
- 493. Design Google Docs collaboration
- 494. Design Rate Limiter (Token Bucket)
- 495. Design Distributed Cache
- 496. Design Distributed Counter
- 497. Design API Throttling system

- 498. Design YouTube MVP
- 499. Design Twitter Threads
- 500. Design Distributed Queue

All The Best!